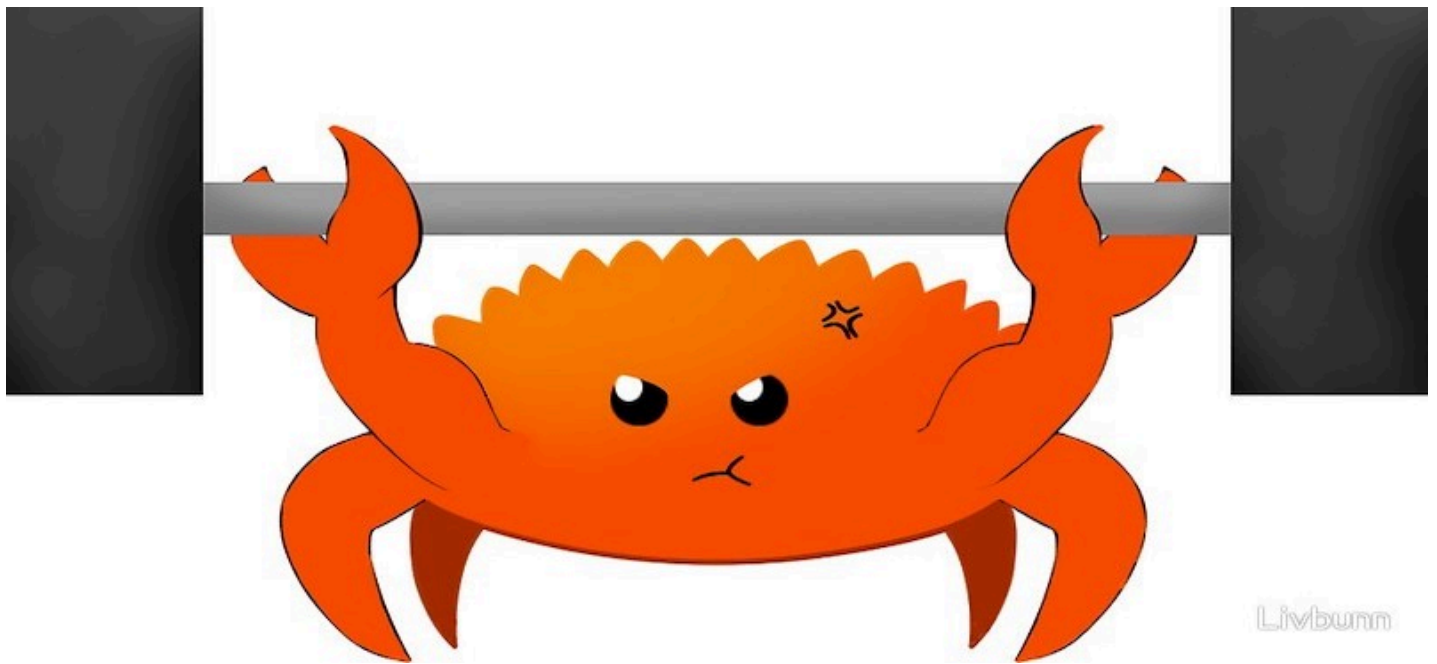




Practice Rust with challenging examples, exercises and projects

stars 14k forks 1.2k license mit

This book was designed for easily diving into and getting skilled with Rust, and it's very easy to use: All you need to do is to make each exercise compile without ERRORS and Panics !

Reading online

- [English](#)
- [🇧🇪](#)

Running locally

We use [mdbook](#) building our exercises. You can run locally with below steps:

```
$ git clone https://github.com/sunface/rust-by-practice
$ cargo install mdbook
$ cd rust-by-practice && mdbook serve en/
```

Features

Part of our examples and exercises are borrowed from [Rust By Example](#), thanks for your great works!

Although they are so awesome, we have our own secret weapons :)

- There are three parts in each chapter: examples, exercises and practices
- Besides examples, we have a lot of exercises, you can Read, Edit and Run them ONLINE
- Covering nearly all aspects of Rust, such as async/await, threads, sync primitives, optimizing, standard libraries, tool chain, data structures and algorithms etc.
- Every exercise has its own solutions
- The overall difficulties are a bit higher and from easy to super hard: easy 🌟 medium 🌟 🌟 hard 🌟🌟🌟 super hard 🌟🌟🌟🌟

What we want to do is to fill the gap between learning and getting started with real projects!

Small projects with Elegant code base

Following questions come up weekly in online Rust discussions:

- I just finished reading The Book, what should I do next ?
- What projects would you recommend to a Rust beginner?
- Looking for small projects with an elegant code base
- Codes that is easy to read and learn

The answers to these questions are always **Practice**: doing some exercises, and then reading some small and excellent Rust projects.

This is precisely the goal of this book, so, collecting relative resources and representing in *Rust By Practice* seems not a bad idea.

1. Ripgrep

Answers for above questions usually came with [ripgrep](#), though I don't think it is a **small** project, but yes, go for it if you are not afraid to delve deep a bit.

2. Building a text editor

Tutorial <https://www.flenker.blog/hecto/> will lead you to build a text editor from scratch.

3. Ncspot

[Ncspot](#), a terminal Spotify client. Small, simple, well organized and async, it's good for learning.

4. Command Line Rust

[This project](#) is for the book *Command-Line Rust* (O'Reilly), it will show you how to write small CLIs (clones of `head`, `cat`, `ls`).

5. pngme book

[This book](#) will guide you to make a command line program that lets you hide secret messages in PNG files. The primary goal here is to get you writing code. The secondary goal is to get you reading documentation.

6. Writing an OS in Rust

[This blog series](#) creates a small operating system in the Rust programming language. Each post is a small tutorial and includes all needed code, so you can follow along if you like. The source code is also available in the corresponding [Github repository](#).

7. CodeCrafters.io: Build your own Git, Docker, SQLite, or Redis

On [CodeCrafters](#), you can recreate your favorite developer tools from scratch. It's a hands-on, minimally-guided approach to master Rust, while appreciating the internals and documentation of popular technology that we use every day.

8. mini-redis

[mini-redis](#) is an incomplete Redis client and server implementation using tokio, it has decent code base and detail explanations, very suitable for learning Rust and asynchronous programming.

9. Writing Interpreters in Rust

[This online book](#) will walk through the basics of interpreted language implementation in Rust with a focus on the challenges that are specific to using Rust.

To be continued...

Variables

Binding and mutability

1. 🌟 A variable can be used only if it has been initialized.

```
|  
// Fix the error below with least amount of modification to the code  
fn main() {  
    let x: i32; // Uninitialized but used, ERROR !  
    let y: i32; // Uninitialized but also unused, only a Warning !  
  
    assert_eq!(x, 5);  
    println!("Success!");  
}
```

2. 🌟 Use `mut` to mark a variable as mutable.

```
|  
// Fill the blanks in the code to make it compile  
fn main() {  
    let __ __ = 1;  
    __ += 2;  
  
    assert_eq!(x, 3);  
    println!("Success!");  
}
```

Scope

A scope is the range within the program for which the item is valid.

3. 🌟

```
|  
// Fix the error below with least amount of modification  
fn main() {  
    let x: i32 = 10;  
    {  
        let y: i32 = 5;  
        println!("Inner scope value of x is {} and value of y is {}", x, y);  
    }  
    println!("Outer scope value of x is {} and value of y is {}", x, y);  
}
```

4. 🌟🌟

```
|
// Fix the error with the use of define_x
fn main() {
    println!("{}", world, x);
}

fn define_x() {
    let x = "hello";
}
```

Shadowing

You can declare a new variable with the same name as a previous variable, here we can say **the first one is shadowed by the second one**.

5. 🌟🌟

```
|
// Only modify `assert_eq!` to make the `println!` work(print `42` in terminal)
fn main() {
    let x: i32 = 5;
    {
        let x = 12;
        assert_eq!(x, 5);
    }

    assert_eq!(x, 12);

    let x = 42;
    println!("{}", x); // Prints "42".
}
```

6. 🌟🌟

```
|
// Remove a line in the code to make it compile
fn main() {
    let mut x: i32 = 1;
    x = 7;
    // Shadowing and re-binding
    let x = x;
    x += 3;

    let y = 4;
    // Shadowing
    let y = "I can also be bound to text!";

    println!("Success!");
}
```

Unused variables

7. Fix the warning below with :

- 🌟 Only one solution
- 🌟🌟 Two distinct solutions

Note: none of the solutions is to remove the line `let x = 1`

```
|  
fn main() {  
    let x = 1;  
}  
  
// Warning: unused variable: `x`
```

Destructuring

8. 🌟🌟 We can use a pattern with `let` to destructure a tuple to separate variables.

Tips: you can use Shadowing or Mutability

```
|  
// Fix the error below with least amount of modification  
fn main() {  
    let (x, y) = (1, 2);  
    x += 2;  
  
    assert_eq!(x, 3);  
    assert_eq!(y, 2);  
  
    println!("Success!");  
}
```

Destructuring assignments

Introduced in Rust 1.59: You can now use tuple, slice, and struct patterns as the left-hand side of an assignment.

9. 🌟🌟

Note: the feature `Destructuring assignments` need 1.59 or higher Rust version

```
|  
fn main() {  
    let (x, y);  
    (x,..) = (3, 4);  
    [.., y] = [1, 2];  
    // Fill the blank to make the code work  
    assert_eq!([x,y], __);  
  
    println!("Success!");  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Basic Types

Learning resources:

- English: [Rust Book 3.2 and 3.3](#)
- 简体中文: [Rust 3.2 - 3.3](#)

Numbers

Integer

1. 🌟

Tips: If we don't explicitly assign a type to a variable, then the compiler will infer one for us.

```
|  
// Remove something to make it work  
fn main() {  
    let x: i32 = 5;  
    let mut y: u32 = 5;  
  
    y = x;  
  
    let z = 10; // Type of z ?  
  
    println!("Success!");  
}
```

2. 🌟

```
|  
// Fill the blank  
fn main() {  
    let v: u16 = 38_u8 as __;  
  
    println!("Success!");  
}
```

3. 🌟🌟🌟

Tips: If we don't explicitly assign a type to a variable, then the compiler will infer one for us.

```

// Modify `assert_eq!` to make it work
fn main() {
    let x = 5;
    assert_eq!("u32".to_string(), type_of(&x));

    println!("Success!");
}

// Get the type of given variable, return a string representation of the type , e.g
fn type_of<T>(_: &T) -> String {
    format!("{}", std::any::type_name::<T>())
}

```

4. 🌟🌟

```

// Fill the blanks to make it work
fn main() {
    assert_eq!(i8::MAX, __);
    assert_eq!(u8::MAX, __);

    println!("Success!");
}

```

5. 🌟🌟

```

// Fix errors and panics to make it work
fn main() {
    let v1 = 251_u8 + 8;
    let v2 = i8::checked_add(251, 8).unwrap();
    println!("{}", v1, v2);
}

```

6. 🌟🌟

```

// Modify `assert!` to make it work
fn main() {
    let v = 1_024 + 0xff + 0o77 + 0b1111_1111;
    assert!(v == 1579);

    println!("Success!");
}

```

Floating-Point

7. 🌟

```

|
// Fill the blank to make it work
fn main() {
    let x = 1_000.000_1; // ?
    let y: f32 = 0.12; // f32
    let z = 0.01_f64; // f64

    assert_eq!(type_of(&x), "__".to_string());
    println!("Success!");
}

fn type_of<T>(_: &T) -> String {
    format!("{}", std::any::type_name::<T>())
}

```

8. 🌟🌟 Make it work in two distinct ways

```

|
fn main() {
    assert!(0.1+0.2==0.3);

    println!("Success!");
}

```

Range

9. 🌟🌟 Two goals: 1. Modify `assert!` to make it work 2. Make `println!` output list of numbers between 97 and 122

```

|fn main() {
    let mut sum = 0;
    for i in -3..2 {
        sum += i
    }

    assert!(sum == -3);

    for c in 'a'..'z' {
        println!("{}",c);
    }
}

```

10. 🌟🌟

```
|
// Fill the blanks
use std::ops::{Range, RangeInclusive};
fn main() {
    assert_eq!((1..__), Range{ start: 1, end: 5 });
    assert_eq!((1..__), RangeInclusive::new(1, 5));

    println!("Success!");
}
```

Computations

11. 🌟

```
|
// Fill the blanks and fix the errors
fn main() {
    // Integer addition
    assert!(1u32 + 2 == __);

    // Integer subtraction
    assert!(1i32 - 2 == __);
    assert!(1u8 - 2 == -1);

    assert!(3 * 50 == __);

    assert!(9.6 / 3.2 == 3.0); // error ! make it work

    assert!(24 % 5 == __);
    // Short-circuiting boolean logic
    assert!(true && false == __);
    assert!(true || false == __);
    assert!(!true == __);

    // Bitwise operations
    println!("0011 AND 0101 is {:04b}", 0b0011u32 & 0b0101);
    println!("0011 OR 0101 is {:04b}", 0b0011u32 | 0b0101);
    println!("0011 XOR 0101 is {:04b}", 0b0011u32 ^ 0b0101);
    println!("1 << 5 is {}", 1u32 << 5);
    println!("0x80 >> 2 is 0x{:x}", 0x80u32 >> 2);
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Char, Bool and Unit

Char

1. 🌟

```
|  
// Make it work  
use std::mem::size_of_val;  
fn main() {  
    let c1 = 'a';  
    assert_eq!(size_of_val(&c1), 1);  
  
    let c2 = ' ';  
    assert_eq!(size_of_val(&c2), 3);  
  
    println!("Success!");  
}
```

2. 🌟

```
|  
// Make it work  
fn main() {  
    let c1 = " ";  
    print_char(c1);  
}  
  
fn print_char(c : char) {  
    println!("{}", c);  
}
```

Bool

3. 🌟

```
|  
// Make println! work  
fn main() {  
    let _f: bool = false;  
  
    let t = true;  
    if !t {  
        println!("Success!");  
    }  
}
```

4. 🌟

```
|  
// Make it work  
fn main() {  
    let f = true;  
    let t = true && false;  
    assert_eq!(t, f);  
  
    println!("Success!");  
}
```

Unit type

5. 🌟🌟

```
|  
// Make it work, don't modify `implicitly_ret_unit` !  
fn main() {  
    let _v: () = ();  
  
    let v = (2, 3);  
    assert_eq!(v, implicitly_ret_unit());  
  
    println!("Success!");  
}  
  
fn implicitly_ret_unit() {  
    println!("I will return a ()");  
}  
  
// Don't use this one  
fn explicitly_ret_unit() -> () {  
    println!("I will return a ()");  
}
```

6. 🌟🌟 What's the size of the unit type?

```
|  
// Modify `4` in assert to make it work  
use std::mem::size_of_val;  
fn main() {  
    let unit: () = ();  
    assert!(size_of_val(&unit) == 4);  
  
    println!("Success!");  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Statements and Expressions

Examples

```
fn main() {  
    let x = 5u32;  
  
    let y = {  
        let x_squared = x * x;  
        let x_cube = x_squared * x;  
  
        // This expression will be assigned to `y`  
        x_cube + x_squared + x  
    };  
  
    let z = {  
        // The semicolon suppresses this expression and `()` is assigned to `z`  
        2 * x;  
    };  
  
    println!("x is {:?}", x);  
    println!("y is {:?}", y);  
    println!("z is {:?}", z);  
}
```

Exercises

1. 🌟🌟

```
// Make it work with two ways  
fn main() {  
    let v = {  
        let mut x = 1;  
        x += 2  
    };  
  
    assert_eq!(v, 3);  
  
    println!("Success!");  
}
```

2. 🌟


```
|  
fn main() {  
    let v = (let x = 3);  
  
    assert!(v == 3);  
  
    println!("Success!");  
}
```

3. 🌟

```
|  
fn main() {  
    let s = sum(1, 2);  
    assert_eq!(s, 3);  
  
    println!("Success!");  
}  
  
fn sum(x: i32, y: i32) -> i32 {  
    x + y  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Functions

1. 🌟🌟🌟

```
|  
fn main() {  
    // Don't modify the following two lines!  
    let (x, y) = (1, 2);  
    let s = sum(x, y);  
  
    assert_eq!(s, 3);  
  
    println!("Success!");  
}  
  
fn sum(x, y: i32) {  
    x + y;  
}
```

2. 🌟

```
|fn main() {  
    print();  
}  
  
// Replace i32 with another type  
fn print() -> i32 {  
    println!("Success!");  
}
```

3. 🌟🌟🌟

```
|// Solve it in two ways  
// DON'T let `println!` work  
fn main() {  
    never_return();  
  
    println!("Failed!");  
}  
  
fn never_return() -> ! {  
    // Implement this function, don't modify the fn signatures  
}
```

Diverging functions

Diverging functions never return to the caller, so they may be used in places where a value of any type is expected.

4. 🌟🌟🌟

```
|
fn main() {
    println!("Success!");
}

fn get_option(tp: u8) -> Option<i32> {
    match tp {
        1 => {
            // TODO
        }
        _ => {
            // TODO
        }
    }
};

// Rather than returning a None, we use a diverging function instead
never_return_fn()
}

// IMPLEMENT this function in THREE ways
fn never_return_fn() -> ! {
}
}
```

5. 🌟🌟🌟

```
|
fn main() {
    // FILL in the blank
    let b = __;

    let _v = match b {
        true => 1,
        // Diverging functions can also be used in match expression to replace a val
        false => {
            println!("Success!");
            panic!("we have no value for `false`, but we can panic");
        }
    };

    println!("Exercise Failed if printing out this line!");
}

```



You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Ownership and Borrowing

Learning resources:

- English: [Rust Book 4.1-4.4](#)
- 繁體中文: [Rust 繁體中文 - 學習資源](#)

Ownership

1. 🌟🌟

```
|  
fn main() {  
    // Use as many approaches as you can to make it work  
    let x = String::from("Hello world");  
    let y = x;  
    println!("{}", x, y);  
}
```

2. 🌟🌟

```
// Don't modify code in main!  
fn main() {  
    let s1 = String::from("Hello world");  
    let s2 = take_ownership(s1);  
  
    println!("{}", s2);  
}  
  
// Only modify the code below!  
fn take_ownership(s: String) {  
    println!("{}", s);  
}
```

3. 🌟🌟

```
|  
fn main() {  
    let s = give_ownership();  
    println!("{}", s);  
}  
  
// Only modify the code below!  
fn give_ownership() -> String {  
    let s = String::from("Hello world");  
    // Convert String to Vec  
    let _s = s.into_bytes();  
    s  
}
```

4. 🌟🌟

// Fix the error without removing any code

```
fn main() {  
    let s = String::from("Hello World");  
  
    print_str(s);  
  
    println!("{}", s);  
}  
  
fn print_str(s: String) {  
    println!("{}",s)  
}
```

5. 🌟🌟

// Don't use clone ,use copy instead

```
fn main() {  
    let x = (1, 2, (), "hello".to_string());  
    let y = x.clone();  
    println!("{:?}", {:?}", x, y);  
}
```

Mutability

Mutability can be changed when ownership is transferred.

6. 🌟

// make the necessary variable mutable

```
fn main() {  
    let s = String::from("Hello ");  
  
    let s1 = s;  
  
    s1.push_str("World!");  
  
    println!("Success!");  
}
```

7. 🌟🌟🌟

```

fn main() {
    let x = Box::new(5);

    let ...    // update this line, don't change other lines!

    *y = 4;

    assert_eq!(*x, 5);

    println!("Success!");
}

```

Partial move

Within the destructuring of a single variable, both by-move and by-reference pattern bindings can be used at the same time. Doing this will result in a partial move of the variable, which means that parts of the variable will be moved while other parts stay. In such a case, the parent variable cannot be used afterwards as a whole, however the parts that are only referenced (and not moved) can still be used.

Example

```

fn main() {
    #[derive(Debug)]
    struct Person {
        name: String,
        age: Box<u8>,
    }

    let person = Person {
        name: String::from("Alice"),
        age: Box::new(20),
    };

    // `name` is moved out of person, but `age` is referenced
    let Person { name, ref age } = person;

    println!("The person's age is {}", age);

    println!("The person's name is {}", name);

    // Error! borrow of partially moved value: `person` partial move occurs
    //println!("The person struct is {:?}", person);

    // `person` cannot be used but `person.age` can be used as it is not moved
    println!("The person's age from person struct is {}", person.age);
}

```

Exercises

8. 🌟

```
|
fn main() {
    let t = (String::from("hello"), String::from("world"));

    let _s = t.0;

    // Modify this line only, don't use `_s`
    println!("{:?}", t);
}
```

9. 🌟🌟

```
|
fn main() {
    let t = (String::from("hello"), String::from("world"));

    // Fill the blanks
    let (_, _) = __;

    println!("{:?}", {:?}, {:?}, s1, s2, t); // -> "hello", "world", ("hello", "worl
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Reference and Borrowing

Reference

1. 🌟

```
|  
fn main() {  
    let x = 5;  
    // Fill the blank  
    let p = __;  
  
    println!("the memory address of x is {:p}", p); // One possible output: 0x16fa3ac  
}
```

2. 🌟

```
|  
fn main() {  
    let x = 5;  
    let y = &x;  
  
    // Modify this line only  
    assert_eq!(5, y);  
  
    println!("Success!");  
}
```

3. 🌟

```
|  
// Fix error  
fn main() {  
    let mut s = String::from("hello, ");  
  
    borrow_object(s);  
  
    println!("Success!");  
}  
  
fn borrow_object(s: &String) {}
```

4. 🌟

```
|
// Fix error
fn main() {
    let mut s = String::from("hello, ");

    push_str(s);

    println!("Success!");
}

fn push_str(s: &mut String) {
    s.push_str("world")
}
```

5. 🌟🌟

```
|
fn main() {
    let mut s = String::from("hello, ");

    // Fill the blank to make it work
    let p = __;

    p.push_str("world");

    println!("Success!");
}
```

Ref

ref can be used to take references to a value, similar to &.

6. 🌟🌟🌟

```

fn main() {
    let c = ' ';

    let r1 = &c;
    // Fill the blank. dont change other code
    let __ r2 = c;

    assert_eq!(*r1, *r2);

    // Check the equality of the two address strings
    assert_eq!(get_addr(r1), get_addr(r2));

    println!("Success!");
}

// Get memory address string
fn get_addr(r: &char) -> String {
    format!("{:p}", r)
}

```

Borrowing rules

7. 🌟

```

// Remove something to make it work
// Don't remove a whole line !
fn main() {
    let mut s = String::from("hello");

    let r1 = &mut s;
    let r2 = &mut s;

    println!("{}", {}, r1, r2);

    println!("Success!");
}

```

Mutability

8. 🌟 Error: Borrow an immutable object as mutable

```
|
fn main() {
    // Fix error by modifying this line
    let s = String::from("hello, ");

    borrow_object(&mut s);

    println!("Success!");
}
```

```
fn borrow_object(s: &mut String) {}
```

9. 🌟🌟 Ok: Borrow a mutable object as immutable

```
|
// This code has no errors!
fn main() {
    let mut s = String::from("hello, ");

    borrow_object(&s);

    s.push_str("world");

    println!("Success!");
}

fn borrow_object(s: &String) {}
```

NLL

10. 🌟🌟

```
|
// Comment one line to make it work
fn main() {
    let mut s = String::from("hello, ");

    let r1 = &mut s;
    r1.push_str("world");
    let r2 = &mut s;
    r2.push_str("!");

    println!("{}", r1);
}
```

11. 🌟🌟

```
fn main() {  
    let mut s = String::from("hello, ");  
  
    let r1 = &mut s;  
    let r2 = &mut s;  
  
    // Add one line below to make a compiler error: cannot borrow `s` as mutable mor  
    // You can't use r1 and r2 at the same time  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Compound Types

Learning resources:

- English: [Rust Book 4.3, 5.1, 6.1, 8.2](#)
- 繁體中文: [Rust 繁體中文 - 學習](#)

String

The type of string literal "hello, world" is `&str`, e.g `let s: &str = "hello, world"`.

Str and &str

1. 🌟 We can't use `str` type in normal ways, but we can use `&str`.

```
|  
// Fix error without adding new line  
fn main() {  
    let s: str = "hello, world";  
  
    println!("Success!");  
}
```

2. 🌟🌟 We can only use `str` by boxing it, & can be used to convert `Box<str>` to `&str`

```
|  
// Fix the error with at least two solutions  
fn main() {  
    let s: Box<str> = "hello, world".into();  
    greetings(s)  
}  
  
fn greetings(s: &str) {  
    println!("{}",s)  
}
```

String

`String` type is defined in `std` and stored as a vector of bytes (`Vec`), but guaranteed to always be a valid UTF-8 sequence. `String` is heap allocated, growable and not null terminated.

3. 🌟

```
|
// Fill the blank
fn main() {
    let mut s = __;
    s.push_str("hello, world");
    s.push('!');

    assert_eq!(s, "hello, world!");

    println!("Success!");
}
```

4. 🌟🌟🌟

```
|
// Fix all errors without adding newline
fn main() {
    let s = String::from("hello");
    s.push(',');
    s.push(" world");
    s += "!".to_string();

    println!("{}", s);
}
```

5. 🌟🌟 replace can be used to replace substring

```
|
// Fill the blank
fn main() {
    let s = String::from("I like dogs");
    // Allocate new memory and store the modified string there
    let s1 = s.__("dogs", "cats");

    assert_eq!(s1, "I like cats");

    println!("Success!");
}
```

More `String` methods can be found under [String](#) module.

6. 🌟🌟 You can only concat a `String` with `&str`, and `String`'s ownership can be moved to another variable.

```
|
// Fix errors without removing any line
fn main() {
    let s1 = String::from("hello,");
    let s2 = String::from("world!");
    let s3 = s1 + s2;
    assert_eq!(s3, "hello,world!");
    println!("{}", s1);
}
```


&str and String

Opposite to the seldom using of `str`, `&str` and `String` are used everywhere!

7. 🌟🌟 `&str` can be converted to `String` in two ways

```
|  
// Fix error with at least two solutions  
fn main() {  
    let s = "hello, world";  
    greetings(s)  
}  
  
fn greetings(s: String) {  
    println!("{}", s)  
}
```

8. 🌟🌟 We can use `String::from` or `to_string` to convert a `&str` to `String`

```
|  
// Use two approaches to fix the error and without adding a new line  
fn main() {  
    let s = "hello, world".to_string();  
    let s1: &str = s;  
  
    println!("Success!");  
}
```

String escapes

9. 🌟

```
fn main() {  
    // You can use escapes to write bytes by their hexadecimal values  
    // Fill the blank below to show "I'm writing Rust"  
    let byte_escape = "I'm writing Ru\x73__!";  
    println!("What are you doing\x3F (\\x3F means ?) {}", byte_escape);  
  
    // ...Or Unicode code points.  
    let unicode_codepoint = "\u{211D}";  
    let character_name = "\"DOUBLE-STRUCK CAPITAL R\"";  
  
    println!("Unicode character {} (U+211D) is called {}",  
            unicode_codepoint, character_name );  
  
    let long_string = "String literals  
                        can span multiple lines.  
                        The linebreak and indentation here \  
                        can be escaped too!";  
    println!("{}", long_string);  
}
```

10. 🌟🌟🌟 Sometimes there are just too many characters that need to be escaped or it's just much more convenient to write a string out as-is. This is where raw string literals come into play.

```
|  
/* Fill in the blank and fix the errors */  
fn main() {  
    let raw_str = r"Escapes don't work here: \x3F \u{211D}";  
    // Modify above line to make it work  
    assert_eq!(raw_str, "Escapes don't work here: ? ℝ");  
  
    // If you need quotes in a raw string, add a pair of #s  
    let quotes = r#"And then I said: "There is no escape!"#;  
    println!("{}", quotes);  
  
    // If you need "#" in your string, just use more #s in the delimiter.  
    // You can use up to 65535 #s.  
    let delimiter = r####"A string with #" in it. And even "##!"###;  
    println!("{}", delimiter);  
  
    // Fill the blank  
    let long_delimiter = __;  
    assert_eq!(long_delimiter, "Hello, \\"##\\"");  
  
    println!("Success!");  
}
```

Byte string

Want a string that's not UTF-8? (Remember, `str` and `String` must be valid UTF-8). Or maybe you want an array of bytes that's mostly text? Byte strings to the rescue!

Example:

```

use std::str;

fn main() {
    // Note that this is not actually a `&str`
    let bytestring: &[u8; 21] = b"this is a byte string";

    // Byte arrays don't have the `Display` trait, so printing them is a bit limited
    println!("A byte string: {:?}", bytestring);

    // Byte strings can have byte escapes...
    let escaped = b"\x52\x75\x73\x74 as bytes";
    // ...But no unicode escapes
    // let escaped = b"\u{211D} Is not allowed";
    println!("Some escaped bytes: {:?}", escaped);

    // Raw byte strings work just like raw strings
    let raw_bytestring = br"\u{211D} is not escaped here";
    println!("{:?}", raw_bytestring);

    // Converting a byte array to `str` can fail
    if let Ok(my_str) = str::from_utf8(raw_bytestring) {
        println!("And the same as text: '{}'", my_str);
    }

    let _quotes = br#"You can also use "fancier" formatting, \
        like with normal raw strings"#;

    // Byte strings don't have to be UTF-8
    let shift_jis = b"\x82\xe6\x82\xa8\x82\xb1\x82\xbb"; // "⌘ ⌘ ⌘ ⌘" In SHIFT-JIS

    // But then they can't always be converted to `str`
    match str::from_utf8(shift_jis) {
        Ok(my_str) => println!("Conversion successful: '{}'", my_str),
        Err(e) => println!("Conversion failed: {:?}", e),
    };
}

```

A more detailed listing of the ways to write string literals and escape characters is given in the ['Tokens' chapter](#) of the Rust Reference.

String index

11. 🌟🌟🌟 You can't use index to access a char in a string, but you can use slice `&s1[start..end]` .

```
fn main() {
    let s1 = String::from("hi, 🐶");
    let h = s1[0]; // Modify this line to fix the error, tips: `h` only takes 1 byte
    assert_eq!(h, "h");

    let h1 = &s1[3..5]; // Modify this line to fix the error, tips: `🐶` takes 3 by
    assert_eq!(h1, " 🐶");

    println!("Success!");
}
```

Operate on UTF8 string

12. 🌟

```
fn main() {
    // Fill the blank to print each char in "🐶 🐶 . 🐶 🐶"
    for c in "🐶 🐶 . 🐶 🐶" {
        println!("{}", c)
    }
}
```

utf8_slice

You can use [utf8_slice](#) to slice UTF8 string, it can index chars instead of bytes.

Example

```
use utf8_slice;
fn main() {
    let s = "The 🚀 goes to the 🌑!";

    let rocket = utf8_slice::slice(s, 4, 5);
    // Will equal "🚀"
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Array

The type of array is `[T; Length]`, as you can see, array's length is part of their type signature. So their length must be known at compile time.

For example, you cant initialize an array like below:

```
fn init_arr(n: i32) {  
    let arr = [1; n];  
}
```

This will cause an error, because the compiler has no idea of the exact size of the array at compile time.

1. 🌟

```
|  
fn main() {  
    // Fill the blank with proper array type  
    let arr: __ = [1, 2, 3, 4, 5];  
  
    // Modify the code below to make it work  
    assert!(arr.len() == 4);  
  
    println!("Success!");  
}
```

2. 🌟🌟

```
|  
fn main() {  
    // We can ignore parts of the array type or even the whole type, let the compile  
    let arr0 = [1, 2, 3];  
    let arr: [_; 3] = ['a', 'b', 'c'];  
  
    // Fill the blank  
    // Arrays are stack allocated, `std::mem::size_of_val` returns the bytes which a  
    // A char takes 4 bytes in Rust: Unicode char  
    assert!(std::mem::size_of_val(&arr) == __);  
  
    println!("Success!");  
}
```



3. 🌟 All elements in an array can be initialized to the same value at once.

```

fn main() {
    // Fill the blank
    let list: [i32; 100] = __ ;

    assert!(list[0] == 1);
    assert!(list.len() == 100);

    println!("Success!");
}

```

4. 🌟 All elements in an array must be of the same type

```

fn main() {
    // Fix the error
    let _arr = [1, 2, '3'];

    println!("Success!");
}

```

5. 🌟 Indexing starts at 0.

```

fn main() {
    let arr = ['a', 'b', 'c'];

    let ele = arr[1]; // Only modify this line to make the code work!

    assert!(ele == 'a');

    println!("Success!");
}

```

6. 🌟 Out of bounds indexing causes panic .

```

// Fix the error
fn main() {
    let names = [String::from("Sunfei"), "Sunface".to_string()];

    // `Get` returns an Option<T>, it's safe to use
    let name0 = names.get(0).unwrap();

    // But indexing is not safe
    let _name1 = &names[2];

    println!("Success!");
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Slice

Slices are similar to arrays, but their length is not known at compile time, so you can't use slice directly.

1. 🌟🌟 Here, both `[i32]` and `str` are slice types, but directly using it will cause errors. You have to use the reference of the slice instead: `&[i32]` , `&str` .

```
|  
// Fix the errors, DON'T add new lines!  
fn main() {  
    let arr = [1, 2, 3];  
    let s1: [i32] = arr[0..2];  
  
    let s2: str = "hello, world" as str;  
  
    println!("Success!");  
}
```

A slice reference is a two-word object, for simplicity reasons, from now on we will use slice instead of `slice reference` . The first word is a pointer to the data, and the second word is the length of the slice. The word size is the same as `usize`, determined by the processor architecture, e.g. 64 bits on an x86-64. Slices can be used to borrow a section of an array, and have the type signature `&[T]` .

2. 🌟🌟🌟

```
|  
fn main() {  
    let arr: [char; 3] = [' ', ' ', '人'];  
  
    let slice = &arr[..2];  
  
    // Modify '8' to make it work  
    // TIPS: slice( reference ) IS NOT an array, if it is an array, then `assert!` w  
    assert!(std::mem::size_of_val(&slice) == 8);  
  
    println!("Success!");  
}
```

3. 🌟🌟

```

fn main() {
    let arr: [i32; 5] = [1, 2, 3, 4, 5];
    // Fill the blanks to make the code work
    let slice: __ = __;
    assert_eq!(slice, &[2, 3, 4]);

    println!("Success!");
}

```

String slices

4. 🌟

```

fn main() {
    let s = String::from("hello");

    let slice1 = &s[0..2];
    // Fill the blank to make the code work, DON'T USE 0..2 again
    let slice2 = &s[__];

    assert_eq!(slice1, slice2);

    println!("Success!");
}

```

5. 🌟

```

fn main() {
    let s = "  .  ":
    // Modify this line to make the code work
    let slice = &s[0..2];

    assert!(slice == " ");

    println!("Success!");
}

```

6. 🌟🌟 &String can be implicitly converted into &str .


```
|  
// Fix errors  
fn main() {  
    let mut s = String::from("hello world");  
  
    // Here, &s is `&String` type, but `first_letter` needs a `&str` type.  
    // It works because `&String` can be implicitly converted to `&str`. If you want  
    let letter = first_letter(&s);  
  
    s.clear(); // error!  
  
    println!("the first letter is: {}", letter);  
}  
fn first_letter(s: &str) -> &str {  
    &s[..1]  
}
```



You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Tuple

1. 🌟 Elements in a tuple can have different types. Tuple's type signature is $(T_1, T_2, \dots)$, where T_1, T_2 are the types of tuple's members.

```
|  
fn main() {  
    let _t0: (u8, i16) = (0, -1);  
    // Tuples can be tuple's members  
    let _t1: (u8, (i16, u32)) = (0, (-1, 1));  
    // Fill the blanks to make the code work  
    let t: (u8, __, i64, __, __) = (1u8, 2u16, 3i64, "hello", String::from(", world"  
  
    println!("Success!");  
}
```

2. 🌟 Members can be extracted from the tuple using indexing.

```
|  
// Make it work  
fn main() {  
    let t = ("i", "am", "sunface");  
    assert_eq!(t.1, "sunface");  
  
    println!("Success!");  
}
```

3. 🌟 Long tuples cannot be printed

```
|  
// Fix the error  
fn main() {  
    let too_long_tuple = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13);  
    println!("too long tuple: {:?}", too_long_tuple);  
}
```

4. 🌟 Destructuring tuple with pattern.

```
|  
fn main() {  
    let tup = (1, 6.4, "hello");  
  
    // Fill the blank to make the code work  
    let __ = tup;  
  
    assert_eq!(x, 1);  
    assert_eq!(y, "hello");  
    assert_eq!(z, 6.4);  
  
    println!("Success!");  
}
```

5. 🌟🌟🌟 Destructure assignments.

```
fn main() {  
    let (x, y, z);  
  
    // Fill the blank  
    __ = (1, 2, 3);  
  
    assert_eq!(x, 3);  
    assert_eq!(y, 1);  
    assert_eq!(z, 2);  
  
    println!("Success!");  
}
```

6. 🌟🌟🌟 Tuples can be used as function arguments and return values

```
fn main() {  
    // Fill the blank, need a few computations here.  
    let (x, y) = sum_multiply(__);  
  
    assert_eq!(x, 5);  
    assert_eq!(y, 6);  
  
    println!("Success!");  
}  
  
fn sum_multiply(nums: (i32, i32)) -> (i32, i32) {  
    (nums.0 + nums.1, nums.0 * nums.1)  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Struct

The types of structs

1. 🌟 We must specify concrete values for each of the fields in struct.

```
|  
// Fix the error  
struct Person {  
    name: String,  
    age: u8,  
    hobby: String  
}  
fn main() {  
    let age = 30;  
    let p = Person {  
        name: String::from("sunface"),  
        age,  
    };  
  
    println!("Success!");  
}
```

2. 🌟 Unit struct don't have any fields. It can be useful when you need to implement a trait on some type but don't have any data that you want to store in the type itself.

```
|  
struct Unit;  
trait SomeTrait {  
    // ...Some behaviors defined here.  
}  
  
// We don't care about what fields are in the Unit, but we care about its behavior  
// So we use a struct with no fields and implement some behaviors for it  
impl SomeTrait for Unit { }  
fn main() {  
    let u = Unit;  
    do_something_with_unit(u);  
  
    println!("Success!");  
}  
  
// Fill the blank to make the code work  
fn do_something_with_unit(u: __) { }
```

3. 🌟🌟🌟 Tuple struct looks similar to tuples, it has added meaning the struct name provides but has no named fields. It's useful when you want to give the whole tuple a name, but don't care about the fields's names.

```
|
// Fix the error and fill the blanks
struct Color(i32, i32, i32);
struct Point(i32, i32, i32);
fn main() {
    let v = Point(__, __, __);
    check_color(v);

    println!("Success!");
}

fn check_color(p: Color) {
    let (x, _, _) = p;
    assert_eq!(x, 0);
    assert_eq!(p.1, 127);
    assert_eq!(__, 255);
}
```

Operating on structs

4. 🌟 You can make a whole struct mutable when instantiating it, but Rust doesn't allow us to mark only certain fields as mutable.

```
|
// Fill the blank and fix the error without adding/removing new line
struct Person {
    name: String,
    age: u8,
}
fn main() {
    let age = 18;
    let p = Person {
        name: String::from("sunface"),
        age,
    };

    // How can you believe sunface is only 18?
    p.age = 30;

    // Fill the blank
    __ = String::from("sunfei");

    println!("Success!");
}
```

5. 🌟 Using *field init shorthand syntax* to reduce repetitions.

```
|
// Fill the blank
struct Person {
    name: String,
    age: u8,
}
fn main() {
    println!("Success!");
}

fn build_person(name: String, age: u8) -> Person {
    Person {
        age,
        --
    }
}
```

6. 🌟 You can create instance from other instance with *struct update syntax*

```
|
// Fill the blank to make the code work
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}
fn main() {
    let u1 = User {
        email: String::from("someone@example.com"),
        username: String::from("sunface"),
        active: true,
        sign_in_count: 1,
    };

    let u2 = set_email(u1);

    println!("Success!");
}

fn set_email(u: User) -> User {
    User {
        email: String::from("contact@im.dev"),
        --
    }
}
```

Print the structs

7. 🌟🌟 We can use `#[derive(Debug)]` to [make a struct printable](#).

```

|
// Fill the blanks to make the code work
#[__]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let scale = 2;
    let rect1 = Rectangle {
        width: dbg!(30 * scale), // Print debug info to stderr and assign the value
        height: 50,
    };

    dbg!(&rect1); // Print debug info to stderr

    println!(__, rect1); // Print debug info to stdout
}

```



Partial move

Within the destructuring of a single variable, both by-move and by-reference pattern bindings can be used at the same time. Doing this will result in a partial move of the variable, which means that parts of the variable will be moved while other parts stay. In such a case, the parent variable cannot be used afterwards as a whole, however the parts that are only referenced (and not moved) can still be used.

Example

```
|  
fn main() {  
    #[derive(Debug)]  
    struct Person {  
        name: String,  
        age: Box<u8>,  
    }  
  
    let person = Person {  
        name: String::from("Alice"),  
        age: Box::new(20),  
    };  
  
    // `name` is moved out of person, but `age` is referenced  
    let Person { name, ref age } = person;  
  
    println!("The person's age is {}", age);  
  
    println!("The person's name is {}", name);  
  
    // Error! borrow of partially moved value: `person` partial move occurs  
    //println!("The person struct is {:?}", person);  
  
    // `person` cannot be used but `person.age` can be used as it is not moved  
    println!("The person's age from person struct is {}", person.age);  
}
```

Exercises

8. 🌟🌟

```
|  
// Fix errors to make it work  
#[derive(Debug)]  
struct File {  
    name: String,  
    data: String,  
}  
fn main() {  
    let f = File {  
        name: String::from("readme.md"),  
        data: "Rust By Practice".to_string()  
    };  
  
    let _name = f.name;  
  
    // ONLY modify this line  
    println!("{}", {}, {:?}, f.name, f.data, f);  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Enum

1. 🌟🌟 Enums can be created with explicit discriminator.

```
|  
// Fix the errors  
enum Number {  
    Zero,  
    One,  
    Two,  
}  
  
enum Number1 {  
    Zero = 0,  
    One,  
    Two,  
}  
  
// C-like enum  
enum Number2 {  
    Zero = 0.0,  
    One = 1.0,  
    Two = 2.0,  
}  
  
fn main() {  
    // An enum variant can be converted to a integer by `as`  
    assert_eq!(Number::One, Number1::One);  
    assert_eq!(Number1::One, Number2::One);  
  
    println!("Success!");  
}
```

2. 🌟 Each enum variant can hold its own data.

```
|  
// Fill in the blank  
enum Message {  
    Quit,  
    Move { x: i32, y: i32 },  
    Write(String),  
    ChangeColor(i32, i32, i32),  
}  
  
fn main() {  
    let msg1 = Message::Move{__}; // Instantiating with x = 1, y = 2  
    let msg2 = Message::Write(__); // Instantiating with "hello, world!"  
  
    println!("Success!");  
}
```

3. 🌟🌟 We can get the data which an enum variant is holding by pattern match.

```
|  
// Fill in the blank and fix the error  
enum Message {  
    Quit,  
    Move { x: i32, y: i32 },  
    Write(String),  
    ChangeColor(i32, i32, i32),  
}  
  
fn main() {  
    let msg = Message::Move{x: 1, y: 2};  
  
    if let Message::Move{__} = msg {  
        assert_eq!(a, b);  
    } else {  
        panic!("NEVER LET THIS RUIN! ");  
    }  
  
    println!("Success!");  
}
```

4. 🌟🌟

```
|  
// Fill in the blank and fix the errors  
enum Message {  
    Quit,  
    Move { x: i32, y: i32 },  
    Write(String),  
    ChangeColor(i32, i32, i32),  
}  
  
fn main() {  
    let msgs: __ = [  
        Message::Quit,  
        Message::Move{x:1, y:3},  
        Message::ChangeColor(255,255,0)  
    ];  
  
    for msg in msgs {  
        show_message(msg)  
    }  
}  
  
fn show_message(msg: Message) {  
    println!("{}", msg);  
}
```

5. 🌟🌟 Since there is no `null` in Rust, we have to use enum `Option<T>` to deal with the cases when the value is absent.

```

|
// Fill in the blank to make the `println` work.
// Also add some code to prevent the `panic` from running.
fn main() {
    let five = Some(5);
    let six = plus_one(five);
    let none = plus_one(None);

    if let __ = six {
        println!("{}", n);

        println!("Success!");
    }

    panic!("NEVER LET THIS RUN! "):
}

fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        __ => None,
        __ => Some(i + 1),
    }
}

```

6. 🌟🌟🌟🌟 Implement a linked-list via enums.

```

use crate::List::*;

enum List {
    // Cons: Tuple struct that wraps an element and a pointer to the next node
    Cons(u32, Box<List>),
    // Nil: A node that signifies the end of the linked list
    Nil,
}

// Methods can be attached to an enum
impl List {
    // Create an empty list
    fn new() -> List {
        // `Nil` has type `List`
        Nil
    }

    // Consume a list, and return the same list with a new element at its front
    fn prepend(self, elem: u32) -> __ {
        // `Cons` also has type List
        Cons(elem, Box::new(self))
    }

    // Return the length of the list
    fn len(&self) -> u32 {
        // `self` has to be matched, because the behavior of this method
        // depends on the variant of `self`
        // `self` has type `&List`, and `*self` has type `List`, matching on a
        // concrete type `T` is preferred over a match on a reference `&T`
        // After Rust 2018 you can use self here and tail (with no ref) below as well
        // rust will infer &s and ref tail.
        // See https://doc.rust-lang.org/edition-guide/rust-2018/ownership-and-lifetimes
        match *self {
            // Can't take ownership of the tail, because `self` is borrowed;
            // Instead take a reference to the tail
            Cons(_, ref tail) => 1 + tail.len(),
            // Base Case: An empty list has zero length
            Nil => 0
        }
    }

    // Return representation of the list as a (heap allocated) string
    fn stringify(&self) -> String {
        match *self {
            Cons(head, __ tail) => {
                // `format!` is similar to `print!`, but returns a heap
                // allocated string instead of printing to the console
                format!("{}", head, tail.__())
            },
            Nil => {
                format!("Nil")
            },
        }
    }
}

```

```
,  
  
fn main() {  
    // Create an empty linked list  
    let mut list = List::new();  
  
    // Prepend some elements  
    list = list.prepend(1);  
    list = list.prepend(2);  
    list = list.prepend(3);  
  
    // Show the final state of the list  
    println!("linked list has length: {}", list.len());  
    println!("{}", list.stringify());  
}
```



You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Flow control

If/else

1. 🌟

```
|  
// Fill in the blanks  
fn main() {  
    let n = 5;  
  
    if n < 0 {  
        println!("{}", is negative, n);  
    } -- n > 0 {  
        println!("{}", is positive, n);  
    } -- {  
        println!("{}", is zero, n);  
    }  
}
```

2. 🌟🌟 If/else expression can be used in assignments.

```
|  
// Fix the errors  
fn main() {  
    let n = 5;  
  
    let big_n =  
        if n < 10 && n > -10 {  
            println!("{}", and is a small number, increase ten-fold");  
  
            10 * n  
        } else {  
            println!("{}", and is a big number, halve the number");  
  
            n / 2.0 ;  
        }  
  
    println!("{}", -> {}, n, big_n);  
}
```

For

3. 🌟 The `for in` construct can be used to iterate through an Iterator, e.g a range `a..b`.

```

fn main() {
    for n in 1..=100 { // modify this line to make the code work
        if n == 100 {
            panic!("NEVER LET THIS RUN")
        }
    }

    println!("Success!");
}

```

4. 🌟🌟

```

// Fix the errors without adding or removing lines
fn main() {
    let names = [String::from("liming"),String::from("hanmeimei")];
    for name in names {
        // Do something with name...
    }

    println!("{:?}", names);

    let numbers = [1, 2, 3];
    // The elements in numbers are Conv. so there is no move here
    for n in numbers {
        // Do something with n...
    }

    println!("{:?}", numbers);
}

```

5. 🌟

```

fn main() {
    let a = [4, 3, 2, 1];

    // Iterate the indexing and value in 'a'
    for (i,v) in a.__ {
        println!("The {}th element is {}",i+1,v);
    }
}

```

While

6. 🌟🌟 The `while` keyword can be used to run a loop when a condition is true.


```

// Fill in the blanks to make the last println! work !
fn main() {
    // A counter variable
    let mut n = 1;

    // Loop while the condition is true
    while n <= 10 {
        if n % 15 == 0 {
            println!("fizzbuzz");
        } else if n % 3 == 0 {
            println!("fizz");
        } else if n % 5 == 0 {
            println!("buzz");
        } else {
            println!("{}", n);
        }

        n++;
    }

    println!("n reached {}, so loop is over",n);
}

```

Continue and break

7. 🌟 Use `break` to break the loop.

```

// Fill in the blank
fn main() {
    let mut n = 0;
    for i in 0..=100 {
        if n == 66 {
            break;
        }
        n += 1;
    }

    assert_eq!(n, 66);

    println!("Success!");
}

```

8. 🌟🌟 `continue` will skip over the remaining code in current iteration and go to the next iteration.

```
|
// Fill in the blanks
fn main() {
    let mut n = 0;
    for i in 0..=100 {
        if n != 66 {
            n+=1;
            --;
        }

        --
    }

    assert_eq!(n, 66);

    println!("Success!");
}
```

Loop

9. 🌟🌟 Loop is usually used together with `break` or `continue`.

```
|
// Fill in the blanks
fn main() {
    let mut count = 0u32;

    println!("Let's count until infinity!");

    // Infinite loop
    loop {
        count += 1;

        if count == 3 {
            println!("three");

            // Skip the rest of this iteration
            --;
        }

        println!("{}", count);

        if count == 5 {
            println!("OK, that's enough");

            --;
        }
    }

    assert_eq!(count, 5);

    println!("Success!");
}
```

10. 🌟🌟 Loop is an expression, so we can use it with `break` to return a value

```
|  
// Fill in the blank  
fn main() {  
    let mut counter = 0;  
  
    let result = loop {  
        counter += 1;  
  
        if counter == 10 {  
            --;  
        }  
    };  
  
    assert_eq!(result, 20);  
  
    println!("Success!");  
}
```

11. 🌟🌟🌟 It's possible to break or continue outer loops when dealing with nested loops. In these cases, the loops must be annotated with some 'label', and the label must be passed to the break/continue statement.

```
|  
// Fill in the blank  
fn main() {  
    let mut count = 0;  
    'outer: loop {  
        'inner1: loop {  
            if count >= 20 {  
                // This would break only the inner1 loop  
                break 'inner1; // `break` is also works.  
            }  
            count += 2;  
        }  
  
        count += 5;  
  
        'inner2: loop {  
            if count >= 30 {  
                // This breaks the outer loop  
                break 'outer;  
            }  
  
            // This will continue the outer loop  
            continue 'outer;  
        }  
    }  
  
    assert!(count == __);  
  
    println!("Success!");  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Pattern Match

Learning resources:

- English: [Rust Book 18](#)
- 简体中文: [Rust 模式匹配 - 中文](#)

Match, if let

Match

1. 🌟🌟

```
|  
// Fill the blanks  
enum Direction {  
    East,  
    West,  
    North,  
    South,  
}  
  
fn main() {  
    let dire = Direction::South;  
    match dire {  
        Direction::East => println!("East"),  
        -- => { // Matching South or North here  
            println!("South or North");  
        },  
        _ => println!(__),  
    };  
}
```

2. 🌟🌟 Match is an expression, so we can use it in assignments.

```
|  
fn main() {  
    let boolean = true;  
  
    // Fill the blank with a match expression:  
    //  
    // boolean = true => binary = 1  
    // boolean = false => binary = 0  
    let binary = __;  
  
    assert_eq!(binary, 1);  
  
    println!("Success!");  
}
```

3. 🌟🌟 Using match to get the data an enum variant holds.

```

|
// Fill in the blanks
enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

fn main() {
    let msgs = [
        Message::Quit,
        Message::Move{x:1, y:3},
        Message::ChangeColor(255,255,0)
    ];

    for msg in msgs {
        show_message(msg)
    }

    println!("Success!");
}

fn show_message(msg: Message) {
    match msg {
        __ => { // match Message::Move
            assert_eq!(a, 1);
            assert_eq!(b, 3);
        },
        Message::ChangeColor(_, g, b) => {
            assert_eq!(g, __);
            assert_eq!(b, __);
        }
        __ => println!("no data in these variants")
    }
}

```

matches!

`matches!` looks like `match`, but can do something different.

4. 🌟🌟🌟

```

fn main() {
    let alphabets = ['a', 'E', 'Z', '0', 'x', '9' , 'Y'];

    // Fill the blank with `matches!` to make the code work
    for ab in alphabets {
        assert!(__)
    }

    println!("Success!");
}

```

5. 🌟🌟

```

enum MyEnum {
    Foo,
    Bar
}

fn main() {
    let mut count = 0;

    let v = vec![MyEnum::Foo, MyEnum::Bar, MyEnum::Foo];
    for e in v {
        if e == MyEnum::Foo { // Fix the error by changing only this line
            count += 1;
        }
    }

    assert_eq!(count, 2);

    println!("Success!");
}

```

If let

For some cases, when matching enums, `match` is too heavy. We can use `if let` instead.

6. 🌟


```
|
fn main() {
    let o = Some(7);

    // Remove the whole `match` block, using `if let` instead
    match o {
        Some(i) => {
            println!("This is a really long string and `{:?}`", i);

            println!("Success!");
        }
        _ => {}
    };
}
```

7. 🌟🌟

```
|
// Fill in the blank
enum Foo {
    Bar(u8)
}

fn main() {
    let a = Foo::Bar(1);

    -- {
        println!("foobar holds the value: {}", i);

        println!("Success!");
    }
}
```

8. 🌟🌟

```
|
enum Foo {
    Bar,
    Baz,
    Qux(u32)
}

fn main() {
    let a = Foo::Qux(10);

    // Remove the codes below, using `match` instead
    if let Foo::Bar = a {
        println!("match foo::bar")
    } else if let Foo::Baz = a {
        println!("match foo::baz")
    } else {
        println!("match others")
    }
}
```

Shadowing

9. 🌟🌟🌟

```
|  
// Fix the errors in-place  
fn main() {  
    let age = Some(30);  
    if let Some(age) = age { // Create a new variable with the same name as previous  
        assert_eq!(age, Some(30));  
    } // The new variable `age` goes out of scope here  
  
    match age {  
        // Match can also introduce a new shadowed variable  
        Some(age) => println!("age is a new variable, it's value is {}",age),  
        _ => ()  
    }  
}
```



You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Patterns

1. 🌟🌟 Use `|` to match several values, use `..=` to match an inclusive range.

```
|  
fn main() {}  
fn match_number(n: i32) {  
    match n {  
        // Match a single value  
        1 => println!("One!"),  
        // Fill in the blank with `|`, DON'T use `..` or `..=`  
        __ => println!("match 2 -> 5"),  
        // Match an inclusive range  
        6..=10 => {  
            println!("match 6 -> 10")  
        },  
        _ => {  
            println!("match -infinite -> 0 or 11 -> +infinite")  
        }  
    }  
}
```

2. 🌟🌟🌟 The `@` operator lets us create a variable that holds a value, at the same time we are testing that value to see whether it matches a pattern.

```
|  
struct Point {  
    x: i32,  
    y: i32,  
}  
  
fn main() {  
    // Fill in the blank to let p match the second arm  
    let p = Point { x: __, y: __ };  
  
    match p {  
        Point { x, y: 0 } => println!("On the x axis at {}", x),  
        // Second arm  
        Point { x: 0..=5, y: y@ (10 | 20 | 30) } => println!("On the y axis at {}", y),  
        Point { x, y } => println!("On neither axis: ( {}, {})", x, y),  
    }  
}
```

3. 🌟🌟🌟

```

|
// Fix the errors
enum Message {
    Hello { id: i32 },
}

fn main() {
    let msg = Message::Hello { id: 5 };

    match msg {
        Message::Hello {
            id: 3..=7,
        } => println!("Found an id in range [3, 7]: {}", id),
        Message::Hello { id: newid@10 | 11 | 12 } => {
            println!("Found an id in another range [10, 12]: {}", newid)
        }
        Message::Hello { id } => println!("Found some other id: {}", id),
    }
}

```

4. 🌟🌟 A match guard is an additional if condition specified after the pattern in a match arm that must also match, along with the pattern matching, for that arm to be chosen.

```

|
// Fill in the blank to make the code work, `split` MUST be used
fn main() {
    let num = Some(4);
    let split = 5;
    match num {
        Some(x) __ => assert!(x < split),
        Some(x) => assert!(x >= split),
        None => (),
    }

    println!("Success!");
}

```

5. 🌟🌟 Ignoring remaining parts of the value with ..

```

|
// Fill the blank to make the code work
fn main() {
    let numbers = (2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048);

    match numbers {
        __ => {
            assert_eq!(first, 2);
            assert_eq!(last, 2048);
        }
    }

    println!("Success!");
}

```

6. 🌟🌟 Using pattern `&mut v` to match a mutable reference requires you to be very careful, due to `v` being a value after matching.

```
|  
// FIX the error with least changing  
// DON'T remove any code line  
fn main() {  
    let mut v = String::from("hello,");  
    let r = &mut v;  
  
    match r {  
        &mut value => value.push_str(" world!")  
    }  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Associated functions & Methods

Examples

```
struct Point {
    x: f64,
    y: f64,
}

// Implementation block, all `Point` associated functions & methods go in here.
impl Point {
    // This is an "associated function" because this function is associated with
    // a particular type, that is, Point.
    //
    // Associated functions don't need to be called with an instance.
    // These functions are generally used like constructors.
    fn origin() -> Point {
        Point { x: 0.0, y: 0.0 }
    }

    // Another associated function, taking two arguments:
    fn new(x: f64, y: f64) -> Point {
        Point { x: x, y: y }
    }
}

struct Rectangle {
    p1: Point,
    p2: Point,
}

impl Rectangle {
    // This is a method.
    // `&self` is sugar for `self: &Self`, where `Self` is the type of the
    // caller object. In this case `Self` = `Rectangle`
    fn area(&self) -> f64 {
        // `self` gives access to the struct fields via the dot operator.
        let Point { x: x1, y: y1 } = self.p1;
        let Point { x: x2, y: y2 } = self.p2;

        // `abs` is a `f64` method that returns the absolute value of the
        // caller
        ((x1 - x2) * (y1 - y2)).abs()
    }

    fn perimeter(&self) -> f64 {
        let Point { x: x1, y: y1 } = self.p1;
        let Point { x: x2, y: y2 } = self.p2;

        2.0 * ((x1 - x2).abs() + (y1 - y2).abs())
    }
}
```

```

// This method requires the caller object to be mutable
// `&mut self` desugars to `self: &mut Self`
fn translate(&mut self, x: f64, y: f64) {
    self.p1.x += x;
    self.p2.x += x;

    self.p1.y += y;
    self.p2.y += y;
}
}

// `Pair` owns resources: two heap allocated integers.
struct Pair(Box<i32>, Box<i32>);

impl Pair {
    // This method "consumes" the resources of the caller object
    // `self` desugars to `self: Self`
    fn destroy(self) {
        // Destructure `self`
        let Pair(first, second) = self;

        println!("Destroying Pair({}, {})", first, second);

        // `first` and `second` go out of scope and get freed.
    }
}

fn main() {
    let rectangle = Rectangle {
        // Associated functions are called using double colons
        p1: Point::origin(),
        p2: Point::new(3.0, 4.0),
    };

    // Methods are called using the dot operator.
    // Note that the first argument `&self` is implicitly passed, i.e.
    // `rectangle.perimeter()` == `Rectangle::perimeter(&rectangle)`
    println!("Rectangle perimeter: {}", rectangle.perimeter());
    println!("Rectangle area: {}", rectangle.area());

    let mut square = Rectangle {
        p1: Point::origin(),
        p2: Point::new(1.0, 1.0),
    };

    // Error! `rectangle` is immutable, but this method requires a mutable
    // object.
    //rectangle.translate(1.0, 0.0);
    // TODO ^ Try uncommenting this line

    // Okay! Mutable objects can call mutable methods
    square.translate(1.0, 1.0);

    let pair = Pair(Box::new(1), Box::new(2));

```



```

pair.destroy();

// Error! Previous `destroy` call "consumed" `pair`
//pair.destroy();
// TODO ^ Try uncommenting this line
}

```

Exercises

Method

1. 🌟🌟 Methods are similar to functions: Declare with `fn`, have parameters and a return value. Unlike functions, methods are defined within the context of a struct (or an enum or a trait object), and their first parameter is always `self`, which represents the instance of the struct the method is being called on.

```

struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    // Complete the area method which return the area of a Rectangle.
    fn area
}

fn main() {
    let rect1 = Rectangle { width: 30, height: 50 };

    assert_eq!(rect1.area(), 1500);

    println!("Success!");
}

```

2. 🌟🌟 `self` will take the ownership of current struct instance, however, `&self` will only borrow a reference from the instance.

```
// Only fill in the blanks, DON'T remove any line!
#[derive(Debug)]
struct TrafficLight {
    color: String,
}

impl TrafficLight {
    pub fn show_state(__) {
        println!("the current state is {}", __.color);
    }
}

fn main() {
    let light = TrafficLight{
        color: "red".to_owned(),
    };
    // Don't take the ownership of `light` here.
    light.show_state();
    // ... Otherwise, there will be an error below
    println!("{:?}", light);
}
```

3. 🌟🌟 The `&self` is actually short for `self: &Self`. Within an `impl` block, the type `Self` is an alias for the type that the `impl` block is for. Methods must have a parameter named `self` of type `Self` for their first parameter, so Rust lets you abbreviate this with only the name `self` in the first parameter spot.

```
struct TrafficLight {
    color: String,
}

impl TrafficLight {
    // Using `Self` to fill in the blank.
    pub fn show_state(__) {
        println!("the current state is {}", self.color);
    }

    // Fill in the blank, DON'T use any variants of `Self`.
    pub fn change_state(__) {
        self.color = "green".to_string()
    }
}

fn main() {
    println!("Success!");
}
```

Associated functions

4. 🌟🌟 All functions defined within an `impl` block are called associated functions because they're associated with the type named after the `impl`. We can define associated

functions that don't have `self` as their first parameter (and thus are not methods) because they don't need an instance of the type to work with.

```
#[derive(Debug)]
struct TrafficLight {
    color: String,
}

impl TrafficLight {
    // 1. Implement an associated function `new`,
    // 2. It will return a TrafficLight contains color "red"
    // 3. Must use `Self`, DONT use `TrafficLight` in fn signatures or body
    pub fn new()

    pub fn get_state(&self) -> &str {
        &self.color
    }
}

fn main() {
    let light = TrafficLight::new();
    assert_eq!(light.get_state(), "red");

    println!("Success!");
}
```

Multiple `impl` blocks

5. 🌟 Each struct is allowed to have multiple `impl` blocks.

```
|
struct Rectangle {
    width: u32,
    height: u32,
}

// Using multiple `impl` blocks to rewrite the code below.
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}

fn main() {
    println!("Success!");
}
```

Enums

6. 🌟🌟🌟 We can also implement methods for enums.

```
|  
#[derive(Debug)]  
enum TrafficLightColor {  
    Red,  
    Yellow,  
    Green,  
}  
  
// Implement TrafficLightColor with a method.  
impl TrafficLightColor {  
  
}  
  
fn main() {  
    let c = TrafficLightColor::Yellow;  
  
    assert_eq!(c.color(), "yellow");  
  
    println!("{:?}",c);  
}
```

Practice

@todo

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Generics and Traits

Learning resources:

- English: [Rust Book 10.1, 10.2](#)
- 繁體中文: [Rust 泛型 - 泛型](#)

Generics

Functions

1. 🌟🌟🌟

```
|  
// Fill in the blanks to make it work  
struct A;           // Concrete type `A`.  
struct S(A);        // Concrete type `S`.  
struct SGen<T>(T);  // Generic type `SGen`.  
  
fn reg_fn(_s: S) {}  
  
fn gen_spec_t(_s: SGen<A>) {}  
  
fn gen_spec_i32(_s: SGen<i32>) {}  
  
fn generic<T>(_s: SGen<T>) {}  
  
fn main() {  
    // Using the non-generic functions  
    reg_fn(__);           // Concrete type.  
    gen_spec_t(__);       // Implicitly specified type parameter `A`.  
    gen_spec_i32(__);     // Implicitly specified type parameter `i32`.  
  
    // Explicitly specified type parameter `char` to `generic()`.  
    generic::<char>(__);  
  
    // Implicitly specified type parameter `char` to `generic()`.  
    generic(__);  
  
    println!("Success!");  
}
```

2. 🌟🌟 A function call with explicitly specified type parameters looks like: `fun::<A, B, ...>()`.

```
|  
// Implement the generic function below.  
fn sum  
  
fn main() {  
    assert_eq!(5, sum(2i8, 3i8));  
    assert_eq!(50, sum(20, 30));  
    assert_eq!(2.46, sum(1.23, 1.23));  
  
    println!("Success!");  
}
```

Struct and impl

3. 🌟

```
|  
// Implement struct Point to make it work.
```

```
fn main() {  
    let integer = Point { x: 5, y: 10 };  
    let float = Point { x: 1.0, y: 4.0 };  
  
    println!("Success!");  
}
```

4. 🌟🌟

```
|  
// Modify this struct to make the code work  
struct Point<T> {  
    x: T,  
    y: T,  
}  
  
fn main() {  
    // DON'T modify this code.  
    let p = Point{x: 5, y : "hello".to_string()};  
  
    println!("Success!");  
}
```

5. 🌟🌟

```
|  
// Add generic for Val to make the code work, DON'T modify the code in `main`.  
struct Val {  
    val: f64,  
}  
  
impl Val {  
    fn value(&self) -> &f64 {  
        &self.val  
    }  
}  
  
fn main() {  
    let x = Val{ val: 3.0 };  
    let y = Val{ val: "hello".to_string()};  
    println!("{}", {}, x.value(), y.value());  
}
```

Method

6. 🌟🌟🌟

```
struct Point<T, U> {
    x: T,
    y: U,
}

impl<T, U> Point<T, U> {
    // Implement mixup to make it work, DON'T modify other code.
    fn mixup
}

fn main() {
    let p1 = Point { x: 5, y: 10 };
    let p2 = Point { x: "Hello", y: ' ' };

    let p3 = p1.mixup(p2);

    assert_eq!(p3.x, 5);
    assert_eq!(p3.y, ' ');

    println!("Success!");
}
```

7. 🌟🌟

```
// Fix the errors to make the code work.
struct Point<T> {
    x: T,
    y: T,
}

impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}

fn main() {
    let p = Point{x: 5, y: 10};
    println!("{}",p.distance_from_origin());
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Const Generics

Const generics are generic arguments that range over constant values, rather than types or lifetimes. This allows, for instance, types to be parameterized by integers. In fact, there has been one example of const generic types since early on in Rust's development: the array types `[T; N]`, for some type `T` and `N: usize`. However, there has previously been no way to abstract over arrays of an arbitrary size: if you wanted to implement a trait for arrays of any size, you would have to do so manually for each possible value. For a long time, even the standard library methods for arrays were limited to arrays of length at most 32 due to this problem.

Examples

1. Here's an example of a type and implementation making use of const generics: a type wrapping a pair of arrays of the same size.

```
struct ArrayPair<T, const N: usize> {  
    left: [T; N],  
    right: [T; N],  
}  
  
impl<T: Debug, const N: usize> Debug for ArrayPair<T, N> {  
    // ...  
}
```

2. Currently, const parameters may only be instantiated by const arguments of the following forms:
 - A standalone const parameter.
 - A literal (i.e. an integer, bool, or character).
 - A concrete constant expression (enclosed by `{}`), involving no generic parameters.

```
fn foo<const N: usize>() {}

fn bar<T, const M: usize>() {
    foo::<M>(); // Okay: `M` is a const parameter
    foo::<2021>(); // Okay: `2021` is a literal
    foo::<{20 * 100 + 20 * 10 + 1}>(); // Okay: const expression contains no generic

    foo::<{ M + 1 }>(); // Error: const expression contains the generic parameter `M`
    foo::<{ std::mem::size_of::<T>() }>(); // Error: const expression contains the g

    let _: [u8; M]; // Okay: `M` is a const parameter
    let _: [u8; std::mem::size_of::<T>()]; // Error: const expression contains the g
}

fn main() {}
```

3. Const generics can also let us avoid some runtime checks.

```
/// A region of memory containing at least `N` `T`s.
pub struct MinSlice<T, const N: usize> {
    /// The bounded region of memory. Exactly `N` `T`s.
    pub head: [T; N],
    /// Zero or more remaining `T`s after the `N` in the bounded region.
    pub tail: [T],
}

fn main() {
    let slice: &[u8] = b"Hello, world";
    let reference: Option<&u8> = slice.get(6);
    // We know this value is `Some(b' ')`,
    // but the compiler can't know that.
    assert!(reference.is_some());

    let slice: &[u8] = b"Hello, world";
    // Length check is performed when we construct a MinSlice,
    // and it's known at compile time to be of length 12.
    // If the `unwrap()` succeeds, no more checks are needed
    // throughout the `MinSlice`'s lifetime.
    let minslice = MinSlice::<u8, 12>::from_slice(slice).unwrap();
    let value: u8 = minslice.head[6];
    assert_eq!(value, b' ')
}
```

Exercises

1. 🌟🌟 <T, const N: *usize*> is part of the struct type, it means `Array<i32, 3>` and `Array<i32, 4>` are different types.

```

struct Array<T, const N: usize> {
    data : [T; N]
}

fn main() {
    let arrays = [
        Array{
            data: [1, 2, 3],
        },
        Array {
            data: [1.0, 2.0, 3.0],
        },
        Array {
            data: [1, 2]
        }
    ];

    println!("Success!");
}

```

2. 🌟🌟

```

// Fill in the blanks to make it work.
fn print_array<__>(__) {
    println!("{:?}", arr);
}

fn main() {
    let arr = [1, 2, 3];
    print_array(arr);

    let arr = ["hello", "world"];
    print_array(arr);
}

```

3. 🌟🌟🌟 Sometimes we want to limit the size of a variable, e.g when using in embedding environments, then `const expressions` will fit your needs.

```

#![allow(incomplete_features)]
#![feature(generic_const_exprs)]

fn check_size<T>(val: T)
where
    Assert<{ core::mem::size_of::<T>() < 768 }>: IsTrue,
{
    //...
}

// Fix the errors in main.
fn main() {
    check_size([0u8; 767]);
    check_size([0i32; 191]);
    check_size(["hello "]; // Size of &str ?
    check_size([(); __].map(|_| "hello ".to_string())); // Size of String?
    check_size([' ']; // Size of char ?

    println!("Success!");
}

pub enum Assert<const CHECK: bool> {}

pub trait IsTrue {}

impl IsTrue for Assert<true> {}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Traits

A trait tells the Rust compiler about functionality a particular type has and can share with other types. We can use traits to define shared behavior in an abstract way. We can use trait bounds to specify that a generic type can be any type that has certain behavior.

Note: Traits are similar to interfaces in other languages, although with some differences.

Examples

```
|  
struct Sheep { naked: bool, name: String }  
  
trait Animal {  
    // Associated function signature; `Self` refers to the implementor type.  
    fn new(name: String) -> Self;  
  
    // Method signatures; these will return a string.  
    fn name(&self) -> String;  
  
    fn noise(&self) -> String;  
  
    // Traits can provide default method definitions.  
    fn talk(&self) {  
        println!("{}", self.name(), self.noise());  
    }  
}  
  
impl Sheep {  
    fn is_naked(&self) -> bool {  
        self.naked  
    }  
  
    fn shear(&mut self) {  
        if self.is_naked() {  
            // Implementor methods can use the implementor's trait methods.  
            println!("{}", self.name());  
        } else {  
            println!("{}", self.name());  
  
            self.naked = true;  
        }  
    }  
}  
  
// Implement the `Animal` trait for `Sheep`.  
impl Animal for Sheep {  
    // `Self` is the implementor type: `Sheep`.  
    fn new(name: String) -> Sheep {  
        Sheep { name: name, naked: false }  
    }  
  
    fn name(&self) -> String {  
        self.name.clone()  
    }  
  
    fn noise(&self) -> String {  
        if self.is_naked() {  
            "baaaaah?".to_string()  
        } else {  
            "baaaaah!".to_string()  
        }  
    }  
}
```

```

// Default trait methods can be overridden.
fn talk(&self) {
    // For example, we can add some quiet contemplation.
    println!("{}", pauses briefly... {}", self.name, self.noise());
}

fn main() {
    // Type annotation is necessary in this case.
    let mut dolly: Sheep = Animal::new("Dolly".to_string());
    // TODO ^ Try removing the type annotations.

    dolly.talk();
    dolly.shear();
    dolly.talk();
}

```

Exercises

1. 🌟🌟

```

// Fill in the two impl blocks to make the code work.
// DON'T modify the code in `main`.
trait Hello {
    fn say_hi(&self) -> String {
        String::from("hi")
    }

    fn say_something(&self) -> String;
}

struct Student {}
impl Hello for Student {
}
struct Teacher {}
impl Hello for Teacher {
}

fn main() {
    let s = Student {};
    assert_eq!(s.say_hi(), "hi");
    assert_eq!(s.say_something(), "I'm a good student");

    let t = Teacher {};
    assert_eq!(t.say_hi(), "Hi, I'm your new teacher");
    assert_eq!(t.say_something(), "I'm not a bad teacher");

    println!("Success!");
}

```

Derive

The compiler is capable of providing basic implementations for some traits via the `#[derive]` attribute. For more info, please visit [here](#).

2. 🌟🌟

```
|
// `Centimeters`, a tuple struct that can be compared
#[derive(PartialEq, PartialOrd)]
struct Centimeters(f64);

// `Inches`, a tuple struct that can be printed
#[derive(Debug)]
struct Inches(i32);

impl Inches {
    fn to_centimeters(&self) -> Centimeters {
        let &Inches(inches) = self;

        Centimeters(inches as f64 * 2.54)
    }
}

// ADD some attributes to make the code work!
// DON'T modify other code!
struct Seconds(i32);

fn main() {
    let _one_second = Seconds(1);

    println!("One second looks like: {:?}", _one_second);
    let _this_is_true = (_one_second == _one_second);
    let _this_is_false = (_one_second > _one_second);

    let foot = Inches(12);

    println!("One foot equals {:?}", foot);

    let meter = Centimeters(100.0);

    let cmp =
        if foot.to_centimeters() < meter {
            "smaller"
        } else {
            "bigger"
        };

    println!("One foot is {} than one meter.", cmp);
}
```


Operator

In Rust, many of the operators can be overloaded via traits. That is, some operators can be used to accomplish different tasks based on their input arguments. This is possible because operators are syntactic sugar for method calls. For example, the `+` operator in `a + b` calls the `add` method (as in `a.add(b)`). This `add` method is part of the `Add` trait. Hence, the `+` operator can be used by any implementor of the `Add` trait.

3. 🌟🌟

```
|  
use std::ops;  
  
// Implement fn multiply to make the code work.  
// As mentioned above, `+` needs `T` to implement `std::ops::Add` Trait.  
// So, what about `*`? You can find the answer here: https://doc.rust-lang.org/core.  
fn multiply  
  
fn main() {  
    assert_eq!(6, multiply(2u8, 3u8));  
    assert_eq!(5.0, multiply(1.0, 5.0));  
  
    println!("Success!");  
}
```



4. 🌟🌟🌟

```

|
// Fix the errors, DON'T modify the code in `main`.
use std::ops;

struct Foo;
struct Bar;

struct FooBar;

struct BarFoo;

// The `std::ops::Add` trait is used to specify the functionality of `+`.
// Here, we make `Add<Bar>` - the trait for addition with a RHS of type `Bar`.
// The following block implements the operation: Foo + Bar = FooBar
impl ops::Add<Bar> for Foo {
    type Output = FooBar;

    fn add(self, _rhs: Bar) -> FooBar {
        FooBar
    }
}

impl ops::Sub<Foo> for Bar {
    type Output = BarFoo;

    fn sub(self, _rhs: Foo) -> BarFoo {
        BarFoo
    }
}

fn main() {
    // DON'T modify the code below.
    // You need to derive some trait for FooBar to make it comparable.
    assert_eq!(Foo + Bar, FooBar);
    assert_eq!(Foo - Bar, BarFoo);

    println!("Success!");
}

```

Use trait as function parameters

Instead of a concrete type for the item parameter, we specify the impl keyword and the trait name. This parameter accepts any type that implements the specified trait.

5. 🌟🌟🌟

```

|
// Implement `fn summary` to make the code work.
// Fix the errors without removing any code line
trait Summary {
    fn summarize(&self) -> String;
}

#[derive(Debug)]
struct Post {
    title: String,
    author: String,
    content: String,
}

impl Summary for Post {
    fn summarize(&self) -> String {
        format!("The author of post {} is {}", self.title, self.author)
    }
}

#[derive(Debug)]
struct Weibo {
    username: String,
    content: String,
}

impl Summary for Weibo {
    fn summarize(&self) -> String {
        format!("{}", published a weibo {}", self.username, self.content)
    }
}

fn main() {
    let post = Post {
        title: "Popular Rust".to_string(),
        author: "Sunface".to_string(),
        content: "Rust is awesome!".to_string(),
    };
    let weibo = Weibo {
        username: "sunface".to_string(),
        content: "Weibo seems to be worse than Tweet".to_string(),
    };

    summary(post);
    summary(weibo);

    println!("{:?}", post);
    println!("{:?}", weibo);
}

// Implement `fn summary` below.

```

Returning Types that Implement Traits

We can also use the `impl Trait` syntax in the return position to return a value of some type that implements a trait.

However, you can only use `impl Trait` if you're returning a single type, use Trait Objects instead when you really need to return several types.

6. 🌟🌟

```
|
struct Sheep {}
struct Cow {}

trait Animal {
    fn noise(&self) -> String;
}

impl Animal for Sheep {
    fn noise(&self) -> String {
        "baaaaah!".to_string()
    }
}

impl Animal for Cow {
    fn noise(&self) -> String {
        "moooooo!".to_string()
    }
}

// Returns some struct that implements Animal, but we don't know which one at compile time
// FIX the errors here, you can make a fake random, or you can use trait object.
fn random_animal(random_number: f64) -> impl Animal {
    if random_number < 0.5 {
        Sheep {}
    } else {
        Cow {}
    }
}

fn main() {
    let random_number = 0.234;
    let animal = random_animal(random_number);
    println!("You've randomly chosen an animal, and it says {}", animal.noise());
}
```



Trait bound

The `impl Trait` syntax works for straightforward cases but is actually syntax sugar for a longer form, which is called a trait bound.

When working with generics, the type parameters often must use traits as bounds to stipulate what functionality a type implements.

7. 🌟🌟

```
fn main() {
    assert_eq!(sum(1, 2), 3);
}

// Implement `fn sum` with trait bound in two ways.
fn sum<T>(x: T, y: T) -> T {
    x + y
}
```

8. 🌟🌟

```
// FIX the errors.
struct Pair<T> {
    x: T,
    y: T,
}

impl<T> Pair<T> {
    fn new(x: T, y: T) -> Self {
        Self {
            x,
            y,
        }
    }
}

impl<T: std::fmt::Debug + PartialOrd> Pair<T> {
    fn cmp_display(&self) {
        if self.x >= self.y {
            println!("The largest member is x = {:?}", self.x);
        } else {
            println!("The largest member is y = {:?}", self.y);
        }
    }
}

struct Unit(i32);

fn main() {
    let pair = Pair{
        x: Unit(1),
        y: Unit(3)
    };

    pair.cmp_display();
}
```

9. 🌟🌟🌟

```

|
// Fill in the blanks to make it work
fn example1() {
    // `T: Trait` is the commonly used way.
    // `T: Fn(u32) -> u32` specifies that we can only pass a closure to `T`.
    struct Cacher<T: Fn(u32) -> u32> {
        calculation: T,
        value: Option<u32>,
    }

    impl<T: Fn(u32) -> u32> Cacher<T> {
        fn new(calculation: T) -> Cacher<T> {
            Cacher {
                calculation,
                value: None,
            }
        }

        fn value(&mut self, arg: u32) -> u32 {
            match self.value {
                Some(v) => v,
                None => {
                    let v = (self.calculation)(arg);
                    self.value = Some(v);
                    v
                },
            }
        }
    }

    let mut cacher = Cacher::new(|x| x+1);
    assert_eq!(cacher.value(10), 11);
    assert_eq!(cacher.value(15), 16);
}

```

```

fn example2() {
    // We can also use `where` to construct `T`
    struct Cacher<T>
        where T: Fn(u32) -> u32,
    {
        calculation: T,
        value: Option<u32>,
    }

    impl<T> Cacher<T>
        where T: Fn(u32) -> u32,
    {
        fn new(calculation: T) -> Cacher<T> {
            Cacher {
                calculation,
                value: None,
            }
        }

        fn value(&mut self, arg: u32) -> u32 {

```

```

        match self.value {
            Some(v) => v,
            None => {
                let v = (self.calculation)(arg);
                self.value = Some(v);
                v
            },
        }
    }
}

let mut cacher = Cacher::new(|x| x+1);
assert_eq!(cacher.value(20), __);
assert_eq!(cacher.value(25), __);
}

```

```

fn main() {
    example1();
    example2();

    println!("Success!");
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Trait Object

In [traits chapter](#) we have seen that we can't use `impl Trait` when returning multiple types.

Another limitation of arrays is that they can only store elements of one type. Using enums is not a bad solution when we have a fixed set of types at compile time, but trait objects would be more flexible and powerful.

Returning Traits with `dyn`

The Rust compiler needs to know how much space a function's return type requires. Because the different implementations of a trait probably uses different amounts of memory, functions need to either return a concrete type or the same type when using `impl Trait`, or return a trait object with `dyn`.

1. 🌟🌟🌟


```

|
trait Bird {
    fn quack(&self) -> String;
}

struct Duck;
impl Duck {
    fn swim(&self) {
        println!("Look, the duck is swimming")
    }
}

struct Swan;
impl Swan {
    fn fly(&self) {
        println!("Look, the duck.. oh sorry, the swan is flying")
    }
}

impl Bird for Duck {
    fn quack(&self) -> String{
        "duck duck".to_string()
    }
}

impl Bird for Swan {
    fn quack(&self) -> String{
        "swan swan".to_string()
    }
}

fn main() {
    // FILL in the blank.
    let duck = __;
    duck.swim();

    let bird = hatch_a_bird(2);
    // This bird has forgotten how to swim, so below line will cause an error.
    // bird.swim();
    // But it can quak.
    assert_eq!(bird.quack(), "duck duck");

    let bird = hatch_a_bird(1);
    // This bird has forgotten how to fly, so below line will cause an error.
    // bird.fly();
    // But it can quak too.
    assert_eq!(bird.quack(), "swan swan");

    println!("Success!");
}

// IMPLEMENT this function.
fn hatch_a_bird...

```

Array with trait objects

2. 🌟🌟

```
trait Bird {
    fn quack(&self);
}

struct Duck;
impl Duck {
    fn fly(&self) {
        println!("Look, the duck is flying")
    }
}

struct Swan;
impl Swan {
    fn fly(&self) {
        println!("Look, the duck.. oh sorry, the swan is flying")
    }
}

impl Bird for Duck {
    fn quack(&self) {
        println!("{}", "duck duck");
    }
}

impl Bird for Swan {
    fn quack(&self) {
        println!("{}", "swan swan");
    }
}

fn main() {
    // FILL in the blank to make the code work.
    let birds __;

    for bird in birds {
        bird.quack();
        // When duck and swan turn into Birds, they all forgot how to fly, only reme.
        // So, the code below will cause an error.
        // bird.fly();
    }
}
```

&dyn and Box<dyn>

3. 🌟🌟

```

|
// FILL in the blanks.
trait Draw {
    fn draw(&self) -> String;
}

impl Draw for u8 {
    fn draw(&self) -> String {
        format!("u8: {}", *self)
    }
}

impl Draw for f64 {
    fn draw(&self) -> String {
        format!("f64: {}", *self)
    }
}

fn main() {
    let x = 1.1f64;
    let y = 8u8;

    // Draw x.
    draw_with_box(__);

    // Draw y.
    draw_with_ref(&y);

    println!("Success!");
}

fn draw_with_box(x: Box<dyn Draw>) {
    x.draw();
}

fn draw_with_ref(x: __) {
    x.draw();
}

```

Static and Dynamic dispatch

When we use trait bounds on generics, the compiler generates nongeneric implementations of functions and methods for each concrete type that we use in place of a generic type parameter. The code that results from monomorphization is doing static dispatch, which is when the compiler knows what method you're calling at compile time.

When we use trait objects, Rust must use dynamic dispatch. The compiler doesn't know all the types that might be used with the code that is using trait objects, so it doesn't know which method implemented on which type to call. Instead, at runtime, Rust uses the pointers inside

the trait object to know which method to call. There is a runtime cost when this lookup happens that doesn't occur with static dispatch. Dynamic dispatch also prevents the compiler from choosing to inline a method's code, which in turn prevents some optimizations.

However, we do get extra flexibility when using dynamic dispatch.

4. 🌟🌟

```
|  
trait Foo {  
    fn method(&self) -> String;  
}  
  
impl Foo for u8 {  
    fn method(&self) -> String { format!("u8: {}", *self) }  
}  
  
impl Foo for String {  
    fn method(&self) -> String { format!("string: {}", *self) }  
}  
  
// IMPLEMENT below with generics.  
fn static_dispatch...  
  
// Implement below with trait objects.  
fn dynamic_dispatch...  
  
fn main() {  
    let x = 5u8;  
    let y = "Hello".to_string();  
  
    static_dispatch(x);  
    dynamic_dispatch(&y);  
  
    println!("Success!");  
}
```

Object safe

You can only make object-safe traits into trait objects. A trait is object safe if all the methods defined in the trait have the following properties:

- The return type isn't `Self`.
- There are no generic type parameters.

5. 🌟🌟🌟🌟

```

|
// Use at least two approaches to make it work.
// DON'T add/remove any code line.
trait MyTrait {
    fn f(&self) -> Self;
}

impl MyTrait for u32 {
    fn f(&self) -> Self { 42 }
}

impl MyTrait for String {
    fn f(&self) -> Self { self.clone() }
}

fn my_function(x: Box<dyn MyTrait>) {
    x.f()
}

fn main() {
    my_function(Box::new(13_u32));
    my_function(Box::new(String::from("abc")));

    println!("Success!");
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Advance Traits

Associated types

The use of "Associated types" improves the overall readability of code by moving inner types locally into a trait as output types. For example :

```
pub trait CacheableItem: Clone + Default + fmt::Debug + Decodable + Encodable {  
    type Address: AsRef<[u8]> + Clone + fmt::Debug + Eq + Hash;  
    fn is_null(&self) -> bool;  
}
```

Using of `Address` is much more clearer and convenient than `AsRef<[u8]> + Clone + fmt::Debug + Eq + Hash`.

1. 🌟🌟🌟

```

struct Container(i32, i32);

// USING associated types to re-implement trait Contains.
// trait Contains {
//     type A;
//     type B;

trait Contains<A, B> {
    fn contains(&self, _: &A, _: &B) -> bool;
    fn first(&self) -> i32;
    fn last(&self) -> i32;
}

impl Contains<i32, i32> for Container {
    fn contains(&self, number_1: &i32, number_2: &i32) -> bool {
        (&self.0 == number_1) && (&self.1 == number_2)
    }
    // Grab the first number.
    fn first(&self) -> i32 { self.0 }

    // Grab the last number.
    fn last(&self) -> i32 { self.1 }
}

fn difference<A, B, C: Contains<A, B>>(container: &C) -> i32 {
    container.last() - container.first()
}

fn main() {
    let number_1 = 3;
    let number_2 = 10;

    let container = Container(number_1, number_2);

    println!("Does container contain {} and {}: {}",
        &number_1, &number_2,
        container.contains(&number_1, &number_2));
    println!("First number: {}", container.first());
    println!("Last number: {}", container.last());

    println!("The difference is: {}", difference(&container));
}

```

Default Generic Type Parameters

When we use generic type parameters, we can specify a default concrete type for the generic type. This eliminates the need for implementors of the trait to specify a concrete type if the default type works.

```

use std::ops::Sub;

#[derive(Debug, PartialEq)]
struct Point<T> {
    x: T,
    y: T,
}

// FILL in the blank in three ways: two of them use the default generic parameters,
// Notice that the implementation uses the associated type `Output`.
impl __ {
    type Output = Self;

    fn sub(self, other: Self) -> Self::Output {
        Point {
            x: self.x - other.x,
            y: self.y - other.y,
        }
    }
}

fn main() {
    assert_eq!(Point { x: 2, y: 3 } - Point { x: 1, y: 0 },
        Point { x: 1, y: 3 });

    println!("Success!");
}

```



Fully Qualified Syntax

Nothing in Rust prevents a trait from having a method with the same name as another trait's method, nor does Rust prevent you from implementing both traits on one type. It's also possible to implement a method directly on the type with the same name as methods from traits.

When calling methods with the same name, we have to use Fully Qualified Syntax.

Example

```
trait UsernameWidget {  
    // Get the selected username out of this widget  
    fn get(&self) -> String;  
}  
  
trait AgeWidget {  
    // Get the selected age out of this widget  
    fn get(&self) -> u8;  
}  
  
// A form with both a UsernameWidget and an AgeWidget.  
struct Form {  
    username: String,  
    age: u8,  
}  
  
impl UsernameWidget for Form {  
    fn get(&self) -> String {  
        self.username.clone()  
    }  
}  
  
impl AgeWidget for Form {  
    fn get(&self) -> u8 {  
        self.age  
    }  
}  
  
fn main() {  
    let form = Form{  
        username: "rustacean".to_owned(),  
        age: 28,  
    };  
  
    // If you uncomment this line, you'll get an error saying  
    // "multiple `get` found". Because, after all, there are multiple methods  
    // named `get`.  
    // println!("{}", form.get());  
  
    let username = UsernameWidget::get(&form);  
    assert_eq!("rustacean".to_owned(), username);  
    let age = AgeWidget::get(&form); // You can also use `<Form as AgeWidget>::get`  
    assert_eq!(28, age);  
  
    println!("Success!");  
}
```

Exercise

3. 🌟🌟

```

trait Pilot {
    fn fly(&self) -> String;
}

trait Wizard {
    fn fly(&self) -> String;
}

struct Human;

impl Pilot for Human {
    fn fly(&self) -> String {
        String::from("This is your captain speaking.")
    }
}

impl Wizard for Human {
    fn fly(&self) -> String {
        String::from("Up!")
    }
}

impl Human {
    fn fly(&self) -> String {
        String::from("*waving arms furiously*")
    }
}

fn main() {
    let person = Human;

    assert_eq!(_, "This is your captain speaking.");
    assert_eq!(_, "Up!");

    assert_eq!(_, "*waving arms furiously*");

    println!("Success!");
}

```

Supertraits

Sometimes, you might need one trait to use another trait's functionality(like the "inheritance" in other languages). In this case, you need to rely on the dependent trait also being implemented. The trait you rely on is a `supertrait` of the trait you're implementing.

4. 🌟🌟🌟

```

|
trait Person {
    fn name(&self) -> String;
}

// Person is a supertrait of Student.
// Implementing Student requires you to also impl Person.
trait Student: Person {
    fn university(&self) -> String;
}

trait Programmer {
    fn fav_language(&self) -> String;
}

// CompSciStudent (computer science student) is a subtrait of both Programmer
// and Student. Implementing CompSciStudent requires you to impl both supertraits.
trait CompSciStudent: Programmer + Student {
    fn git_username(&self) -> String;
}

fn comp_sci_student_greeting(student: &dyn CompSciStudent) -> String {
    format!(
        "My name is {} and I attend {}. My favorite language is {}. My Git username
        student.name(),
        student.university(),
        student.fav_language(),
        student.git_username()
    )
}

struct CSStudent {
    name: String,
    university: String,
    fav_language: String,
    git_username: String
}

// IMPLEMENT the necessary traits for CSStudent to make the code work
impl ...

fn main() {
    let student = CSStudent {
        name: "Sunfei".to_string(),
        university: "XXX".to_string(),
        fav_language: "Rust".to_string(),
        git_username: "sunface".to_string()
    };

    // FILL in the blank
    println!("{}", comp_sci_student_greeting(_));
}

```

Orphan Rules

We can't implement external traits on external types. For example, we can't implement the `Display` trait on `Vec<T>` within our own crate, because `Display` and `Vec<T>` are defined in the standard library and aren't local to our crate.

This restriction is often called the orphan rule, so named because the parent type is not present. This rule ensures that other people's code can't break your code and vice versa.

It's possible to get around this restriction using the newtype pattern, which involves creating a new type in a tuple struct.

5. 🌟🌟

```
use std::fmt;

// DEFINE a newtype `Pretty` here

impl fmt::Display for Pretty {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}\{}", self.0.clone() + ", world")
    }
}

fn main() {
    let w = Pretty("hello".to_string());
    println!("w = {}", w);
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Collection Types

Learning resources:

- English: [Rust Book Chapter 8](#)
- 繁體中文: [Rust 繁體中文 - 集合](#)

String

`std::string::String` is a UTF-8 encoded, growable string. It is the most common string type we used in daily development, it also has ownership over the string contents.

Basic operations

1. 🌟🌟

```
|  
// FILL in the blanks and FIX errors  
// 1. Don't use `to_string()`  
// 2. Don't add/remove any code line  
fn main() {  
    let mut s: String = "hello, ";  
    s.push_str("world".to_string());  
    s.push(__);  
  
    move_ownership(s);  
  
    assert_eq!(s, "hello, world!");  
  
    println!("Success!");  
}  
  
fn move_ownership(s: String) {  
    println!("ownership of \"{}\" is moved here!", s)  
}
```

String and &str

A `String` is stored as a vector of bytes (`Vec<u8>`), but guaranteed to always be a valid UTF-8 sequence. `String` is heap allocated, growable and not null terminated.

`&str` is a slice (`&[u8]`) that always points to a valid UTF-8 sequence, and can be used to view into a `String`, just like `&[T]` is a view into `Vec<T>`.

2. 🌟🌟

```
// FILL in the blanks
fn main() {
    let mut s = String::from("hello, world");

    let slice1: &str = __; // In two ways
    assert_eq!(slice1, "hello, world");

    let slice2 = __;
    assert_eq!(slice2, "hello");

    let slice3: __ = __;
    slice3.push('!');
    assert_eq!(slice3, "hello, world!");

    println!("Success!");
}
```

3. 🌟🌟

```
// Question: how many heap allocations are happening here?
// Your answer:
fn main() {
    // Create a String type based on `&str`
    // The type of string literals is `&str`
    let s: String = String::from("hello, world!");

    // Create a slice point to String `s`
    let slice: &str = &s;

    // Create a String type based on the recently created slice
    let s: String = slice.to_string();

    assert_eq!(s, "hello, world!");

    println!("Success!");
}
```

UTF-8 & Indexing

Strings are always valid UTF-8. This has a few implications:

- The first of which is that if you need a non-UTF-8 string, consider [OsString](#). It is similar, but without the UTF-8 constraint.
- The second implication is that you cannot index into a String.

Indexing is intended to be a constant-time operation, but UTF-8 encoding does not allow us to do this. Furthermore, it's not clear what sort of thing the index should return: a byte, a codepoint, or a grapheme cluster. The `bytes` and `chars` methods return iterators over the first two, respectively.

4. 🌟🌟🌟 You can't use index to access a char in a string, but you can use slice
 &s1[start..end] .

```
|  
// FILL in the blank and FIX errors  
fn main() {  
    let s = String::from("hello, 🌟 🌟");  
    let slice1 = s[0]; //tips: `h` only takes 1 byte in UTF8 format  
    assert_eq!(slice1, "h");  
  
    let slice2 = &s[3..5]; // Tips: `🌟` takes 3 bytes in UTF8 format  
    assert_eq!(slice2, "🌟");  
  
    // Iterate through all chars in s  
    for (i, c) in s.chars() {  
        if i == 7 {  
            assert_eq!(c, '🌟');  
        }  
    }  
  
    println!("Success!");  
}
```

UTF8_slice

You can use `utf8_slice` to slice UTF8 string, it can index chars instead of bytes.

Example

```
use utf8_slice;  
fn main() {  
    let s = "The 🚀 goes to the 🌑!";  
  
    let rocket = utf8_slice::slice(s, 4, 5);  
    // Will equal "🚀"  
}
```

5. 🌟🌟🌟

Tips: maybe you need `from_utf8` method

```

|
// FILL in the blanks
fn main() {
    let mut s = String::new();
    __;

    // Some bytes, in a vector
    let v = vec![104, 101, 108, 108, 111];

    // Turn a byte's vector into a String
    let s1 = __;

    assert_eq!(s, s1);

    println!("Success!");
}

```

Representation

A String is made up of three components: a pointer to some bytes, a length, and a capacity.

The pointer points to an internal buffer String uses to store its data. The length is the number of bytes currently stored in the buffer(always stored on the heap), and the capacity is the size of the buffer in bytes. As such, the length will always be less than or equal to the capacity.

6. 🌟🌟 If a String has enough capacity, adding elements to it will not re-allocate

```

|
// Modify the code below to print out:
// 25
// 25
// 25
// Here, there's no need to allocate more memory inside the loop.
fn main() {
    let mut s = String::new();

    println!("{}", s.capacity());

    for _ in 0..2 {
        s.push_str("hello");
        println!("{}", s.capacity());
    }

    println!("Success!");
}

```

7. 🌟🌟🌟

```

|
// FILL in the blanks
use std::mem;

fn main() {
    let story = String::from("Rust By Practice");

    // Prevent automatically dropping of the String's data
    let mut story = mem::ManuallyDrop::new(story);

    let ptr = story.__();
    let len = story.__();
    let capacity = story.__();

    assert_eq!(16, len);

    // We can rebuild a String out of ptr, len, and capacity. This is all
    // unsafe because we are responsible for making sure the components are
    // valid:
    let s = unsafe { String::from_raw_parts(ptr, len, capacity) };

    assert_eq!(*story, s);

    println!("Success!");
}

```

Common methods

More exercises of String methods can be found [here](#).

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Vector

Vectors are resizable arrays. Like slices, their size is not known at compile time, but they can grow or shrink at any time.

Basic Operations

1. 🌟🌟🌟

```
|  
fn main() {  
    let arr: [u8; 3] = [1, 2, 3];  
  
    let v = Vec::from(arr);  
    is_vec(&v);  
  
    let v = vec![1, 2, 3];  
    is_vec(&v);  
  
    // vec!(..) and vec![..] are same macros, so  
    let v = vec!(1, 2, 3);  
    is_vec(&v);  
  
    // In code below, v is Vec<[u8; 3]> , not Vec<u8>  
    // USE Vec::new and `for` to rewrite the below code  
    let v1 = vec!(arr);  
    is_vec(&v1);  
  
    assert_eq!(v, v1);  
  
    println!("Success!");  
}  
  
fn is_vec(v: &Vec<u8>) {}
```

2. 🌟🌟 A Vec can be extended with `extend` method

```

|
// FILL in the blank
fn main() {
    let mut v1 = Vec::from([1, 2, 4]);
    v1.pop();
    v1.push(3);

    let mut v2 = Vec::new();
    v2.__;

    assert_eq!(v1, v2);

    println!("Success!");
}

```

Turn X Into Vec

3. 🌟🌟🌟

```

|
// FILL in the blanks
fn main() {
    // Array -> Vec
    // impl From<[T; N]> for Vec
    let arr = [1, 2, 3];
    let v1 = __ (arr);
    let v2: Vec<i32> = arr.__ ();

    assert_eq!(v1, v2);

    // String -> Vec
    // impl From<String> for Vec
    let s = "hello".to_string();
    let v1: Vec<u8> = s.__ ();

    let s = "hello".to_string();
    let v2 = s.into_bytes();
    assert_eq!(v1, v2);

    // impl<'_> From<&'_ str> for Vec
    let s = "hello";
    let v3 = Vec::__ (s);
    assert_eq!(v2, v3);

    // Iterators can be collected into vectors
    let v4: Vec<i32> = [0; 10].into_iter().collect();
    assert_eq!(v4, vec![0; 10]);

    println!("Success!");
}

```

Indexing

4. 🌟🌟🌟

```
|  
// FIX the error and IMPLEMENT the code  
fn main() {  
    let mut v = Vec::from([1, 2, 3]);  
    for i in 0..5 {  
        println!("{:?}", v[i])  
    }  
  
    for i in 0..5 {  
        // IMPLEMENT the code here...  
    }  
  
    assert_eq!(v, vec![2, 3, 4, 5, 6]);  
  
    println!("Success!");  
}
```

Slicing

Immutable or mutable slices of Vectors can be taken, using `&` or `&mut`, respectively.

In Rust, it's more common to pass immutable slices as arguments rather than vectors when you just want to provide read access, as this is more flexible (no move) and efficient (no copy). The same goes for `String` and `&str`.

5. 🌟🌟

```

|
// FIX the errors
fn main() {
    let mut v = vec![1, 2, 3];

    let slice1 = &v[..];
    // Out of bounds will cause a panic
    // You must use `v.len` here
    let slice2 = &v[0..4];

    assert_eq!(slice1, slice2);

    // A slice can also be mutable, in which
    // case mutating it will mutate its underlying Vec.
    // Note: slice and &Vec are different
    let vec_ref: &mut Vec<i32> = &mut v;
    (*vec_ref).push(4);
    let slice3 = &mut v[0..3];
    slice3[3] = 42;

    assert_eq!(slice3, &[1, 2, 3, 42]);
    assert_eq!(v, &[1, 2, 3, 42]);

    println!("Success!");
}

```

Capacity

The capacity of a vector is the amount of space allocated for any future elements that will be added onto the vector. This is not to be confused with the length of a vector, which specifies the number of actual elements within the vector. If a vector's length exceeds its capacity, its capacity will automatically be increased, but its elements will have to be reallocated.

For example, a vector with capacity 10 and length 0 would be an empty vector with space for 10 more elements. Pushing 10 or fewer elements onto the vector will not change its capacity or cause reallocation to occur. However, if the vector's length is increased to 11, it will have to reallocate, which can be slow. For this reason, it is recommended to use `Vec::with_capacity` whenever possible to specify how big the vector is expected to get.

6. 🌟🌟

```
// FIX the errors
fn main() {
    let mut vec = Vec::with_capacity(10);

    // The vector contains no items, even though it has capacity for more
    assert_eq!(vec.len(), 0);
    assert_eq!(vec.capacity(), 10);

    // These are all done without reallocating...
    for i in 0..10 {
        vec.push(i);
    }
    assert_eq!(vec.len(), 10);
    assert_eq!(vec.capacity(), 10);

    // ...but this may make the vector reallocate
    vec.push(11);
    assert_eq!(vec.len(), 11);
    assert!(vec.capacity() >= 11);

    // Fill in an appropriate value to make the `for` done without reallocating
    let mut vec = Vec::with_capacity(100);
    for i in 0..100 {
        vec.push(i);
    }

    assert_eq!(vec.len(), 100);
    assert_eq!(vec.capacity(), 100);

    println!("Success!");
}
```

Store distinct types in Vector

The elements in a vector must be the same type, for example , the code below will cause an error:

```
fn main() {
    let v = vec![1, 2.0, 3];
}
```

But we can use enums or trait objects to store distinct types.

7. 🌟🌟🌟

```

#[derive(Debug)]
enum IpAddr {
    V4(String),
    V6(String),
}

fn main() {
    // FILL in the blank
    let v : Vec<IpAddr>= __;

    // Comparing two enums need to derive the PartialEq trait
    assert_eq!(v[0], IpAddr::V4("127.0.0.1".to_string()));
    assert_eq!(v[1], IpAddr::V6("::1".to_string()));

    println!("Success!");
}

```

8. 🌟🌟

```

trait IpAddr {
    fn display(&self);
}

struct V4(String);
impl IpAddr for V4 {
    fn display(&self) {
        println!("ipv4: {:?}",self.0)
    }
}

struct V6(String);
impl IpAddr for V6 {
    fn display(&self) {
        println!("ipv6: {:?}",self.0)
    }
}

fn main() {
    // FILL in the blank
    let v: __= vec![
        Box::new(V4("127.0.0.1".to_string())),
        Box::new(V6("::1".to_string())),
    ];

    for ip in v {
        ip.display();
    }
}

```


HashMap

Where vectors store values by an integer index, HashMaps store values by key. It is a hash map implemented with quadratic probing and SIMD lookup. By default, `HashMap` uses a hashing algorithm selected to provide resistance against HashDoS attacks.

The default hashing algorithm is currently `SipHash 1-3`, though this is subject to change at any point in the future. While its performance is very competitive for medium sized keys, other hashing algorithms will outperform it for small keys such as integers as well as large keys such as long strings, though those algorithms will typically not protect against attacks such as HashDoS.

The hash table implementation is a Rust port of Google's [SwissTable](#). The original C++ version of SwissTable can be found [here](#), and this [CppCon talk](#) gives an overview of how the algorithm works.

Basic Operations

1. 🌟🌟

```
|  
// FILL in the blanks and FIX the errors  
use std::collections::HashMap;  
fn main() {  
    let mut scores = HashMap::new();  
    scores.insert("Sunface", 98);  
    scores.insert("Daniel", 95);  
    scores.insert("Ashley", 69.0);  
    scores.insert("Katie", "58");  
  
    // Get returns an Option<V>  
    let score = scores.get("Sunface");  
    assert_eq!(score, Some(98));  
  
    if scores.contains_key("Daniel") {  
        // Indexing returns a value V  
        let score = scores["Daniel"];  
        assert_eq!(score, __);  
        scores.remove("Daniel");  
    }  
  
    assert_eq!(scores.len(), __);  
  
    for (name, score) in scores {  
        println!("The score of {} is {}", name, score);  
    }  
}
```

2. 🌟🌟

```
|  
use std::collections::HashMap;  
fn main() {  
    let teams = [  
        ("Chinese Team", 100),  
        ("American Team", 10),  
        ("France Team", 50),  
    ];  
  
    let mut teams_map1 = HashMap::new();  
    for team in &teams {  
        teams_map1.insert(team.0, team.1);  
    }  
  
    // IMPLEMENT team_map2 in two ways  
    // Tips: one of the approaches is to use `collect` method  
    let teams_map2...  
  
    assert_eq!(teams_map1, teams_map2);  
  
    println!("Success!");  
}
```

3. 🌟🌟

```

|
// FILL in the blanks
use std::collections::HashMap;
fn main() {
    // Type inference lets us omit an explicit type signature (which
    // would be `HashMap<&str, u8>` in this example).
    let mut player_stats = HashMap::new();

    // Insert a key only if it doesn't already exist
    player_stats.entry("health").or_insert(100);

    assert_eq!(player_stats["health"], 100);

    // Insert a key using a function that provides a new value only if it
    // doesn't already exist
    player_stats.entry("health").or_insert_with(random_stat_buff);
    assert_eq!(player_stats["health"], 100);

    // Ensures a value is in the entry by inserting the default if empty, and return
    // a mutable reference to the value in the entry.
    let health = player_stats.entry("health").or_insert(50);
    assert_eq!(health, 50);
    *health -= 50;
    assert_eq!(*health, 50);

    println!("Success!");
}

fn random_stat_buff() -> u8 {
    // Could actually return some random value here - let's just return
    // some fixed value for now
    42
}

```

Requirements of HashMap key

Any type that implements the `Eq` and `Hash` traits can be a key in `HashMap`. This includes:

- `bool` (though not very useful since there is only two possible keys)
- `int`, `uint`, and all variations thereof
- `String` and `&str` (tips: you can have a `HashMap` keyed by `String` and call `.get()` with an `&str`)

Note that `f32` and `f64` do not implement `Hash`, likely because floating-point precision errors would make using them as hashmap keys horribly error-prone.

All collection classes implement `Eq` and `Hash` if their contained type also respectively implements `Eq` and `Hash`. For example, `Vec<T>` will implement `Hash` if `T` implements `Hash`.

```

|
// FIX the errors
// Tips: `derive` is usually a good way to implement some common used traits
use std::collections::HashMap;

struct Viking {
    name: String,
    country: String,
}

impl Viking {
    /// Creates a new Viking.
    fn new(name: &str, country: &str) -> Viking {
        Viking {
            name: name.to_string(),
            country: country.to_string(),
        }
    }
}

fn main() {
    // Use a HashMap to store the vikings' health points.
    let vikings = HashMap::from([
        (Viking::new("Einar", "Norway"), 25),
        (Viking::new("Olaf", "Denmark"), 24),
        (Viking::new("Harald", "Iceland"), 12),
    ]);

    // Use derived implementation to print the status of the vikings.
    for (viking, health) in &vikings {
        println!("{:?} has {} hp", viking, health);
    }
}

```

Capacity

Like vectors, HashMaps are growable, but HashMaps can also shrink themselves when they have excess space. You can create a `HashMap` with a certain starting capacity using `HashMap::with_capacity(uint)`, or use `HashMap::new()` to get a `HashMap` with a default initial capacity (recommended).

Example

```
|
use std::collections::HashMap;
fn main() {
    let mut map: HashMap<i32, i32> = HashMap::with_capacity(100);
    map.insert(1, 2);
    map.insert(3, 4);
    // Indeed ,the capacity of HashMap is not 100, so we can't compare the equality
    assert!(map.capacity() >= 100);

    // Shrinks the capacity of the map with a lower limit. It will drop
    // down no lower than the supplied limit while maintaining the internal rules
    // and possibly leaving some space in accordance with the resize policy.

    map.shrink_to(50);
    assert!(map.capacity() >= 50);

    // Shrinks the capacity of the map as much as possible. It will drop
    // down as much as possible while maintaining the internal rules
    // and possibly leaving some space in accordance with the resize policy.
    map.shrink_to_fit();
    assert!(map.capacity() >= 2);
    println!("Success!");
}
```

Ownership

For types that implement the `Copy` trait, like `i32`, the values are copied into `HashMap`. For owned values like `String`, the values will be moved and `HashMap` will be the owner of those values.

5. 🌟🌟

```
// FIX the errors with least changes
// DON'T remove any code line
use std::collections::HashMap;
fn main() {
    let v1 = 10;
    let mut m1 = HashMap::new();
    m1.insert(v1, v1);
    println!("v1 is still usable after inserting to hashmap : {}", v1);

    let v2 = "hello".to_string();
    let mut m2 = HashMap::new();
    // Ownership moved here
    m2.insert(v2, v1);

    assert_eq!(v2, "hello");

    println!("Success!");
}
```

Third-party Hash libs

If the performance of `SipHash` 1-3 doesn't meet your requirements, you can find replacements in crates.io or github.com.

The usage of third-party hash looks like this:

```
use std::hash::BuildHasherDefault;
use std::collections::HashMap;
// Introduce a third party hash function
use twox_hash::XxHash64;

let mut hash: HashMap<_, _, BuildHasherDefault<XxHash64>> = Default::default();
hash.insert(42, "the answer");
assert_eq!(hash.get(&42), Some(&"the answer"));
```

Type Conversion

Learning resources:

- English: [Standary library](#)
- 简体中文: [Rust 简体中文 - 简体中文](#)

Convert by as

Rust provides no implicit type conversion(coercion) between primitive types. But explicit type conversions can be performed using the `as` keyword.

1. 🌟

```
// FIX the errors and FILL in the blank
// DON'T remove any code
fn main() {
    let decimal = 97.123_f32;

    let integer: __ = decimal as u8;

    let c1: char = decimal as char;
    let c2 = integer as char;

    assert_eq!(integer, 'b' as u8);

    println!("Success!");
}
```

2. 🌟🌟 By default, overflow will cause compile errors, but we can add an global annotation to suppress these errors.

```
fn main() {
    assert_eq!(u8::MAX, 255);
    // The max of `u8` is 255 as shown above.
    // so the below code will cause an overflow error: literal out of range for `u8`
    // PLEASE looking for clues within compile errors to FIX it.
    // DON'T modify any code in main.
    let v = 1000 as u8;

    println!("Success!");
}
```

3. 🌟🌟 When casting any value to an unsigned type `T`, `T::MAX + 1` is added or subtracted until the value fits into the new type.


```
fn main() {
    assert_eq!(1000 as u16, __);

    assert_eq!(1000 as u8, __);

    // For positive numbers, this is the same as the modulus
    println!("1000 mod 256 is : {}", 1000 % 256);

    assert_eq!(-1_i8 as u8, __);

    // Since Rust 1.45, the `as` keyword performs a *saturating cast*
    // when casting from float to int. If the floating point value exceeds
    // the upper bound or is less than the lower bound, the returned value
    // will be equal to the bound crossed.
    assert_eq!(300.1_f32 as u8, __);
    assert_eq!(-100.1_f32 as u8, __);

    // This behavior incurs a small runtime cost and can be avoided
    // with unsafe methods, however the results might overflow and
    // return **unsound values**. Use these methods wisely:
    unsafe {
        // 300.0 is 44
        println!("300.0 is {}", 300.0_f32.to_int_unchecked::<u8>());
        // -100.0 as u8 is 156
        println!("-100.0 as u8 is {}", (-100.0_f32).to_int_unchecked::<u8>());
        // nan as u8 is 0
        println!("nan as u8 is {}", f32::NAN.to_int_unchecked::<u8>());
    }
}
```

4. 🌟🌟🌟 Raw pointers can be converted to memory address (integer) and vice versa.

```
// FILL in the blanks
fn main() {
    let mut values: [i32; 2] = [1, 2];
    let p1: *mut i32 = values.as_mut_ptr();
    let first_address: usize = p1 __;
    let second_address = first_address + 4; // 4 == std::mem::size_of::<i32>()
    let p2: *mut i32 = second_address __; // p2 points to the 2nd element in values
    unsafe {
        // Add one to the second element
        __
    }

    assert_eq!(values[1], 3);

    println!("Success!");
}
```

5. 🌟🌟🌟

```
fn main() {  
    let arr : [u64; 13] = [0; 13];  
    assert_eq!(std::mem::size_of_val(&arr), 8 * 13);  
    let a: *const [u64] = &arr;  
    let b = a as *const [u8];  
    unsafe {  
        assert_eq!(std::mem::size_of_val(&*b), __)  
    }  
  
    println!("Success!");  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

From/Into

The `From` trait allows for a type to define how to create itself from another type, hence providing a very simple mechanism for converting between several types.

The `From` and `Into` traits are inherently linked, and this is actually part of its implementation. It means if we write something like this: `impl From<T> for U`, then we can use `let u: U = U::from(T)` or `let u:U = T.into()`.

The `Into` trait is simply the reciprocal of the `From` trait. That is, if you have implemented the `From` trait for your type, then the `Into` trait will be automatically implemented for the same type.

Using the `Into` trait will typically require the type annotations as the compiler is unable to determine this most of the time.

For example, we can easily convert `&str` into `String` :

```
fn main() {  
    let my_str = "hello";  
  
    // three conversions below all depends on the fact: String implements  
    From<&str>:  
    let string1 = String::from(my_str);  
    let string2 = my_str.to_string();  
    // Explicit type annotation is required here  
    let string3: String = my_str.into();  
}
```

Because the standard library has already implemented this for us: `impl From<&'_ str> for String`.

Some implementations of `From` trait can be found [here](#).

1. 🌟🌟🌟

```
fn main() {
    // impl From<bool> for i32
    let i1: i32 = false.into();
    let i2: i32 = i32::from(false);
    assert_eq!(i1, i2);
    assert_eq!(i1, 0);

    // FIX the error in two ways
    /* 1. use a similar type which `impl From<char>`, maybe you
       should check the docs mentioned above to find the answer */
    // 2. a keyword from the last chapter
    let i3: i32 = 'a'.into();

    // FIX the error in two ways
    let s: String = 'a' as String;

    println!("Success!");
}
```

Implement From for custom types

2. 🌟🌟

*// From is now included in `std::prelude`, so there is no need to introduce it into
 // use std::convert::From;*

```
#[derive(Debug)]
struct Number {
    value: i32,
}

impl From<i32> for Number {
    // IMPLEMENT `from` method
}

// FILL in the blanks
fn main() {
    let num = __(30);
    assert_eq!(num.value, 30);

    let num: Number = __;
    assert_eq!(num.value, 30);

    println!("Success!");
}
```

3. 🌟🌟🌟 When performing error handling it is often useful to implement `From` trait for our own error type. Then we can use `?` to automatically convert the underlying error type to our own error type.

```

use std::fs;
use std::io;
use std::num;

enum CliError {
    IOError(io::Error),
    ParseError(num::ParseIntError),
}

impl From<io::Error> for CliError {
    // IMPLEMENT from method
}

impl From<num::ParseIntError> for CliError {
    // IMPLEMENT from method
}

fn open_and_parse_file(file_name: &str) -> Result<i32, CliError> {
    // ? automatically converts io::Error to CliError
    let contents = fs::read_to_string(&file_name)?;
    // num::ParseIntError -> CliError
    let num: i32 = contents.trim().parse()?;
    Ok(num)
}

fn main() {
    println!("Success!");
}

```

TryFrom/TryInto

Similar to `From` and `Into`, `TryFrom` and `TryInto` are generic traits for converting between types.

Unlike `From/Into`, `TryFrom` and `TryInto` are used for fallible conversions and return a `Result` instead of a plain value.

4. 🌟🌟

```
// TryFrom and TryInto are included in `std::prelude`, so there is no need to introd
// use std::convert::TryInto;

fn main() {
    let n: i16 = 256;

    // Into trait has a method `into`,
    // hence TryInto has a method ?
    let n: u8 = match n.__(()) {
        Ok(n) => n,
        Err(e) => {
            println!("there is an error when converting: {:?}, but we catch it", e.t
            0
        }
    };

    assert_eq!(n, 256);

    println!("Success!");
}

```

5. 🌟🌟🌟

```
#[derive(Debug, PartialEq)]
struct EvenNum(i32);

impl TryFrom<i32> for EvenNum {
    type Error = ();

    // IMPLEMENT `try_from`
    fn try_from(value: i32) -> Result<Self, Self::Error> {
        if value % 2 == 0 {
            Ok(EvenNum(value))
        } else {
            Err(())
        }
    }
}

fn main() {
    assert_eq!(EvenNum::try_from(8), Ok(EvenNum(8)));
    assert_eq!(EvenNum::try_from(5), Err(()));

    // FILL in the blanks
    let result: Result<EvenNum, ()> = 8i32.try_into();
    assert_eq!(result, Ok(EvenNum(8)));
    let result: Result<EvenNum, ()> = 5i32.try_into();
    assert_eq!(result, Err(()));

    println!("Success!");
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Others

Convert any type to String

To convert any type to `String`, you can simply use the `ToString` trait for that type. Rather than doing that directly, you should implement the `fmt::Display` trait which will automatically provides `ToString` and also allows you to print the type with `println!`.

1. 🌟🌟

```
use std::fmt;

struct Point {
    x: i32,
    y: i32,
}

impl fmt::Display for Point {
    // IMPLEMENT fmt method
}

fn main() {
    let origin = Point { x: 0, y: 0 };
    // FILL in the blanks
    assert_eq!(origin.__, "The point is (0, 0)");
    assert_eq!(format!(__), "The point is (0, 0)");

    println!("Success!");
}
```

Parse a String

2. 🌟🌟🌟 We can use `parse` method to convert a `String` into a `i32` number, this is because `FromStr` is implemented for `i32` type in standard library: `impl FromStr for i32`


```
// To use `from_str` method, you need to introduce this trait into the current scope
use std::str::FromStr;
fn main() {
    let parsed: i32 = "5".__.unwrap();
    let turbo_parsed = "10".__.unwrap();
    let from_str = __.unwrap();
    let sum = parsed + turbo_parsed + from_str;
    assert_eq!(sum, 35);

    println!("Success!");
}
```

3. 🌟🌟 We can also implement the `FromStr` trait for our custom types

```
use std::str::FromStr;
use std::num::ParseIntError;

#[derive(Debug, PartialEq)]
struct Point {
    x: i32,
    y: i32
}

impl FromStr for Point {
    type Err = ParseIntError;

    fn from_str(s: &str) -> Result<Self, Self::Err> {
        let coords: Vec<&str> = s.trim_matches(|p| p == '(' || p == ')')
            .split(',')
            .map(|x| x.trim())
            .collect();

        let x_fromstr = coords[0].parse::<i32>()?;
        let y_fromstr = coords[1].parse::<i32>()?;

        Ok(Point { x: x_fromstr, y: y_fromstr })
    }
}

fn main() {
    // FILL in the blanks in two ways
    // DON'T change code anywhere else
    let p = __;
    assert_eq!(p.unwrap(), Point{ x: 3, y: 4 } );

    println!("Success!");
}
```

Deref

You can find all the examples and exercises of the `Deref` trait [here](#).

Transmute

`std::mem::transmute` is a **unsafe function** can be used to reinterpret the bits of a value of one type as another type. Both of the original and the result types must have the same size and neither of them can be invalid.

`transmute` is semantically equivalent to a bitwise move of one type into another. It copies the bits from the source value into the destination value, then forgets the original, seems equivalent to C's `memcpy` under the hood.

So, **transmute is incredibly unsafe !** The caller has to ensure all the safes himself!

Examples

1. `transmute` can be used to turn a pointer into a function pointer, this is not portable on machines where function pointer and data pointer have different sizes.

```
fn foo() -> i32 {
    0
}

fn main() {
    let pointer = foo as *const ();
    let function = unsafe {
        std::mem::transmute::<*const (), fn() -> i32>(pointer)
    };
    assert_eq!(function(), 0);
}
```

2. Extending a lifetime or shortening the lifetime of an invariant is an advanced usage of `transmute`, yeah, **very unsafe Rust!**

```
struct R<'a>(&'a i32);
unsafe fn extend_lifetime<'b>(r: R<'b>) -> R<'static> {
    std::mem::transmute::<R<'b>, R<'static>>(r)
}

unsafe fn shorten_invariant_lifetime<'b, 'c>(r: &'b mut R<'static>)
    -> &'b mut R<'c> {
    std::mem::transmute::<&'b mut R<'static>, &'b mut R<'c>>(r)
}
```

3. Rather than using `transmute`, you can use some alternatives instead.

```

fn main() {
    /*Turning raw bytes(&[u8]) to u32, f64, etc.: */
    let raw_bytes = [0x78, 0x56, 0x34, 0x12];

    let num = unsafe { std::mem::transmute::<[u8; 4], u32>(raw_bytes) };

    // Use `u32::from_ne_bytes` instead
    let num = u32::from_ne_bytes(raw_bytes);
    // Or use `u32::from_le_bytes` or `u32::from_be_bytes` to specify the endianness
    let num = u32::from_le_bytes(raw_bytes);
    assert_eq!(num, 0x12345678);
    let num = u32::from_be_bytes(raw_bytes);
    assert_eq!(num, 0x78563412);

    /*Turning a pointer into a usize: */
    let ptr = &0;
    let ptr_num_transmute = unsafe { std::mem::transmute::<&i32, usize>(ptr) };

    // Use an `as` cast instead
    let ptr_num_cast = ptr as *const i32 as usize;

    /*Turning an &mut T into an &mut U: */
    let ptr = &mut 0;
    let val_transmuted = unsafe { std::mem::transmute::<&mut i32, &mut u32>(ptr) };

    // Now, put together `as` and reborrowing - note the chaining of `as`
    // `as` is not transitive
    let val_casts = unsafe { &mut *(ptr as *mut i32 as *mut u32) };

    /*Turning an &str into a &[u8]: */
    // This is not a good way to do this.
    let slice = unsafe { std::mem::transmute::<&str, &[u8]>("Rust") };
    assert_eq!(slice, &[82, 117, 115, 116]);

    // You could use `str::as_bytes`
    let slice = "Rust".as_bytes();
    assert_eq!(slice, &[82, 117, 115, 116]);

    // Or, just use a byte string, if you have control over the string
    // literal
    assert_eq!(b"Rust", &[82, 117, 115, 116]);
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Result and panic

Learning resources:

- English: [Rust Book 9.1, 9.2](#)
- 简体中文: [Rust 简体中文 - 快速入门](#)

panic!

The simplest error handling mechanism is to use `panic`. It just prints an error message and starts unwinding the stack, finally exit the current thread:

- if panic occurred in `main` thread, then the program will be exited.
- if in spawned thread, then this thread will be terminated, but the program won't

1. 🌟🌟

```
|  
// FILL the blanks  
fn drink(beverage: &str) {  
    if beverage == "lemonade" {  
        println!("Success!");  
        // IMPLEMENT the below code  
    } --  
  
    println!("Exercise Failed if printing out this line!");  
}  
  
fn main() {  
    drink(__);  
  
    println!("Exercise Failed if printing out this line!");  
}
```

common panic cases

2. 🌟🌟

```
// MAKE the code work by fixing all panics
fn main() {
    assert_eq!("abc".as_bytes(), [96, 97, 98]);

    let v = vec![1, 2, 3];
    let ele = v[3];
    // unwrap may panic when get return a None
    let ele = v.get(3).unwrap();

    // Sometimes, the compiler is unable to find the overflow errors for you in comp
    let v = production_rate_per_hour(2);

    // because of the same reason as above, we have to wrap it in a function to make
    divide(15, 0);

    println!("Success!")
}

fn divide(x:u8, y:u8) {
    println!("{}", x / y)
}

fn production_rate_per_hour(speed: u8) -> f64 {
    let cph: u8 = 221;
    match speed {
        1..=4 => (speed * cph) as f64,
        5..=8 => (speed * cph) as f64 * 0.9,
        9..=10 => (speed * cph) as f64 * 0.77,
        _ => 0 as f64,
    }
}

pub fn working_items_per_minute(speed: u8) -> u32 {
    (production_rate_per_hour(speed) / 60 as f64) as u32
}
```

Detailed call stack

By default the stack unwinding will only give something like this:

```
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 99',
src/main.rs:4:5
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

Though there is the reason of panic and the line of the code is showing where the panic has occurred, sometimes we want to get more info about the call stack.

```

## FILL in the blank to display the whole call stack
## Tips: you can find the clue in the default panic info
$ __ cargo run
thread 'main' panicked at 'assertion failed: `(left == right)`
  left: `[97, 98, 99]`,
 right: `[96, 97, 98]`', src/main.rs:3:5
stack backtrace:
   0: rust_begin_unwind
       at
/rustc/9d1b2106e23b1abd32fce1f17267604a5102f57a/library/std/src/panicking.rs:498:5
   1: core::panicking::panic_fmt
       at
/rustc/9d1b2106e23b1abd32fce1f17267604a5102f57a/library/core/src/panicking.rs:116:
14
   2: core::panicking::assert_failed_inner
   3: core::panicking::assert_failed
       at
/rustc/9d1b2106e23b1abd32fce1f17267604a5102f57a/library/core/src/panicking.rs:154:
5
   4: study_cargo::main
       at ./src/main.rs:3:5
   5: core::ops::function::FnOnce::call_once
       at
/rustc/9d1b2106e23b1abd32fce1f17267604a5102f57a/library/core/src/ops/function.rs:2
27:5
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose
backtrace.

```

unwinding and abort

By default, when a `panic` occurs, the program starts *unwinding*, which means Rust walks back up the stack and cleans up the data from each function it encounters.

But this walk back and clean up is a lot of work. The alternative is to immediately abort the program without cleaning up.

If in your project you need to make the resulting binary as small as possible, you can switch from unwinding to aborting by adding below content to `Cargo.toml`:

```

[profile.release]
panic = 'abort'

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

result and ?

`Result<T>` is an enum to describe possible errors. It has two variants:

- `Ok(T)` : A value `T` was found
- `Err(e)` : An error was found with a value `e`

In short words, the expected outcome is `Ok`, while the unexpected outcome is `Err`.

1. 🌟🌟

```
|  
// FILL in the blanks and FIX the errors  
use std::num::ParseIntError;  
  
fn multiply(n1_str: &str, n2_str: &str) -> __ {  
    let n1 = n1_str.parse::<i32>();  
    let n2 = n2_str.parse::<i32>();  
    Ok(n1.unwrap() * n2.unwrap())  
}  
  
fn main() {  
    let result = multiply("10", "2");  
    assert_eq!(result, __);  
  
    let result = multiply("t", "2");  
    assert_eq!(result.__, 8);  
  
    println!("Success!");  
}
```

?

`?` is almost exactly equivalent to `unwrap`, but `?` returns instead of panic on `Err`.

2. 🌟🌟


```

use std::num::ParseIntError;

// IMPLEMENT multiply with ?
// DON'T use unwrap here
fn multiply(n1_str: &str, n2_str: &str) -> __ {
}

fn main() {
    assert_eq!(multiply("3", "4").unwrap(), 12);
    println!("Success!");
}

```

3. 🌟🌟

```

use std::fs::File;
use std::io::{self, Read};

fn read_file1() -> Result<String, io::Error> {
    let f = File::open("hello.txt");
    let mut f = match f {
        Ok(file) => file,
        Err(e) => return Err(e),
    };

    let mut s = String::new();
    match f.read_to_string(&mut s) {
        Ok(_) => Ok(s),
        Err(e) => Err(e),
    }
}

// FILL in the blanks with one code line
// DON'T change any code lines
fn read_file2() -> Result<String, io::Error> {
    let mut s = String::new();

    __;

    Ok(s)
}

fn main() {
    assert_eq!(read_file1().unwrap_err().to_string(), read_file2().unwrap_err().to_string());
    println!("Success!");
}

```

map & and_then

[map](#) and [and_then](#) are two common combinators for `Result<T, E>` (also for `Option<T>`).

4. 🌟🌟

```
use std::num::ParseIntError;

// FILL in the blank in two ways: map, and then
fn add_two(n_str: &str) -> Result<i32, ParseIntError> {
    n_str.parse::<i32>().__
}

fn main() {
    assert_eq!(add_two("4").unwrap(), 6);

    println!("Success!");
}
```

5. 🌟🌟🌟

```

use std::num::ParseIntError;

// With the return type rewritten, we use pattern matching without `unwrap()`.
// But it's so Verbose...
fn multiply(n1_str: &str, n2_str: &str) -> Result<i32, ParseIntError> {
    match n1_str.parse::<i32>() {
        Ok(n1) => {
            match n2_str.parse::<i32>() {
                Ok(n2) => {
                    Ok(n1 * n2)
                },
                Err(e) => Err(e),
            }
        },
        Err(e) => Err(e),
    }
}

// Rewriting `multiply` to make it succinct
// You should use BOTH of `and_then` and `map` here.
fn multiply1(n1_str: &str, n2_str: &str) -> Result<i32, ParseIntError> {
    // IMPLEMENT...
}

fn print(result: Result<i32, ParseIntError>) {
    match result {
        Ok(n) => println!("n is {}", n),
        Err(e) => println!("Error: {}", e),
    }
}

fn main() {
    // This still presents a reasonable answer.
    let twenty = multiply1("10", "2");
    print(twenty);

    // The following now provides a much more helpful error message.
    let tt = multiply("t", "2");
    print(tt);

    println!("Success!");
}

```

Type alias

Using `std::result::Result<T, ParseIntError>` everywhere is verbose and tedious, we can use alias for this purpose.

At a module level, creating aliases can be particularly helpful. Errors found in a specific module often has the same `Err` type, so a single alias can succinctly defined all associated `Results`. This is so useful even the `std` library supplies one: `io::Result`.

6. ☀

```
use std::num::ParseIntError;

// FILL in the blank
type __;

// Use the above alias to refer to our specific `Result` type.
fn multiply(first_number_str: &str, second_number_str: &str) -> Res<i32> {
    first_number_str.parse::<i32>().and_then(|first_number| {
        second_number_str.parse::<i32>().map(|second_number| first_number * second_n
    })
}

// Here, the alias again allows us to save some space.
fn print(result: Res<i32>) {
    match result {
        Ok(n) => println!("n is {}", n),
        Err(e) => println!("Error: {}", e),
    }
}

fn main() {
    print(multiply("10", "2"));
    print(multiply("t", "2"));

    println!("Success!");
}
```



Using Result in fn main

Typically the main function will look like this:

```
fn main() {
    println!("Hello World!");
}
```

However `main` is also able to have a return type of `Result`. If an error occurs within the `main` function it will return an error code and print a debug representation of the error(`Debug` trait).

The following example shows such a scenario:

```
|  
use std::num::ParseIntError;  
  
fn main() -> Result<(), ParseIntError> {  
    let number_str = "10";  
    let number = match number_str.parse::<i32>() {  
        Ok(number) => number,  
        Err(e) => return Err(e),  
    };  
    println!("{}", number);  
    Ok(())  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it
:)

Crate and module

Learning resources:

- English: [Rust Book Chapter 7](#)
- 繁體中文: [Rust 繁體中文 - 模塊](#)

Package and Crate

A package is a project which you create with Cargo (in most cases), so it contains a `Cargo.toml` file in it.

1. 🌟 Create a package with below layout:

```
.
├── Cargo.toml
└── src
    └── main.rs
```

1 directory, 2 files

```
# in Cargo.toml
[package]
name = "hello-package"
version = "0.1.0"
edition = "2021"
```

Note! We will use this package across the whole chapter as a practice project.

2. 🌟 Create a package with below layout:

```
.
├── Cargo.toml
└── src
    └── lib.rs
```

1 directory, 2 files

```
# in Cargo.toml
[package]
name = "hello-package1"
version = "0.1.0"
edition = "2021"
```

Note! This package could be safely removed due to the first one's existence.

3. 🌟

```
/* FILL in the blank with your ANSWER */
```

```
// Q: What's the difference between package number 1 and number 2?
// A: __
```

Crate

A crate is a binary or library. The crate root is a source file that the Rust compiler starts from and makes up the root module of the crate.

In package `hello-package`, there is binary crate with the same name as the package: `hello-package`, and `src/main.rs` is the crate root of this binary crate.

Similar to `hello-package`, `hello-package1` also has a crate in it, however, this package doesn't contain a binary crate but a library crate, and `src/lib.rs` is the crate root.

4. 

/ FILL in the blank with your ANSWER */*

```
// Q: What's the name of the library crate in package `hello-package1`?  
// A: --
```

5. 🌟🌟 Add a library crate for `hello-package` and describe it's files tree below:

```
# FILL in the blanks
```

```

.
├── Cargo.lock
├── Cargo.toml
├── src
├── .
├── ..
└── .

```

After this step, there should be two crates in package `hello-package`: **a binary crate and a library crate, both with the same name as the package.**

6. 🌟🌟🌟 A package can contain at most one library crate, but it can contain as many binary crates as you would like by placing files in `src/bin` directory: **each file will be a separate binary crate with the same name as the file.**


```
# Create a package which contains
# 1. three binary crates: `hello-package`, `main1` and `main2`
# 2. one library crate
# describe the directory tree below
```

```
.
├── Cargo.toml
├── Cargo.lock
├── src
│   ├── --
│   ├── --
│   └── --
├── tests # directory for integrated tests files
│   └── some_integration_tests.rs
├── benches # dir for benchmark files
│   └── simple_bench.rs
└── examples # dir for example files
    └── simple_example.rs
```

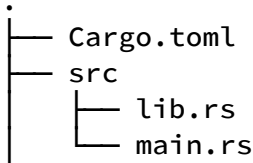
Yep, as you can see, the above package structure is very standard and is widely used in many Rust projects.

You can find the solutions [here](#) (under the solutions path), but only use it when you need it :)

Module

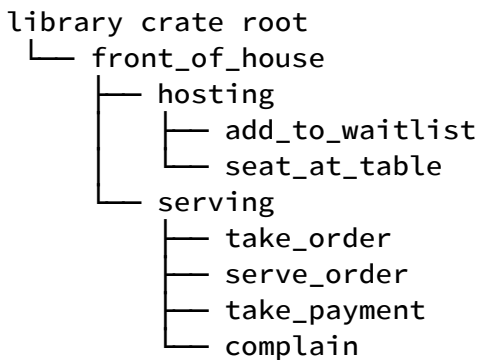
Modules let us organize the code within a crate into groups for readability and ease of reuse. Module also controls the privacy of items, which is whether an item can be seen by outside code(`public`), or is just an internal implementation and not available for outside code(`private`).

We have created a package named `hello-package` in previous chapter, and it looks like this:



Now it's time to create some modules in the library crate and use them in the binary crate, let's start.

1. 🌟🌟 Implement module `front_of_house` based on the module tree below:



```
// FILL in the blank
// in __.rs
```

```
mod front_of_house {
    // IMPLEMENT this module..
}
```

2. 🌟🌟 Let's call `add_to_waitlist` from a function `eat_at_restaurant` which is within the library crate root.

```
// In lib.rs
```

```
// FILL in the blanks and FIX the errors
// You need to make something public with `pub` to provide accessibility for outside
mod front_of_house {
    /* ...snip... */
}

pub fn eat_at_restaurant() {
    // Call add_to_waitlist with **absolute path**:
    __::add_to_waitlist();

    // Call with **relative path**
    __::add_to_waitlist();
}
```



3. 🌟🌟 You can use `super` to import items within the parent module

```
// In lib.rs
```

```
mod back_of_house {
    fn fix_incorrect_order() {
        cook_order();
        // FILL in the blank in three ways
        //1. using keyword `super`
        //2. using absolute path
        __::serve_order();
    }

    fn cook_order() {}
}
```

Separating modules into different files

```
// In lib.rs
pub mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}

        pub fn seat_at_table() -> String {
            String::from("sit down please")
        }
    }

    pub mod serving {
        pub fn take_order() {}

        pub fn serve_order() {}

        pub fn take_payment() {}

        // Maybe you don't want the guest hearing the your complaining about them
        // So just make it private
        fn complain() {}
    }
}

pub fn eat_at_restaurant() -> String {
    front_of_house::hosting::add_to_waitlist();

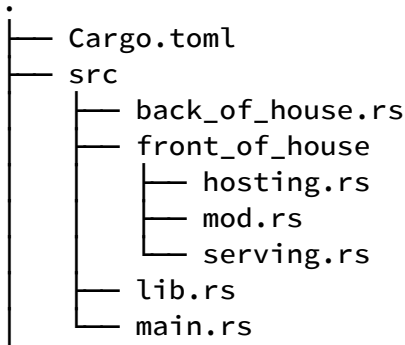
    back_of_house::cook_order();

    String::from("yummy yummy!")
}

pub mod back_of_house {
    pub fn fix_incorrect_order() {
        cook_order();
        crate::front_of_house::serving::serve_order();
    }

    pub fn cook_order() {}
}
```

4. 🌟🌟🌟🌟 Please separate the modules and codes above into files resident in below dir tree :



```
// In src/lib.rs
```

```
// IMPLEMENT...
```

```
// In src/back_of_house.rs
```

```
// IMPLEMENT...
```

```
// In src/front_of_house/mod.rs
```

```
// IMPLEMENT...
```

```
// In src/front_of_house/hosting.rs
```

```
// IMPLEMENT...
```

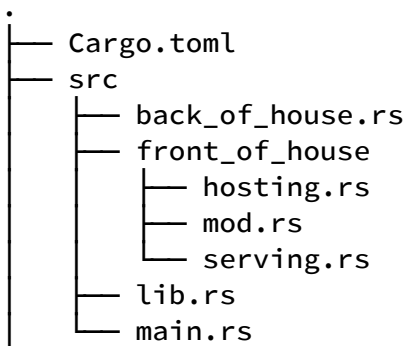
```
// In src/front_of_house/serving.rs
```

```
// IMPLEMENT...
```

Accessing code in library crate from binary crate

Please ensure you have completed the 4th exercise before making further progress.

You should have below structures and the corresponding codes in them when reaching here:



5. 🌟🌟🌟 Now we will call a few library functions from the binary crate.

```
// In src/main.rs

// FILL in the blank and FIX the errors
fn main() {
    assert_eq!(__, "sit down please");
    assert_eq!(__, "yummy yummy!");
}
```

You can find the solutions [here](#) (under the solutions path), but only use it when you need it :)

Use and pub

1. 🌟 We can bring two types of the same name into the same scope with use, but you need `as` keyword.

```
use std::fmt::Result;
use std::io::Result;

fn main() {}
```

2. 🌟🌟 If we are using multiple items defined in the same crate or module, then listing each item on its own line will take up too much vertical space.

```
// FILL in the blank in two ways
// DON'T add new code line
use std::collections::__;

fn main() {
    let _c1:HashMap<&str, i32> = HashMap::new();
    let mut c2 = BTreeMap::new();
    c2.insert(1, "a");
    let _c3: HashSet<i32> = HashSet::new();
}
```

Re-exporting names with `pub use`

3. 🌟🌟🌟 In our recently created package `hello-package`, add something to make the below code work

```
fn main() {
    assert_eq!(hello_package::hosting::seat_at_table(), "sit down please");
    assert_eq!(hello_package::eat_at_restaurant(), "yummy yummy!");
}
```

Pub(in Crate)

Sometimes we want an item only be public to a certain crate. For this we can use the `pub(in Crate)` syntax.

Example

```
pub mod a {  
    pub const I: i32 = 3;  
  
    fn semisecret(x: i32) -> i32 {  
        use self::b::c::J;  
        x + J  
    }  
  
    pub fn bar(z: i32) -> i32 {  
        semisecret(I) * z  
    }  
    pub fn foo(y: i32) -> i32 {  
        semisecret(I) + y  
    }  
  
    mod b {  
        pub(in crate::a) mod c {  
            pub(in crate::a) const J: i32 = 4;  
        }  
    }  
}
```

Full Code

The full code of `hello-package` is [here](#).

You can find the solutions [here](#) (under the solutions path), but only use it when you need it :)

Comments and Docs

Every program requires comments:

Comments

- Regular comments which are ignored by the compiler:
 - `//` Line comment, which goes to the end of the line
 - `/*` Block comment, which goes to the end of the closing delimiter `*/`

Examples

```
fn main() {  
    // This is an example of a line comment  
    // There are two slashes at the beginning of the line  
    // And nothing written inside these will be read by the compiler  
  
    // println!("Hello, world!");  
  
    // Run it. See? Now try deleting the two slashes, and run it again.  
  
    /*  
    * This is another type of comment, a block comment. In general,  
    * line comments are the recommended comment style. But  
    * block comments are extremely useful for temporarily disabling  
    * chunks of code. /* Block comments can be /* nested, */ */  
    * so it takes only a few keystrokes to comment out everything  
    * in this main() function. /*/*/* Try it yourself! /*/*/*  
    */  
  
    /*  
    Note: The previous column of `*` was entirely for style. There's  
    no actual need for it.  
    */  
}
```

Exercises

1. 🌟🌟

```
|
/* Make it work, only using comments! */
fn main() {
    todo!();
    unimplemented!();

    assert_eq!(6, 5 + 3 + 2 + 1 )
}
```

Doc Comments

- Doc comments which are parsed into HTML and supported Markdown
 - `///` Generate library docs for the following item
 - `//!` Generate library docs for the enclosing item

Before starting, we need to create a new package for practice: `cargo new --lib doc-comments`.

Line doc comments `///`

Add docs for function `add_one`

```
// in lib.rs

/// Add one to the given value and return the value
///
/// # Examples
///
/// ```
/// let arg = 5;
/// let answer = my_crate::add_one(arg);
///
/// assert_eq!(6, answer);
/// ```
pub fn add_one(x: i32) -> i32 {
    x + 1
}
```

Cargo doc

We can use `cargo doc --open` to generate html files and open them in the browser.

Block doc comments `/** ... */`

Add docs for function `add_two`:

```
/** Add two to the given value and return a new value

let arg = 5;
let answer = my_crate::add_two(arg);

assert_eq!(7, answer);

*/
pub fn add_two(x: i32) -> i32 {
    x + 2
}
```

Doc comments for crate and module

We can also add doc comments for our crates and modules.

Firstly, let's add some doc comments for our library crate:

Note: We must place crates and module comments at the top of crate root or module file.

```
//! # Doc comments
//!
//! A library for showing how to use doc comments

// in lib.rs
pub mod compute;
```

You can also use block comments to achieve this:

```
/** # Doc comments

A library for showing how to use doc comments */
```

Next, create a new module file `src/compute.rs`, and add following comments to it:

```
//! Do some complicated arithmetic that you can't do by yourself

// in compute.rs
```

Then run `cargo doc --open` and see the results.

Doc tests

The doc comments of `add_one` and `add_two` contain two example code blocks.

The examples can not only demonstrate how to use your library, but also running as test with `cargo test` command.

2. 🌟🌟 But there are errors in the two examples, please fix them, and running with `cargo test` to get following result:

```
running 0 tests
```

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;  
finished in 0.00s
```

```
Doc-tests doc-comments
```

```
running 2 tests
```

```
test src/lib.rs - add_one (line 11) ... ok
```

```
test src/lib.rs - add_two (line 26) ... ok
```

```
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;  
finished in 0.55s
```

3. 🌟🌟 Sometimes we expect an example to be panic, add following code to `src/compute.rs` and make the `cargo test` passed.

You can only modify the comments, DON'T modify `fn div`

```
// in src/compute.rs

/// # Panics
///
/// The function panics if the second argument is zero.
///
/// ```rust,should_panic
/// // panics on division by zero
/// doc_comments::compute::div(10, 0);
/// ```
pub fn div(a: i32, b: i32) -> i32 {
    if b == 0 {
        panic!("Divide-by-zero error");
    }

    a / b
}
```

4. 🌟🌟 Sometimes we want to hide the doc comments, but keep the doc tests.

Add following code to `src/compute.rs` ,

```
// in src/compute.rs

/// ```
/// # fn try_main() -> Result<(), String> {
/// # let res = doc_comments::compute::try_div(10, 0)?;
/// # Ok(()) // returning from try_main
/// # }
/// # fn main() {
/// #     try_main().unwrap();
/// # }
/// # ```
pub fn try_div(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        Err(String::from("Divide-by-zero"))
    } else {
        Ok(a / b)
    }
}
```

and modify this code to achieve two goals:

- The doc comments must not be presented in html files generated by `cargo doc --open`
- run the tests, you should see results as below:

```
running 0 tests
```

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;  
finished in 0.00s
```

```
Doc-tests doc-comments
```

```
running 4 tests
```

```
test src/compute.rs - compute::div (line 7) ... ok  
test src/lib.rs - add_two (line 27) ... ok  
test src/lib.rs - add_one (line 11) ... ok  
test src/compute.rs - compute::try_div (line 20) ... ok
```

```
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;  
finished in 0.51s
```

Code navigation

Rust provide a very powerful feature for us, that is code navigation in doc comments.

Add following code to `src/lib.rs`:

```
// in lib.rs  
  
/// Add three to the given value and return a [`Option`] type  
pub fn add_three(x: i32) -> Option<i32> {  
    Some(x + 3)  
}
```

Besides jump into the standard library, you can also jump to another module in the package.

```
// in lib.rs  
  
mod a {  
    /// Add four to the given value and return a [`Option`] type  
    /// [`crate::MySpecialFormatter`]  
    pub fn add_four(x: i32) -> Option<i32> {  
        Some(x + 4)  
    }  
}  
  
struct MySpecialFormatter;
```

Doc attributes

Below are a few examples of the most common `#[doc]` attributes used with `rustdoc`.

inline

Used to inline docs, instead of linking out to separate page.

```
#[doc(inline)]
pub use bar::Bar;

/// bar docs
mod bar {
    /// the docs for Bar
    pub struct Bar;
}
```

no_inline

Used to prevent linking out to separate page or anywhere.

```
// Example from libcore/prelude
#[doc(no_inline)]
pub use crate::mem::drop;
```

hidden

Using this tells `rustdoc` not to include this in documentation:

```
// Example from the futures-rs library
#[doc(hidden)]
pub use self::async_await::*;
```

For documentation, `rustdoc` is widely used by the community. It's what is used to generate the [std library docs](#).

Full Code

The full code of package `doc-comments` is [here](#).

Formatted output

```
fn main() {  
    // In general, the `{}` will be automatically replaced with any  
    // arguments. These will be stringified.  
    println!("{}", 31);  
  
    // Without a suffix, 31 becomes an i32. You can change what type 31 is  
    // by providing a suffix. The number 31i64 for example has the type i64.  
  
    // There are various optional patterns this works with. Positional  
    // arguments can be used.  
    println!("{0}, this is {1}. {1}, this is {0}", "Alice", "Bob");  
  
    // As can named arguments.  
    println!("{subject} {verb} {object}",  
             object="the lazy dog",  
             subject="the quick brown fox",  
             verb="jumps over");  
  
    // Special formatting can be specified after a `:`.  
    println!("{0} of {:b} people know binary, the other half doesn't", 1, 2);  
  
    // You can right-align text with a specified width. This will output  
    // "    1". 5 white spaces and a "1".  
    println!("{number:>width$}", number=1, width=6);  
  
    // You can pad numbers with extra zeroes. This will output "000001".  
    println!("{number:0>width$}", number=1, width=6);  
  
    // Rust even checks to make sure the correct number of arguments are  
    // used.  
    println!("My name is {0}, {1} {0}", "Bond");  
    // FIXME ^ Add the missing argument: "James"  
  
    // Create a structure named `Structure` which contains an `i32`.  
    #[allow(dead_code)]  
    struct Structure(i32);  
  
    // However, custom types such as this structure require more complicated  
    // handling. This will not work.  
    println!("This struct `{}` won't print...", Structure(3));  
    // FIXME ^ Comment out this line.  
  
    // For Rust 1.58 and above, you can directly capture the argument from  
    // surrounding variable. Just like the above, this will output  
    // "    1". 5 white spaces and a "1".  
    let number: f64 = 1.0;  
    let width: usize = 6;  
    println!("{number:>width$}");  
}
```


[`std::fmt`][fmt] contains many [`traits`][traits] which govern the display of text. The base form of two important ones are listed below:

- `fmt::Debug` : Uses the `{:?}` marker. Format text for debugging purposes.
- `fmt::Display` : Uses the `{}` marker. Format text in a more elegant, user friendly fashion.

Here, we used `fmt::Display` because the std library provides implementations for these types. To print text for custom types, more steps are required.

Implementing the `fmt::Display` trait automatically implements the [`ToString`] trait which allows us to [convert] the type to [`String`][string].

println! and format!

Printing is handled by a series of [`macros`][macros] defined in [`std::fmt`][fmt] Some of which include:

- `format!` : write formatted text to [`String`][string]
- `print!` : same as `format!` but the text is printed to the console (`io::stdout`).
- `println!` : same as `print!` but a newline is appended.
- `eprint!` : same as `format!` but the text is printed to the standard error (`io::stderr`).
- `eprintln!` : same as `eprint!` but a newline is appended.

All parse text in the same fashion. As a plus, Rust checks format correctness at compile time.

format!

1. 🌟

```
|  
fn main() {  
    let s1 = "hello";  
    /* Fill in the blank */  
    let s = format!(__);  
    assert_eq!(s, "hello, world!");  
}
```

print!, println!

2. 🌟

```
|  
fn main() {  
    /* Fill in the blanks to make it print:  
    Hello world, I am  
    Sunface!  
    */  
    __("hello world, ");  
    __("I am");  
    __("Sunface!");  
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Debug and Display

All types which want to be printable must implement the `std::fmt` formatting trait:

`std::fmt::Debug` OR `std::fmt::Display`.

Automatic implementations are only provided for types such as in the `std` library. All others have to be manually implemented.

Debug

The implementation of `Debug` is very straightforward: All types can derive the `std::fmt::Debug` implementation. This is not true for `std::fmt::Display` which must be manually implemented.

`{:?}` must be used to print out the type which has implemented the `Debug` trait.

```
// This structure cannot be printed either with `fmt::Display` or
// with `fmt::Debug`.
struct UnPrintable(i32);

// To make this struct printable with `fmt::Debug`, we can derive the automatic
// implementations provided by Rust
#[derive(Debug)]
struct DebugPrintable(i32);
```

1. 🌟

```
|
/* Fill in the blanks and Fix the errors */
struct Structure(i32);

fn main() {
    // Types in std and Rust have implemented the fmt::Debug trait
    println!("__ months in a year.", 12);

    println!("Now __ will print!", Structure(3));
}
```

2. 🌟🌟 So `fmt::Debug` definitely makes one type printable, but sacrifices some elegance. Maybe we can get more elegant by replacing `{:?}` with something else(but not `{}` !)

```

#[derive(Debug)]
struct Person {
    name: String,
    age: u8
}

fn main() {
    let person = Person { name: "Sunface".to_string(), age: 18 };

    /* Make it output:
    Person {
        name: "Sunface",
        age: 18,
    }
    */
    println!("{:?}", person);
}

```

3. 🌟🌟 We can also manually implement `Debug` trait for our types

```

#[derive(Debug)]
struct Structure(i32);

#[derive(Debug)]
struct Deep(Structure);

fn main() {
    // The problem with `derive` is there is no control over how
    // the results look. What if I want this to just show a `7`?

    /* Make it print: Now 7 will print! */
    println!("Now {:?} will print!", Deep(Structure(7)));
}

```

Display

Yeah, `Debug` is simple and easy to use. But sometimes we want to customize the output appearance of our type. This is where `Display` really shines.

Unlike `Debug`, there is no way to derive the implementation of the `Display` trait, we have to manually implement it.

Another thing to note: the placeholder for `Display` is `{}` not `{:?}`.

4. 🌟🌟

```

|
/* Make it work*/
use std::fmt;

struct Point2D {
    x: f64,
    y: f64,
}

impl fmt::Display for Point2D {
    /* Implement.. */
}

impl fmt::Debug for Point2D {
    /* Implement.. */
}

fn main() {
    let point = Point2D { x: 3.3, y: 7.2 };
    assert_eq!(format!("{}",point), "Display: 3.3 + 7.2i");
    assert_eq!(format!("{:?}",point), "Debug: Complex { real: 3.3, imag: 7.2 }");

    println!("Success!");
}

```

? operator

Implementing `fmt::Display` for a structure whose elements must be handled separately is tricky. The problem is each `write!` generates a `fmt::Result` which must be handled in the same place.

Fortunately, Rust provides the `?` operator to help us eliminate some unnecessary codes for dealing with `fmt::Result`.

5. 🌟🌟

```

|
/* Make it work */
use std::fmt;

struct List(Vec<i32>);

impl fmt::Display for List {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        // Extract the value using tuple indexing,
        // and create a reference to `vec`.
        let vec = &self.0;

        write!(f, "[")?;

        // Iterate over `v` in `vec` while enumerating the iteration
        // count in `count`.
        for (count, v) in vec.iter().enumerate() {
            // For every element except the first, add a comma.
            // Use the ? operator to return on errors.
            if count != 0 { write!(f, ", ")?; }
            write!(f, "{}", v)?;
        }

        // Close the opened bracket and return a fmt::Result value.
        write!(f, "]")
    }
}

fn main() {
    let v = List(vec![1, 2, 3]);
    assert_eq!(format!("{}",v), "[0: 1, 1: 2, 2: 3]");
    println!("Success!");
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Formatting

Positional arguments

1. 🌟🌟🌟

```
/* Fill in the blanks */
fn main() {
    println!("{0}, this is {1}. {1}, this is {0}", "Alice", "Bob"); // => Alice, thi
    assert_eq!(format!("{1}{0}", 1, 2), __);
    assert_eq!(format!(__, 1, 2), "2112");
    println!("Success!");
}
```



Named arguments

2. 🌟🌟🌟

```
fn main() {
    println!("{argument}", argument = "test"); // => "test"

    /* Fill in the blanks */
    assert_eq!(format!("{name}{}", 1, __), "21");
    assert_eq!(format!(__, a = "a", b = 'b', c = 3 ), "a 3 b");

    /* Fix the error */
    // Named argument must be placed after other arguments
    println!("{abc} {1}", abc = "def", 2);

    println!("Success!");
}
```

Padding with string

3. 🌟🌟🌟 By default, you can pad string with spaces


```
fn main() {
    // The following two are padding with 5 spaces
    println!("Hello {:5}!", "x"); // => "Hello x    !"
    println!("Hello {:1$}!", "x", 5); // => "Hello x    !"

    /* Fill in the blanks */
    assert_eq!(format!("Hello __!", 5, "x"), "Hello x    !");
    assert_eq!(format!("Hello __!", "x", width = 5), "Hello x    !");

    println!("Success!");
}
```

4. 🌟🌟🌟 Left align, right align, pad with specified characters.

```
fn main() {
    // Left align
    println!("Hello {:<5}!", "x"); // => Hello x    !
    // Right align
    assert_eq!(format!("Hello __!", "x"), "Hello    x!");
    // Center align
    assert_eq!(format!("Hello __!", "x"), "Hello  x  !");

    // Left align, pad with '&'
    assert_eq!(format!("Hello {:&<5}!", "x"), __);

    println!("Success!");
}
```

5. 🌟🌟 You can pad numbers with extra zeros.

```
fn main() {
    println!("Hello {:5}!", 5); // => Hello    5!
    println!("Hello {:+}!", 5); // => Hello +5!
    println!("Hello {:05}!", 5); // => Hello 00005!
    println!("Hello {:05}!", -5); // => Hello -0005!

    /* Fill in the blank */
    assert!(format!("{number:0>width$}", number=1, width=6) == __);

    println!("Success!")
};}
```

Precision

6. 🌟🌟 Floating point precision

```

/* Fill in the blanks */
fn main() {
    let v = 3.1415926;

    println!("{:.1$}", v, 4); // same as {:.4} => 3.1416

    assert_eq!(format!("{}", v), "3.14");
    assert_eq!(format!("{}", v), "+3.14");
    assert_eq!(format!("{}", v), "3");

    println!("Success!");
}

```

7. 🌟🌟🌟 String length

```

fn main() {
    let s = "Hello, world!";

    println!("{0:.5}", s); // => Hello

    assert_eq!(format!("Hello __!", 3, "abcdefg"), "Hello abc!");

    println!("Success!");
}

```

Binary, octal, hex

- `format!("{}", foo) -> "3735928559"`
- `format!("0x{:X}", foo) -> "0xDEADBEEF"`
- `format!("0o{:o}", foo) -> "0o33653337357"`

8. 🌟🌟

```

fn main() {
    assert_eq!(format!("{}", 27), "0b11011");
    assert_eq!(format!("{}", 27), "0o33");
    assert_eq!(format!("{}", 27), "0x1b");
    assert_eq!(format!("{}", 27), "0x1B");

    println!("{:x}!", 27); // Hex with no prefix => 1b

    println!("{:#010b}", 27); // Pad binary with 0, width = 10, => 0b00011011

    println!("Success!");
}

```

Capture the environment

9. 🌟🌟🌟

```
fn get_person() -> String {
    String::from("sunface")
}

fn get_format() -> (usize, usize) {
    (4, 1)
}

fn main() {
    let person = get_person();
    println!("Hello, {person}!");

    let (width, precision) = get_format();
    let scores = [("sunface", 99.12), ("jack", 60.34)];
    /* Make it print:
    sunface: 99.1
    jack: 60.3
    */
    for (name, score) in scores {
        println!("{name}: __");
    }
}
```

Others

Example

```
fn main() {
    // Exponent
    println!("{:2e}", 1000000000); // => 1e9
    println!("{:2E}", 1000000000); // => 1E9

    // Pointer address
    let v= vec![1, 2, 3];
    println!("{:p}", v.as_ptr()); // => 0x600002324050

    // Escape
    println!("Hello {}", "Hello {}"); // => Hello {}
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Lifetime

Learning resources:

- English: [Rust Book 10.3](#)
- 简体中文: [Rust 生命周期 - 简体中文](#)

Lifetime

The compiler uses lifetime to ensure all borrows are valid. Typically, a variable's lifetime begins when it is created and ends when it is destroyed.

The scope of lifetime

1. 🌟

```
/* Annotate the lifetime of `i` and `borrow2` */

// Lifetimes are annotated below with lines denoting the creation
// and destruction of each variable.
// `i` has the longest lifetime because its scope entirely encloses
// both `borrow1` and `borrow2`. The duration of `borrow1` compared
// to `borrow2` is irrelevant since they are disjoint.
fn main() {
    let i = 3;
    {
        let borrow1 = &i; // `borrow1` lifetime starts. ┌
        // ────────────────────────────────────────────┐
        println!("borrow1: {}", borrow1); // ───────────┘
    } // `borrow1` ends. ───────────────────────────────────┐
    {
        let borrow2 = &i;
        println!("borrow2: {}", borrow2);
    }
}
```

2. 🌟🌟

Example

```
{
    let x = 5;           // -----+-- 'b
                        //          |
    let r = &x;          // --+-- 'a  |
                        //      |    |
    println!("r: {}", r); //      |    |
                        // --+      |
}                        // -----+
```

/ Annotate `r` and `x` as above, and explain why this code fails to compile, in the*

```
fn main() {  
  {  
    let r; // -----+-- 'a  
           //      |  
    {  
      let x = 5; // -+-- 'b |  
      r = &x;    // |      |  
    }           // -+      |  
           //      |  
    println!("r: {}", r); //      |  
  }           // -----+  
}
```



Lifetime annotating

The **borrow checker uses explicit lifetime annotations** to determine how long a reference should be valid.

But for us users, in most cases, there is no need to annotate the lifetime, because there are several elision rules, before learning these rules, we need to know how to annotate lifetime manually.

Function

Ignoring elision rules, lifetimes in function signatures have a few constraints:

- Any reference must have an annotated lifetime
- Any reference being returned must have the same lifetime as one of the inputs or be static

Example

```

// One input reference with lifetime 'a' which must live
// at least as long as the function.
fn print_one<'a>(x: &'a i32) {
    println!("`print_one`: x is {}", x);
}

// Mutable references are possible with lifetimes as well.
fn add_one<'a>(x: &'a mut i32) {
    *x += 1;
}

// Multiple elements with different lifetimes. In this case, it
// would be fine for both to have the same lifetime 'a', but
// in more complex cases, different lifetimes may be required.
fn print_multi<'a, 'b>(x: &'a i32, y: &'b i32) {
    println!("`print_multi`: x is {}, y is {}", x, y);
}

// Returning references that have been passed in is acceptable.
// However, the correct lifetime must be returned.
fn pass_x<'a, 'b>(x: &'a i32, _: &'b i32) -> &'a i32 { x }

fn main() {
    let x = 7;
    let y = 9;

    print_one(&x);
    print_multi(&x, &y);

    let z = pass_x(&x, &y);
    print_one(z);

    let mut t = 3;
    add_one(&mut t);
    print_one(&t);
}

```

3. 🌟

```

/* Make it work by adding proper lifetime annotation */
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {}

```

4. 🌟🌟🌟

```
// `a` must live longer than the function.
// Here, `&String::from("foo")` would create a `String`, followed by a
// reference. Then the data is dropped upon exiting the scope, leaving
// a reference to invalid data to be returned.
```

```
/* Fix the error in three ways */
fn invalid_output<'a>() -> &'a String {
    &String::from("foo")
}
```

```
fn main() {
}
```

5. 🌟🌟

```
// `print_refs` takes two references to `i32` which have different
// lifetimes `a` and `b`. These two lifetimes must both be at
// least as long as the function `print_refs`.
fn print_refs<'a, 'b>(x: &'a i32, y: &'b i32) {
    println!("x is {} and y is {}", x, y);
}
```

```
/* Make it work */
// A function which takes no arguments, but has a lifetime parameter `a`.
fn failed_borrow<'a>() {
    let _x = 12;

    // ERROR: `_x` does not live long enough
    let y: &'a i32 = &_x;
    // Attempting to use the lifetime `a` as an explicit type annotation
    // inside the function will fail because the lifetime of `_x` is shorter
    // than `a`. A short lifetime cannot be coerced into a longer one.
}
```

```
fn main() {
    let (four, nine) = (4, 9);

    // Borrows (`&`) of both variables are passed into the function.
    print_refs(&four, &nine);
    // Any input which is borrowed must outlive the borrower.
    // In other words, the lifetime of `four` and `nine` must
    // be longer than that of `print_refs`.

    failed_borrow();
    // `failed_borrow` contains no references to force `a` to be
    // longer than the lifetime of the function, but `a` is longer.
    // Because the lifetime is never constrained, it defaults to `'static`.
}
```

Structs

6. 🌟


```

/* Make it work by adding proper lifetime annotation */

// A type `Borrowed` which houses a reference to an
// `i32`. The reference to `i32` must outlive `Borrowed`.
#[derive(Debug)]
struct Borrowed(&i32);

// Similarly, both references here must outlive this structure.
#[derive(Debug)]
struct NamedBorrowed {
    x: &i32,
    y: &i32,
}

// An enum which is either an `i32` or a reference to one.
#[derive(Debug)]
enum Either {
    Num(i32),
    Ref(&i32),
}

fn main() {
    let x = 18;
    let y = 15;

    let single = Borrowed(&x);
    let double = NamedBorrowed { x: &x, y: &y };
    let reference = Either::Ref(&x);
    let number = Either::Num(y);

    println!("x is borrowed in {:?}", single);
    println!("x and y are borrowed in {:?}", double);
    println!("x is borrowed in {:?}", reference);
    println!("y is not borrowed in {:?}", number);
}

```

7. 🌟🌟

```

/* Make it work */

#[derive(Debug)]
struct NoCopyType {}

#[derive(Debug)]
struct Example<'a, 'b> {
    a: &'a u32,
    b: &'b NoCopyType
}

fn main()
{
    /* 'a tied to fn-main stackframe */
    let var_a = 35;
    let example: Example;

    {
        /* Lifetime 'b tied to new stackframe/scope */
        let var_b = NoCopyType {};

        /* fixme */
        example = Example { a: &var_a, b: &var_b };
    }

    println!("(Success!) {:?}" , example);
}

```

8. 🌟🌟

```

#[derive(Debug)]
struct NoCopyType {}

#[derive(Debug)]
#[allow(dead_code)]
struct Example<'a, 'b> {
    a: &'a u32,
    b: &'b NoCopyType
}

/* Fix function signature */
fn fix_me(foo: &Example) -> &NoCopyType
{ foo.b }

fn main()
{
    let no_copy = NoCopyType {};
    let example = Example { a: &1, b: &no_copy };
    fix_me(&example);
    println!("Success!")
}

```

Method

Methods are annotated similarly to functions.

Example

```
struct Owner(i32);

impl Owner {
    // Annotate lifetimes as in a standalone function.
    fn add_one<'a>(&'a mut self) { self.0 += 1; }
    fn print<'a>(&'a self) {
        println!("`print`: {}", self.0);
    }
}

fn main() {
    let mut owner = Owner(18);

    owner.add_one();
    owner.print();
}
```

9. 🌟🌟🌟

```
/* Make it work by adding proper lifetime annotations */
struct ImportantExcerpt {
    part: &str,
}

impl ImportantExcerpt {
    fn level(&'a self) -> i32 {
        3
    }
}

fn main() {}
```

Elision

Some lifetime patterns are so common that borrow checker will allow you to omit them to save typing and improve readability.

This is known as **Elision**. Elision exist in Rust only because these patterns are common.

For a more comprehensive understanding of elision, please see [lifetime elision](#) in the official book.

10. 🌟🌟🌟

```

/* Remove all the lifetimes that can be elided */

fn input<'a>(x: &'a i32) {
    println!("`annotated_input`: {}", x);
}

fn pass<'a>(x: &'a i32) -> &'a i32 { x }

fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
    x
}

struct Owner(i32);

impl Owner {
    // Annotate lifetimes as in a standalone function.
    fn add_one<'a>(&'a mut self) { self.0 += 1; }
    fn print<'a>(&'a self) {
        println!("`print`: {}", self.0);
    }
}

struct Person<'a> {
    age: u8,
    name: &'a str,
}

enum Either<'a> {
    Num(i32),
    Ref(&'a i32),
}

fn main() {}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

&'static and T: 'static

'static is a reserved lifetime name, you might have encountered it several times:

```
// A reference with 'static lifetime:  
let s: &'static str = "hello world";  
  
// 'static as part of a trait bound:  
fn generic<T>(x: T) where T: 'static {}
```

Though they are all 'static, but subtly different.

&'static

As a reference lifetime, &'static indicates the data pointed to by the reference lives as long as the running program. But it can still be coerced to a shorter lifetime.

1. 🌟🌟 There are several ways to make a variable with 'static lifetime, two of them are stored in the read-only memory of the binary.

```
|  
/* Fill in the blank in two ways */  
fn main() {  
    --;  
    need_static(v);  
  
    println!("Success!")  
}  
  
fn need_static(r : &'static str) {  
    assert_eq!(r, "hello");  
}
```

2. 🌟🌟🌟🌟 Another way to make 'static lifetime is using `Box::leak`

```

#[derive(Debug)]
struct Config {
    a: String,
    b: String,
}
static mut config: Option<&mut Config> = None;

/* Make it work without changing the function signatures of `init` */
fn init() -> Option<&'static mut Config> {
    Some(&mut Config {
        a: "A".to_string(),
        b: "B".to_string(),
    })
}

fn main() {
    unsafe {
        config = init();

        println!("{:?}", config)
    }
}

```

3. 🌟 `&'static` only indicates that the data can live forever, not the reference. The latter one will be constrained by its scope.

```

fn main() {
    {
        // Make a `string` literal and print it:
        let static_string = "I'm in read-only memory";
        println!("static_string: {}", static_string);

        // When `static_string` goes out of scope, the reference
        // can no longer be used, but the data remains in the binary.
    }

    println!("static_string reference remains alive: {}", static_string);
}

```

4. `&'static` can be coerced to a shorter lifetime.

Example

```

// Make a constant with `static` lifetime.
static NUM: i32 = 18;

// Returns a reference to `NUM` where its `static`
// lifetime is coerced to that of the input argument.
fn coerce_static<'a>(_: &'a i32) -> &'a i32 {
    &NUM
}

fn main() {
    {
        // Make an integer to use for `coerce_static`:
        let lifetime_num = 9;

        // Coerce `NUM` to lifetime of `lifetime_num`:
        let coerced_static = coerce_static(&lifetime_num);

        println!("coerced_static: {}", coerced_static);
    }

    println!("NUM: {} stays accessible!", NUM);
}

```

T: 'static

As a trait bound, it means the type does not contain any non-static references. Eg. the receiver can hold on to the type for as long as they want and it will never become invalid until they drop it.

It's important to understand this means that any owned data always passes a 'static lifetime bound, but a reference to that owned data generally does not.

5. 🌟🌟

```

/* Make it work */
use std::fmt::Debug;

fn print_it<T: Debug + 'static>( input: T) {
    println!( "'static value passed in is: {:?}'", input );
}

fn print_it1( input: impl Debug + 'static ) {
    println!( "'static value passed in is: {:?}'", input );
}

fn print_it2<T: Debug + 'static>( input: &T) {
    println!( "'static value passed in is: {:?}'", input );
}

fn main() {
    // i is owned and contains no references, thus it's 'static:
    let i = 5;
    print_it(i);

    // oops, &i only has the lifetime defined by the scope of
    // main(), so it's not 'static:
    print_it(&i);

    print_it1(&i);

    // but this one WORKS !
    print_it2(&i);
}

```

6. 🌟🌟🌟


```

use std::fmt::Display;

fn main() {
    let mut string = "First".to_owned();

    string.push_str(string.to_uppercase().as_str());
    print_a(&string);
    print_b(&string);
    print_c(&string); // Compilation error
    print_d(&string); // Compilation error
    print_e(&string);
    print_f(&string);
    print_g(&string); // Compilation error
}

fn print_a<T: Display + 'static>(t: &T) {
    println!("{}", t);
}

fn print_b<T>(t: &T)
where
    T: Display + 'static,
{
    println!("{}", t);
}

fn print_c(t: &'static dyn Display) {
    println!("{}", t)
}

fn print_d(t: &'static impl Display) {
    println!("{}", t)
}

fn print_e(t: &(dyn Display + 'static)) {
    println!("{}", t)
}

fn print_f(t: &(impl Display + 'static)) {
    println!("{}", t)
}

fn print_g(t: &'static String) {
    println!("{}", t);
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Advance lifetime

Trait Bounds

Just like generic types can be bounded, lifetimes can also be bounded as below:

- `T: 'a` , all references in `T` must outlive the lifetime `'a`
- `T: Trait + 'a: T` must implement trait `Trait` and all references in `T` must outlive `'a`

Example

```
use std::fmt::Debug; // Trait to bound with.

#[derive(Debug)]
struct Ref<'a, T: 'a>(&'a T);
// `Ref` contains a reference to a generic type `T` that has
// an unknown lifetime `a`. `T` is bounded such that any
// *references* in `T` must outlive `a`. Additionally, the lifetime
// of `Ref` may not exceed `a`.

// A generic function which prints using the `Debug` trait.
fn print<T>(t: T) where
    T: Debug {
    println!("`print`: t is {:?}", t);
}

// Here a reference to `T` is taken where `T` implements
// `Debug` and all *references* in `T` outlive `a`. In
// addition, `a` must outlive the function.
fn print_ref<'a, T>(t: &'a T) where
    T: Debug + 'a {
    println!("`print_ref`: t is {:?}", t);
}

fn main() {
    let x = 7;
    let ref_x = Ref(&x);

    print_ref(&ref_x);
    print(ref_x);
}
```

1. 🌟

```

/* Annotate struct with lifetime:
1. `r` and `s` must have different lifetimes
2. lifetime of `s` is bigger than that of `r`
*/
struct DoubleRef<T> {
    r: &T,
    s: &T
}
fn main() {
    println!("Success!")
}

```

2. 🌟🌟

```

/* Adding trait bounds to make it work */
struct ImportantExcerpt<'a> {
    part: &'a str,
}

impl<'a, 'b> ImportantExcerpt<'a> {
    fn announce_and_return_part(&'a self, announcement: &'b str) -> &'b str {
        println!("Attention please: {}", announcement);
        self.part
    }
}

fn main() {
    println!("Success!")
}

```

3. 🌟🌟

```

/* Adding trait bounds to make it work */
fn f<'a, 'b>(x: &'a i32, mut y: &'b i32) {
    y = x;
    let r: &'b &'a i32 = &&0;
}

fn main() {
    println!("Success!")
}

```

HRTB(Higher-ranked trait bounds)

Type bounds may be higher ranked over lifetimes. These bounds specify a bound is true for all lifetimes. For example, a bound such as `for<'a> &'a T: PartialEq<i32>` would require an implementation like:

```
impl<'a> PartialEq<i32> for &'a T {
    // ...
}
```

and could then be used to compare a `&'a T` with any lifetime to an `i32`.

Only a higher-ranked bound can be used here, because the lifetime of the reference is shorter than any possible lifetime parameter on the function.

4. 🌟🌟🌟

```
/* Adding HRTB to make it work! */
fn call_on_ref_zero<'a, F>(f: F) where F: Fn(&'a i32) {
    let zero = 0;
    f(&zero);
}

fn main() {
    println!("Success!");
}
```

NLL (Non-Lexical Lifetime)

Before explaining NLL, let's see some code first:

```
fn main() {
    let mut s = String::from("hello");

    let r1 = &s;
    let r2 = &s;
    println!("{}", r1, r2);

    let r3 = &mut s;
    println!("{}", r3);
}
```

Based on our current knowledge, this code will cause an error due to violating the borrowing rules in Rust.

But if you `cargo run` it, then everything will be ok, so what's going on here?

The ability of the compiler to tell that a reference is no longer used at a point before the end of the scope, is called **Non-Lexical Lifetimes (NLL)** for short).

With this ability the compiler knows when is the last time that a reference is used and optimizing the borrowing rules based on this knowledge.

```

let mut u = 0i32;
let mut v = 1i32;
let mut w = 2i32;

// lifetime of `a` =  $\alpha \cup \beta \cup \gamma$ 
let mut a = &mut u;      // --+  $\alpha$ . lifetime of `&mut u`  --+ lexical "lifetime" of
`&mut u`, `&mut u`, `&mut w` and `a`
use(a);                  //   |
*a = 3; // <-----+   |
...                       //   |
a = &mut v;               // --+  $\beta$ . lifetime of `&mut v`  |
use(a);                  //   |
*a = 4; // <-----+   |
...                       //   |
a = &mut w;               // --+  $\gamma$ . lifetime of `&mut w` |
use(a);                  //   |
*a = 5; // <-----+ <-----+

```

Reborrow

After learning NLL, we can easily understand reborrow now.

Example

```

#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

impl Point {
    fn move_to(&mut self, x: i32, y: i32) {
        self.x = x;
        self.y = y;
    }
}

fn main() {
    let mut p = Point { x: 0, y: 0 };
    let r = &mut p;
    // Here comes the reborrow
    let rr: &Point = &*r;

    println!("{:?}", rr); // Reborrow ends here, NLL introduced

    // Reborrow is over, we can continue using `r` now
    r.move_to(10, 10);
    println!("{:?}", r);
}

```

5. 🌟🌟

```

/* Make it work by reordering some code */
fn main() {
    let mut data = 10;
    let ref1 = &mut data;
    let ref2 = &mut *ref1;

    *ref1 += 1;
    *ref2 += 2;

    println!("{}", data);
}

```

Unbound lifetime

See more info in [Nomicon - Unbounded Lifetimes](#).

More elision rules

```
impl<'a> Reader for BufReader<'a> {  
    // 'a is not used in the following methods  
}
```

// can be written as :

```
impl Reader for BufReader<'_> {  
  
}
```

// Rust 2015

```
struct Ref<'a, T: 'a> {  
    field: &'a T  
}
```

// Rust 2018

```
struct Ref<'a, T> {  
    field: &'a T  
}
```

A difficult exercise

6. 🌟🌟🌟🌟

```

/* Make it work */
struct Interface<'a> {
    manager: &'a mut Manager<'a>
}

impl<'a> Interface<'a> {
    pub fn noop(self) {
        println!("interface consumed");
    }
}

struct Manager<'a> {
    text: &'a str
}

struct List<'a> {
    manager: Manager<'a>,
}

impl<'a> List<'a> {
    pub fn get_interface(&'a mut self) -> Interface {
        Interface {
            manager: &mut self.manager
        }
    }
}

fn main() {
    let mut list = List {
        manager: Manager {
            text: "hello"
        }
    };

    list.get_interface().noop();

    println!("Interface should be dropped here and the borrow released");

    use_list(&list);
}

fn use_list(list: &List) {
    println!("{}", list.manager.text);
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

Functional programing

Learning resources:

- English: [Rust Book 13](#)
- 繁體中文: [Rust 繁體中文 - 學習資源](#)

Closure

Closures can capture the enclosing environments. For example we can capture the `x` variable :

```
fn main() {  
    let x = 1;  
    let closure = |val| val + x;  
    assert_eq!(closure(2), 3);  
}
```

From the syntax, we can see that closures are very convenient for on the fly usage. Unlike functions, both the input and return types of a closure can be inferred by the compiler.

```
fn main() {  
    // Increment via closures and functions.  
    fn function(i: i32) -> i32 { i + 1 }  
  
    // Closures are anonymous, here we are binding them to references  
    //  
    // These nameless functions are assigned to appropriately named variables.  
    let closure_annotated = |i: i32| -> i32 { i + 1 };  
    let closure_inferred = |i| i + 1 ;  
  
    let i = 1;  
    // Call the function and closures.  
    println!("function: {}", function(i));  
    println!("closure_annotated: {}", closure_annotated(i));  
    println!("closure_inferred: {}", closure_inferred(i));  
  
    // A closure taking no arguments which returns an `i32`.  
    // The return type is inferred.  
    let one = || 1;  
    println!("closure returning one: {}", one());  
}
```

Capturing

Closures can capture variables by borrowing or moving. But they prefer to capture by borrowing and only go lower when required:

- By reference: `&T`
- By mutable reference: `&mut T`
- By value: `T`

1. 🌟

```
/* Make it work with least amount of changes*/
fn main() {
    let color = String::from("green");

    let print = move || println!("`color`: {}", color);

    print();
    print();

    // `color` can be borrowed immutably again, because the closure only holds
    // an immutable reference to `color`.
    let _reborrow = &color;

    println!("{}",color);
}
```

2. 🌟🌟

```
/* Make it work
- Dont use `_reborrow` and `_count_reborrowed`
- Dont modify `assert_eq`
*/
fn main() {
    let mut count = 0;

    let mut inc = || {
        count += 1;
        println!("`count`: {}", count);
    };

    inc();

    let _reborrow = &count;

    inc();

    // The closure no longer needs to borrow `&mut count`. Therefore, it is
    // possible to reborrow without an error
    let _count_reborrowed = &mut count;

    assert_eq!(count, 0);
}
```

3. 🌟🌟

```

/* Make it work in two ways, none of them is to remove `take(movable)` away from the */
fn main() {
    let movable = Box::new(3);

    let consume = || {
        println!("`movable`: {:?}", movable);
        take(movable);
    };

    consume();
    consume();
}

fn take<T>( v: T) {}

```

For comparison, the following code has no error:

```

fn main() {
    let movable = Box::new(3);

    let consume = move || {
        println!("`movable`: {:?}", movable);
    };

    consume();
    consume();
}

```

Type inferred

The following four closures has no difference in input and return types.

```

fn add_one_v1 (x: u32) -> u32 { x + 1 }
let add_one_v2 = |x: u32| -> u32 { x + 1 };
let add_one_v3 = |x|           { x + 1 };
let add_one_v4 = |x|           x + 1 ;

```

4. 🌟

```

fn main() {
    let example_closure = |x| x;

    let s = example_closure(String::from("hello"));

    /* Make it work, only change the following line */
    let n = example_closure(5);
}

```

Fn, FnMut, FnOnce

When taking a closure as an input parameter, the closure's complete type must be annotated using one of the following traits:

- Fn: the closure uses the captured value by reference (&T)
- FnMut: the closure uses the captured value by mutable reference (&mut T)
- FnOnce: the closure uses the captured value by value (T)

5. 🌟🌟

```
/* Make it work by changing the trait bound, in two ways*/
fn fn_once<F>(func: F)
where
    F: FnOnce(usize) -> bool,
{
    println!("{}", func(3));
    println!("{}", func(4));
}

fn main() {
    let x = vec![1, 2, 3];
    fn_once(|z| {z == x.len()})
}
```

6. 🌟🌟

```
fn main() {
    let mut s = String::new();

    let update_string = |str| s.push_str(str);

    exec(update_string);

    println!("{:?}", s);
}

/* Fill in the blank */
fn exec<'a, F: __>(mut f: F) {
    f("hello")
}
```

Which trait does the compiler prefer to use?

- Fn: the closure uses the captured value by reference (&T)
- FnMut: the closure uses the captured value by mutable reference (&mut T)
- FnOnce: the closure uses the captured value by value (T)

On a variable-by-variable basis, the compiler will capture variables in the least restrictive manner possible.

For instance, consider a parameter annotated as `FnOnce`. This specifies that the closure may capture by `&T`, `&mut T`, or `T`, but the compiler will ultimately choose based on how the captured variables are used in the closure. Which trait to use is determined by what the closure does with captured value.

This is because if a move is possible, then any type of borrow should also be possible. Note that the reverse is not true. If the parameter is annotated as `Fn`, then capturing variables by `&mut T` or `T` are not allowed.

7. 🌟🌟

```

/* Fill in the blank */

// A function which takes a closure as an argument and calls it.
// <F> denotes that F is a "Generic type parameter"
fn apply<F>(f: F) where
    // The closure takes no input and returns nothing.
    F: __ {

    f();
}

// A function which takes a closure and returns an `i32`.
fn apply_to_3<F>(f: F) -> i32 where
    // The closure takes an `i32` and returns an `i32`.
    F: Fn(i32) -> i32 {

    f(3)
}

fn main() {
    use std::mem;

    let greeting = "hello";
    // A non-copy type.
    // `to_owned` creates owned data from borrowed one
    let mut farewell = "goodbye".to_owned();

    // Capture 2 variables: `greeting` by reference and
    // `farewell` by value.
    let diary = || {
        // `greeting` is by reference: requires `Fn`.
        println!("I said {}. ", greeting);

        // Mutation forces `farewell` to be captured by
        // mutable reference. Now requires `FnMut`.
        farewell.push_str("!!!");
        println!("Then I screamed {}. ", farewell);
        println!("Now I can sleep. zzzzz");

        // Manually calling drop forces `farewell` to
        // be captured by value. Now requires `FnOnce`.
        mem::drop(farewell);
    };

    // Call the function which applies the closure.
    apply(diary);

    // `double` satisfies `apply_to_3`'s trait bound
    let double = |x| 2 * x;

    println!("3 doubled: {}", apply_to_3(double));
}

```

Move closures may still implement `Fn` or `FnMut`, even though they capture variables by move. This is because the traits implemented by a closure type are determined by what the closure

does with captured values, not how it captures them. The `move` keyword only specifies the latter.

```
fn main() {
    let s = String::new();

    let update_string = move || println!("{}",s);

    exec(update_string);
}

fn exec<F: FnOnce()>(f: F) {
    f()
}
```

The following code also has no error:

```
fn main() {
    let s = String::new();

    let update_string = move || println!("{}",s);

    exec(update_string);
}

fn exec<F: Fn()>(f: F) {
    f()
}
```

8. 🌟🌟

```
/* Fill in the blank */
fn main() {
    let mut s = String::new();

    let update_string = |str| -> String {s.push_str(str); s };

    exec(update_string);
}

fn exec<'a, F: __>(mut f: F) {
    f("hello");
}
```

Input functions

Since closure can be used as arguments, you might wonder can we use functions as arguments too? And indeed we can.

9. 🌟🌟

```
|  
/* Implement `call_me` to make it work */  
fn call_me {  
    f();  
}  
  
fn function() {  
    println!("I'm a function!");  
}  
  
fn main() {  
    let closure = || println!("I'm a closure!");  
  
    call_me(closure);  
    call_me(function);  
}
```

Closure as return types

Returning a closure is much harder than you may have thought of.

10. 🌟🌟

```
/* Fill in the blank using two approaches,  
and fix the error */  
fn create_fn() -> __ {  
    let num = 5;  
  
    // How does the following closure capture the environment variable `num`  
    // &T, &mut T, T ?  
    |x| x + num  
}  
  
fn main() {  
    let fn_plain = create_fn();  
    fn_plain(1);  
}
```

11. 🌟🌟

```
/* Fill in the blank and fix the error*/  
fn factory(x:i32) -> __ {  
  
    let num = 5;  
  
    if x > 1{  
        move |x| x + num  
    } else {  
        move |x| x + num  
    }  
}
```

Closure in structs

Example

```

struct Cacher<T,E>
where
    T: Fn(E) -> E,
    E: Copy
{
    query: T,
    value: Option<E>,
}

impl<T,E> Cacher<T,E>
where
    T: Fn(E) -> E,
    E: Copy
{
    fn new(query: T) -> Cacher<T,E> {
        Cacher {
            query,
            value: None,
        }
    }

    fn value(&mut self, arg: E) -> E {
        match self.value {
            Some(v) => v,
            None => {
                let v = (self.query)(arg);
                self.value = Some(v);
                v
            }
        }
    }
}

fn main() {

}

#[test]
fn call_with_different_values() {
    let mut c = Cacher::new(|a| a);

    let v1 = c.value(1);
    let v2 = c.value(2);

    assert_eq!(v2, 1);
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it
:)

Iterator

The iterator pattern allows us to perform some tasks on a sequence of items in turn. An iterator is responsible for the logic of iterating over each item and determining when the sequence has finished.

for and iterator

```
fn main() {  
    let v = vec![1, 2, 3];  
    for x in v {  
        println!("{}",x)  
    }  
}
```

In the code above, You may consider `for` as a simple loop, but actually it is iterating over a iterator.

By default `for` will apply the `into_iter` to the collection, and change it into a iterator. As a result, the following code is equivalent to previous one:

```
fn main() {  
    let v = vec![1, 2, 3];  
    for x in v.into_iter() {  
        println!("{}",x)  
    }  
}
```

1. 🌟

```
/* Refactoring the following code using iterators */  
fn main() {  
    let arr = [0; 10];  
    for i in 0..arr.len() {  
        println!("{}",arr[i]);  
    }  
}
```

2. 🌟 One of the easiest ways to create an iterator is to use the range notion: `a..b`.

```

/* Fill in the blank */
fn main() {
    let mut v = Vec::new();
    for n in __ {
        v.push(n);
    }

    assert_eq!(v.len(), 100);
}

```

next method

All iterators implement a trait named `Iterator` that is defined in the standard library:

```

pub trait Iterator {
    type Item;

    fn next(&mut self) -> Option<Self::Item>;

    // Methods with default implementations elided
}

```

And we can call the `next` method on iterators directly.

3. 🌟🌟

```

/* Fill the blanks and fix the errors.
Using two ways if possible */
fn main() {
    let v1 = vec![1, 2];

    assert_eq!(v1.next(), __);
    assert_eq!(v1.next(), __);
    assert_eq!(v1.next(), __);
}

```

into_iter, iter and iter_mut

In the previous section, we have mentioned that `for` will apply the `into_iter` to the collection, and change it into a iterator. However, this is not the only way to convert collections into iterators.

`into_iter`, `iter`, `iter_mut`, all of them can convert a collection into iterator, but in different ways.

- `into_iter` consumes the collection, once the collection has been consumed, it is no longer available for reuse, because its ownership has been moved within the loop.
- `iter`, this borrows each element of the collection through each iteration, thus leaving the collection untouched and available for reuse after the loop
- `iter_mut`, this mutably borrows each element of the collection, allowing for the collection to be modified in place.

4. 🌟

```
/* Make it work */
fn main() {
    let arr = vec![0; 10];
    for i in arr {
        println!("{}", i);
    }

    println!("{:?}", arr);
}
```

5. 🌟

```
/* Fill in the blank */
fn main() {
    let mut names = vec!["Bob", "Frank", "Ferris"];

    for name in names.__{
        *name = match name {
            &mut "Ferris" => "There is a rustacean among us!",
            _ => "Hello",
        }
    }

    println!("names: {:?}", names);
}
```

6. 🌟🌟

```
/* Fill in the blank */
fn main() {
    let mut values = vec![1, 2, 3];
    let mut values_iter = values.__;

    if let Some(v) = values_iter.__{
        --
    }

    assert_eq!(values, vec![0, 2, 3]);
}
```

Creating our own iterator

We can not only create iterators from collection's types, but also can create iterators by implementing the `Iterator` trait on our own types.

Example

```
struct Counter {
    count: u32,
}

impl Counter {
    fn new() -> Counter {
        Counter { count: 0 }
    }
}

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        if self.count < 5 {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

fn main() {
    let mut counter = Counter::new();

    assert_eq!(counter.next(), Some(1));
    assert_eq!(counter.next(), Some(2));
    assert_eq!(counter.next(), Some(3));
    assert_eq!(counter.next(), Some(4));
    assert_eq!(counter.next(), Some(5));
    assert_eq!(counter.next(), None);
}
```

7. 🌟🌟🌟

```

struct Fibonacci {
    curr: u32,
    next: u32,
}

// Implement `Iterator` for `Fibonacci`.
// The `Iterator` trait only requires a method to be defined for the `next` element.
impl Iterator for Fibonacci {
    // We can refer to this type using Self::Item
    type Item = u32;

    /* Implement next method */
    fn next(&mut self)

}

// Returns a Fibonacci sequence generator
fn fibonacci() -> Fibonacci {
    Fibonacci { curr: 0, next: 1 }
}

fn main() {
    let mut fib = fibonacci();
    assert_eq!(fib.next(), Some(1));
    assert_eq!(fib.next(), Some(1));
    assert_eq!(fib.next(), Some(2));
    assert_eq!(fib.next(), Some(3));
    assert_eq!(fib.next(), Some(5));
}

```

Methods that Consume the Iterator

The `Iterator` trait has a number of methods with default implementations provided by the standard library.

Consuming adaptors

Some of these methods call the method `next` to use up the iterator, so they are called *consuming adaptors*.

8. 🌟🌟


```

/* Fill in the blank and fix the errors */
fn main() {
    let v1 = vec![1, 2, 3];

    let v1_iter = v1.iter();

    // The sum method will take the ownership of the iterator and iterates through it
    let total = v1_iter.sum();

    assert_eq!(total, 6);

    println!("{:?}", {v1, v1_iter});
}

```

Collect

Other than converting a collection into an iterator, we can also `collect` the result values into a collection, `collect` will consume the iterator.

9. 🌟🌟

```

/* Make it work */
use std::collections::HashMap;
fn main() {
    let names = [("sunface",18), ("sunfei",18)];
    let folks: HashMap<_, _> = names.into_iter().collect();

    println!("{:?}", folks);

    let v1: Vec<i32> = vec![1, 2, 3];

    let v2 = v1.iter().collect();

    assert_eq!(v2, vec![1, 2, 3]);
}

```

Iterator adaptors

Methods allowing you to change one iterator into another iterator are known as *iterator adaptors*. You can chain multiple iterator adaptors to perform complex actions in a readable way.

But because **all iterators are lazy**, you have to call one of the consuming adapters to get results from calls to iterator adapters.

10. 🌟🌟

```

/* Fill in the blanks */
fn main() {
    let v1: Vec<i32> = vec![1, 2, 3];

    let v2: Vec<_> = v1.iter().__.__;

    assert_eq!(v2, vec![2, 3, 4]);
}

```

11. 🌟🌟

```

/* Fill in the blanks */
use std::collections::HashMap;
fn main() {
    let names = ["sunface", "sunfei"];
    let ages = [18, 18];
    let folks: HashMap<_, _> = names.into_iter().__.collect();

    println!("{:?}", folks);
}

```

Using closures in iterator adaptors

12. 🌟🌟

```

/* Fill in the blanks */
#[derive(PartialEq, Debug)]
struct Shoe {
    size: u32,
    style: String,
}

fn shoes_in_size(shoes: Vec<Shoe>, shoe_size: u32) -> Vec<Shoe> {
    shoes.into_iter().__.collect()
}

fn main() {
    let shoes = vec![
        Shoe {
            size: 10,
            style: String::from("sneaker"),
        },
        Shoe {
            size: 13,
            style: String::from("sandal"),
        },
        Shoe {
            size: 10,
            style: String::from("boot"),
        },
    ];

    let in_my_size = shoes_in_size(shoes, 10);

    assert_eq!(
        in_my_size,
        vec![
            Shoe {
                size: 10,
                style: String::from("sneaker")
            },
            Shoe {
                size: 10,
                style: String::from("boot")
            },
        ]
    );
}

```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)

newtype and Sized

Newtype

The orphan rule tells us that we are allowed to implement a trait on a type as long as either the trait or the type are local to our crate.

The **newtype pattern** can help us get around this restriction, which involves creating a new type in a **tuple struct**.

1. 🌟

```
use std::fmt;

/* Define the Wrapper type */
--;

// Display is an external trait
impl fmt::Display for Wrapper {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}

fn main() {
    // Vec is an external type, so you cannot implement Display trait on Vec type
    let w = Wrapper(vec![String::from("hello"), String::from("world")]);
    println!("w = {}", w);
}
```

2. 🌟 Hide the methods of the original type.

```
/* Make it workd */
struct Meters(u32);

fn main() {
    let i: u32 = 2;
    assert_eq!(i.pow(2), 4);

    let n = Meters(i);
    // The `pow` method is defined on `u32` type, we can't directly call it
    assert_eq!(n.pow(2), 4);
}
```

3. 🌟🌟 The newtype idiom gives compile time guarantees that the right type of value is supplied to a program.

```

/* Make it work */
struct Years(i64);

struct Days(i64);

impl Years {
    pub fn to_days(&self) -> Days {
        Days(self.0 * 365)
    }
}

impl Days {
    pub fn to_years(&self) -> Years {
        Years(self.0 / 365)
    }
}

// An age verification function that checks age in years, must be given a value of t.
fn old_enough(age: &Years) -> bool {
    age.0 >= 18
}

fn main() {
    let age = Years(5);
    let age_days = age.to_days();
    println!("Old enough {}", old_enough(&age));
    println!("Old enough {}", old_enough(&age_days));
}

```

4. 🌟🌟🌟

```

use std::ops::Add;
use std::fmt::{self, format};

struct Meters(u32);
impl fmt::Display for Meters {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "There are still {} meters left", self.0)
    }
}

impl Add for Meters {
    type Output = Self;

    fn add(self, other: Meters) -> Self {
        Self(self.0 + other.0)
    }
}

fn main() {
    let d = calculate_distance(Meters(10), Meters(20));
    assert_eq!(format!("{}",d), "There are still 30 meters left");
}

/* Implement calculate_distance */
fn calculate_distance

```

Type alias

Type alias is important to improve the readability of our code.

```

type Thunk = Box<dyn Fn() + Send + 'static>;

let f: Thunk = Box::new(|| println!("hi"));

fn takes_long_type(f: Thunk) {
    // --snip--
}

fn returns_long_type() -> Thunk {
    // --snip--
}

type Result<T> = std::result::Result<T, std::io::Error>;

```

And Unlike newtype, type alias don't create new types, so the following code is valid:

```

type Meters = u32;

let x: u32 = 5;
let y: Meters = 5;

println!("x + y = {}", x + y);

```

5. 🌟

```

enum VeryVerboseEnumOfThingsToDoWithNumbers {
    Add,
    Subtract,
}

/* Fill in the blank */
--

fn main() {
    // We can refer to each variant via its alias, not its long and inconvenient
    // name.
    let x = Operations::Add;
}

```

6. 🌟🌟 There are a few preserved aliases in Rust, one of which can be used in `impl` blocks.

```

enum VeryVerboseEnumOfThingsToDoWithNumbers {
    Add,
    Subtract,
}

impl VeryVerboseEnumOfThingsToDoWithNumbers {
    fn run(&self, x: i32, y: i32) -> i32 {
        match self {
            __::Add => x + y,
            __::Subtract => x - y,
        }
    }
}

```

DST and unsized type

These concepts are complicated, so we are not going to explain here, but you can find them in [The Book](#).

7. 🌟🌟🌟 Array with dynamic length is a Dynamic Sized Type (DST), we can't directly use it

```

/* Make it work with const generics */
fn my_function(n: usize) -> [u32; usize] {
    [123; n]
}

fn main() {
    let arr = my_function();
    println!("{:?}",arr);
}

```

8. 🌟🌟🌟 Slice is unsized type, but the reference of slice is not.

```

/* Make it work with slice references */
fn main() {
    let s: str = "Hello there!";

    let arr: [u8] = [1, 2, 3];
}

```

9. 🌟🌟🌟 Trait is also an unsized type

```

/* Make it work in two ways */
use std::fmt::Display;
fn foobar(thing: Display) {}

fn main() {
}

```


Smart pointers

Box

1. 🌟

```
// Make it work
fn main() {
    // Create a new box `b` that contains the integer 5
    assert_eq!(*b, 5);

    println!("Success!");
}
```

2. 🌟

```
// Make it work
fn main() {
    let b = Box::new("Hello");
    print_boxed_string(b);
}

fn print_boxed_string(b : _) {
    println!("{}", b);
}
```

3. 🌟

```
// Make it work
fn main() {
    let b1 = Box::new(5);
    let b2 = b1;
    assert_eq!(_, 5);

    println!("Success!");
}
```

4. 🌟

```
// Make it work
fn main() {
    // Create a box `b` with an array [1, 2, 3, 4, 5]
    // Print each integer in `b`
}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it

Deref

Drop

Rc and Arc

Cell and RefCell

Weak and Circle reference

Self referential

Threads

Basic using

Message passing

Sync

Atomic

Send and Sync

Global variables

Errors

Unsafe todo

Inline assembly

Rust provides support for inline assembly via the `asm!` macro. It can be used to embed handwritten assembly in the assembly output generated by the compiler. Generally this should not be necessary, but might be where the required performance or timing cannot be otherwise achieved. Accessing low level hardware primitives, e.g. in kernel code, may also demand this functionality.

Note: the examples here are given in x86/x86-64 assembly, but other architectures are also supported.

Inline assembly is currently supported on the following architectures:

- x86 and x86-64
- ARM
- AArch64
- RISC-V

Basic usage

Let us start with the simplest possible example:

```
use std::arch::asm;

unsafe {
    asm!("nop");
}
```

This will insert a NOP (no operation) instruction into the assembly generated by the compiler. Note that all `asm!` invocations have to be inside an `unsafe` block, as they could insert arbitrary instructions and break various invariants. The instructions to be inserted are listed in the first argument of the `asm!` macro as a string literal.

Inputs and outputs

Now inserting an instruction that does nothing is rather boring. Let us do something that actually acts on data:

```
use std::arch::asm;

let x: u64;
unsafe {
    asm!("mov {}, 5", out(reg) x);
}
assert_eq!(x, 5);
```

This will write the value `5` into the `u64` variable `x`. You can see that the string literal we use to specify instructions is actually a template string. It is governed by the same rules as Rust [format strings](#). The arguments that are inserted into the template however look a bit different than you may be familiar with. First we need to specify if the variable is an input or an output of the inline assembly. In this case it is an output. We declared this by writing `out`. We also need to specify in what kind of register the assembly expects the variable. In this case we put it in an arbitrary general purpose register by specifying `reg`. The compiler will choose an appropriate register to insert into the template and will read the variable from there after the inline assembly finishes executing.

Let us see another example that also uses an input:

```
use std::arch::asm;

let i: u64 = 3;
let o: u64;
unsafe {
    asm!(
        "mov {0}, {1}",
        "add {0}, 5",
        out(reg) o,
        in(reg) i,
    );
}
assert_eq!(o, 8);
```

This will add `5` to the input in variable `i` and write the result to variable `o`. The particular way this assembly does this is first copying the value from `i` to the output, and then adding `5` to it.

The example shows a few things:

First, we can see that `asm!` allows multiple template string arguments; each one is treated as a separate line of assembly code, as if they were all joined together with newlines between them.

This makes it easy to format assembly code.

Second, we can see that inputs are declared by writing `in` instead of `out`.

Third, we can see that we can specify an argument number, or name as in any format string. For inline assembly templates this is particularly useful as arguments are often used more than once. For more complex inline assembly using this facility is generally recommended, as it improves readability, and allows reordering instructions without changing the argument order.

We can further refine the above example to avoid the `mov` instruction:

```
use std::arch::asm;

let mut x: u64 = 3;
unsafe {
    asm!("add {0}, 5", inout(reg) x);
}
assert_eq!(x, 8);
```

We can see that `inout` is used to specify an argument that is both input and output. This is different from specifying an input and output separately in that it is guaranteed to assign both to the same register.

It is also possible to specify different variables for the input and output parts of an `inout` operand:

```
use std::arch::asm;

let x: u64 = 3;
let y: u64;
unsafe {
    asm!("add {0}, 5", inout(reg) x => y);
}
assert_eq!(y, 8);
```

Late output operands

The Rust compiler is conservative with its allocation of operands. It is assumed that an `out` can be written at any time, and can therefore not share its location with any other argument. However, to guarantee optimal performance it is important to use as few registers as possible, so they won't have to be saved and reloaded around the inline assembly block. To achieve this Rust provides a `lateout` specifier. This can be used on any output that is written only after all inputs have been consumed. There is also a `inlateout` variant of this specifier.

Here is an example where `inlateout` *cannot* be used:

```
use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
let c: u64 = 4;
unsafe {
    asm!(
        "add {0}, {1}",
        "add {0}, {2}",
        inout(reg) a,
        in(reg) b,
        in(reg) c,
    );
}
assert_eq!(a, 12);
```

Here the compiler is free to allocate the same register for inputs `b` and `c` since it knows they have the same value. However it must allocate a separate register for `a` since it uses `inout` and not `inlateout`. If `inlateout` was used, then `a` and `c` could be allocated to the same register, in which case the first instruction to overwrite the value of `c` and cause the assembly code to produce the wrong result.

However the following example can use `inlateout` since the output is only modified after all input registers have been read:

```
use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
unsafe {
    asm!("add {0}, {1}", inlateout(reg) a, in(reg) b);
}
assert_eq!(a, 8);
```

As you can see, this assembly fragment will still work correctly if `a` and `b` are assigned to the same register.

Explicit register operands

Some instructions require that the operands be in a specific register. Therefore, Rust inline assembly provides some more specific constraint specifiers. While `reg` is generally available on any architecture, explicit registers are highly architecture specific. E.g. for x86 the general

purpose registers `eax`, `ebx`, `ecx`, `edx`, `ebp`, `esi`, and `edi` among others can be addressed by their name.

```
use std::arch::asm;

let cmd = 0xd1;
unsafe {
    asm!("out 0x64, eax", in("eax") cmd);
}
```

In this example we call the `out` instruction to output the content of the `cmd` variable to port `0x64`. Since the `out` instruction only accepts `eax` (and its sub registers) as operand we had to use the `eax` constraint specifier.

Note: unlike other operand types, explicit register operands cannot be used in the template string: you can't use `{}` and should write the register name directly instead. Also, they must appear at the end of the operand list after all other operand types.

Consider this example which uses the x86 `mul` instruction:

```
use std::arch::asm;

fn mul(a: u64, b: u64) -> u128 {
    let lo: u64;
    let hi: u64;

    unsafe {
        asm!(
            // The x86 mul instruction takes rax as an implicit input and writes
            // the 128-bit result of the multiplication to rax:rdx.
            "mul {}",
            in(reg) a,
            inlateout("rax") b => lo,
            lateout("rdx") hi
        );
    }

    ((hi as u128) << 64) + lo as u128
}
```

This uses the `mul` instruction to multiply two 64-bit inputs with a 128-bit result. The only explicit operand is a register, that we fill from the variable `a`. The second operand is implicit, and must be the `rax` register, which we fill from the variable `b`. The lower 64 bits of the result are stored in `rax` from which we fill the variable `lo`. The higher 64 bits are stored in `rdx` from which we fill the variable `hi`.

Clobbered registers

In many cases inline assembly will modify state that is not needed as an output. Usually this is either because we have to use a scratch register in the assembly or because instructions modify state that we don't need to further examine. This state is generally referred to as being "clobbered". We need to tell the compiler about this since it may need to save and restore this state around the inline assembly block.

```
use core::arch::asm;

fn main() {
    // three entries of four bytes each
    let mut name_buf = [0_u8; 12];
    // String is stored as ascii in ebx, edx, ecx in order
    // Because ebx is reserved, we get a scratch register and move from
    // ebx into it in the asm. The asm needs to preserve the value of
    // that register though, so it is pushed and popped around the main asm
    // (in 64 bit mode for 64 bit processors, 32 bit processors would use ebx)

    unsafe {
        asm!(
            "push rbx",
            "cpuid",
            "mov [{0}], ebx",
            "mov [{0} + 4], edx",
            "mov [{0} + 8], ecx",
            "pop rbx",
            // We use a pointer to an array for storing the values to simplify
            // the Rust code at the cost of a couple more asm instructions
            // This is more explicit with how the asm works however, as opposed
            // to explicit register outputs such as `out("ecx") val`
            // The *pointer itself* is only an input even though it's written
            behind
            in(reg) name_buf.as_mut_ptr(),
            // select cpuid 0, also specify eax as clobbered
            inout("eax") 0 => _,
            // cpuid clobbers these registers too
            out("ecx") _,
            out("edx") _,
        );
    }

    let name = core::str::from_utf8(&name_buf).unwrap();
    println!("CPU Manufacturer ID: {}", name);
}
```

In the example above we use the `cpuid` instruction to read the CPU manufacturer ID. This instruction writes to `eax` with the maximum supported `cpuid` argument and `ebx`, `ecx`, and `edx` with the CPU manufacturer ID as ASCII bytes in that order.

Even though `eax` is never read we still need to tell the compiler that the register has been modified so that the compiler can save any values that were in these registers before the asm. This is done by declaring it as an output but with `_` instead of a variable name, which indicates that the output value is to be discarded.

This code also works around the limitation that `ebx` is a reserved register by LLVM. That means that LLVM assumes that it has full control over the register and it must be restored to its original state before exiting the asm block, so it cannot be used as an output. To work around this we save the register via `push`, read from `ebx` inside the asm block into a temporary register allocated with `out(reg)` and then restoring `ebx` to its original state via `pop`. The `push` and `pop` use the full 64-bit `rbx` version of the register to ensure that the entire register is saved. On 32 bit targets the code would instead use `ebx` in the `push / pop`.

This can also be used with a general register class (e.g. `reg`) to obtain a scratch register for use inside the asm code:

```
use std::arch::asm;

// Multiply x by 6 using shifts and adds
let mut x: u64 = 4;
unsafe {
    asm!(
        "mov {tmp}, {x}",
        "shl {tmp}, 1",
        "shl {x}, 2",
        "add {x}, {tmp}",
        x = inout(reg) x,
        tmp = out(reg) _,
    );
}
assert_eq!(x, 4 * 6);
```

Symbol operands and ABI clobbers

By default, `asm!` assumes that any register not specified as an output will have its contents preserved by the assembly code. The `clobber_abi` argument to `asm!` tells the compiler to automatically insert the necessary clobber operands according to the given calling convention ABI: any register which is not fully preserved in that ABI will be treated as clobbered. Multiple `clobber_abi` arguments may be provided and all clobbers from all specified ABIs will be inserted.


```

use std::arch::asm;

extern "C" fn foo(arg: i32) -> i32 {
    println!("arg = {}", arg);
    arg * 2
}

fn call_foo(arg: i32) -> i32 {
    unsafe {
        let result;
        asm!(
            "call *{}",
            // Function pointer to call
            in(reg) foo,
            // 1st argument in rdi
            in("rdi") arg,
            // Return value in rax
            out("rax") result,
            // Mark all registers which are not preserved by the "C" calling
            // convention as clobbered.
            clobber_abi("C"),
        );
        result
    }
}

```

Register template modifiers

In some cases, fine control is needed over the way a register name is formatted when inserted into the template string. This is needed when an architecture's assembly language has several names for the same register, each typically being a "view" over a subset of the register (e.g. the low 32 bits of a 64-bit register).

By default the compiler will always choose the name that refers to the full register size (e.g. `rax` on x86-64, `eax` on x86, etc).

This default can be overridden by using modifiers on the template string operands, just like you would with format strings:

```

use std::arch::asm;

let mut x: u16 = 0xab;

unsafe {
    asm!("mov {0:h}, {0:l}", inout(reg_abcd) x);
}

assert_eq!(x, 0xabab);

```

In this example, we use the `reg_abcd` register class to restrict the register allocator to the 4 legacy x86 registers (`ax`, `bx`, `cx`, `dx`) of which the first two bytes can be addressed independently.

Let us assume that the register allocator has chosen to allocate `x` in the `ax` register. The `h` modifier will emit the register name for the high byte of that register and the `l` modifier will emit the register name for the low byte. The asm code will therefore be expanded as `mov ah, al` which copies the low byte of the value into the high byte.

If you use a smaller data type (e.g. `u16`) with an operand and forget the use template modifiers, the compiler will emit a warning and suggest the correct modifier to use.

Memory address operands

Sometimes assembly instructions require operands passed via memory addresses/memory locations. You have to manually use the memory address syntax specified by the target architecture. For example, on x86/x86_64 using Intel assembly syntax, you should wrap inputs/outputs in `[]` to indicate they are memory operands:

```

use std::arch::asm;

fn load_fpu_control_word(control: u16) {
    unsafe {
        asm!("fldcw [{}]", in(reg) &control, options(nostack));
    }
}

```

Labels

Any reuse of a named label, local or otherwise, can result in an assembler or linker error or may cause other strange behavior. Reuse of a named label can happen in a variety of ways

including:

- explicitly: using a label more than once in one `asm!` block, or multiple times across blocks.
- implicitly via inlining: the compiler is allowed to instantiate multiple copies of an `asm!` block, for example when the function containing it is inlined in multiple places.
- implicitly via LTO: LTO can cause code from *other crates* to be placed in the same codegen unit, and so could bring in arbitrary labels.

As a consequence, you should only use GNU assembler **numeric local labels** inside inline assembly code. Defining symbols in assembly code may lead to assembler and/or linker errors due to duplicate symbol definitions.

Moreover, on x86 when using the default Intel syntax, due to [an LLVM bug](#), you shouldn't use labels exclusively made of 0 and 1 digits, e.g. 0, 11 or 101010, as they may end up being interpreted as binary values. Using `options(att_syntax)` will avoid any ambiguity, but that affects the syntax of the *entire* `asm!` block. (See [Options](#), below, for more on `options`.)

```
use std::arch::asm;

let mut a = 0;
unsafe {
    asm!(
        "mov {0}, 10",
        "2:",
        "sub {0}, 1",
        "cmp {0}, 3",
        "jle 2f",
        "jmp 2b",
        "2:",
        "add {0}, 2",
        out(reg) a
    );
}
assert_eq!(a, 5);
```

This will decrement the `{0}` register value from 10 to 3, then add 2 and store it in `a`.

This example shows a few things:

- First, that the same number can be used as a label multiple times in the same inline block.
- Second, that when a numeric label is used as a reference (as an instruction operand, for example), the suffixes “b” (“backward”) or “f” (“forward”) should be added to the numeric label. It will then refer to the nearest label defined by this number in this direction.

Options

By default, an inline assembly block is treated the same way as an external FFI function call with a custom calling convention: it may read/write memory, have observable side effects, etc. However, in many cases it is desirable to give the compiler more information about what the assembly code is actually doing so that it can optimize better.

Let's take our previous example of an `add` instruction:

```
use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
unsafe {
    asm!(
        "add {0}, {1}",
        inlateout(reg) a, in(reg) b,
        options(pure, nomem, nostack),
    );
}
assert_eq!(a, 8);
```

Options can be provided as an optional final argument to the `asm!` macro. We specified three options here:

- `pure` means that the asm code has no observable side effects and that its output depends only on its inputs. This allows the compiler optimizer to call the inline asm fewer times or even eliminate it entirely.
- `nomem` means that the asm code does not read or write to memory. By default the compiler will assume that inline assembly can read or write any memory address that is accessible to it (e.g. through a pointer passed as an operand, or a global).
- `nostack` means that the asm code does not push any data onto the stack. This allows the compiler to use optimizations such as the stack red zone on x86-64 to avoid stack pointer adjustments.

These allow the compiler to better optimize code using `asm!`, for example by eliminating pure `asm!` blocks whose outputs are not needed.

See the [reference](#) for the full list of available options and their effects.

macro

Tests

Write Tests

Benchmark

<https://doc.rust-lang.org/unstable-book/library-features/test.html>

Unit and Integration

Assertions

Async/Await

async and await!

Future

Pin and Unpin

Stream

Stand Library todo

String

Fighting with Compiler

Fighting with compiler is very common in our daily coding, especially for those unfamiliar with Rust.

This chapter will provide some exercises to help us avoid such cases to lower the steep learning curve.

Borrowing

1. 🌟🌟

```
// FIX the error without removing any code line
struct test {
    list: Vec<i32>,
    a: i32
}

impl test {
    pub fn new() -> Self {
        test { list:vec![1,2,3,4,5,6,7], a:0 }
    }

    pub fn run(&mut self) {
        for i in self.list.iter() {
            self.do_something(*i)
        }
    }

    pub fn do_something(&mut self, n: i32) {
        self.a = n;
    }
}

fn main() {}
```

You can find the solutions [here](#)(under the solutions path), but only use it when you need it :)
